
AI Governance Engineering

The Thesis & Body of Knowledge · v0.3 · September 2026

Jorge García Aibar

aigovernanceengineer.com

Licensed CC BY 4.0

Contents

The Thesis

00 · Preface

01 · The definition

02 · Why now

03 · Values and principles

04 · The stack (five layers)

05 · Patterns

06 · The role

07 · Maturity model

08 · Regulatory map

09 · Glossary

10 · Reading list

Contributors & signatories

Changelog

The AI Governance Engineering Thesis

Version 0.4.0 · 2026-09-19 · Jorge García Aibar and Aurélie Pols

AI governance engineering is the application of engineering practice — systems thinking, product thinking and code — to the governance of AI systems. It treats governance not as a document to be signed but as a system to be built, run and measured, with the same rigour engineers already apply to the models and agents it governs.

It is more than "AI governance plus a few scripts." It is a change in how the work is done. Legacy AI governance describes an AI system on paper and hopes the paper stays true. Engineering the governance means the policy is executable, the control runs in the pipeline, the evidence is produced as a by-product of the build, and the whole thing is judged by one test: did realised risk actually fall, and can a regulator or auditor read the proof? It is a capability, not a job title. Anyone close enough to the build can develop it. It is measured by realised risk reduction and by audit-ready evidence — never by how many frameworks appear on a slide.

The idea does not appear from nowhere. It inherits from a line of engineering movements that turned process into running systems — site reliability engineering, DevSecOps, policy-as-code and software supply-chain security — and, most directly, from GRC engineering, which over the last two years turned governance, risk and compliance into a product built with code, tested in CI/CD and shipping evidence through APIs [1]. AI governance now needs the same step-change, because the thing being governed — models that retrain, prompts that change, agents that act on their own — moves faster than any document can follow.

Fundamental problems with legacy AI governance

1. Governance written for systems that no longer exist. Legacy AI governance runs on PDF policies and spreadsheet inventories that describe an AI system as it was on the day it was reviewed. But models are retrained, prompts are rewritten and agents acquire new tools by the day. The artefact is stale before it is signed. It is no accident that the function still sits mostly with privacy, legal and IT and only 5% with security [3] — far from the pipeline where the system actually changes.

2. Point-in-time review of a continuously changing thing. Annual assessments and committee sign-offs assume a system that holds still long enough to be judged. Frontier models and autonomous agents do not. Gartner expects more than 40% of agentic AI projects to be cancelled by the end of 2027, citing inadequate risk controls among the causes [4], and predicts that by 2029 more than half of successful attacks on AI agents will exploit access-control weaknesses and prompt injection [5] — runtime failure modes that a once-a-year review is structurally blind to.

3. Governance as a gate at the end, not a property of the build. Governance arrives after the model is trained, as a checkpoint to clear before launch. Engineers experience it as a tax collected at the door, and nothing it produces is wired back into how the system is built. A policy that can only recommend cannot stop a bad release. An eval that can fail the build can. Governance placed at the end can only describe risk; governance built into the pipeline can prevent it.

4. Framework theatre. Mapping to NIST AI RMF or ISO/IEC 42001 becomes the end state instead of the starting point. A green mapping matrix is mistaken for a working control. Yet as of today no harmonised standard is cited in the EU's Official Journal, so even an ISO 42001 certificate confers no presumption of conformity with the AI Act [6]. Coverage is not assurance. A crosswalk proves you have read the framework, not that the control it points to actually fires; a green matrix over a broken control is "theatre with extra steps" [2].

5. No runtime data path. The registry does not know what is running. Most of the AI governance platform category "manages the program — inventories, assessments, framework mappings, evidence workflows — without any runtime data path" [7]. An independent read of the category's own leader points the same way: IBM's account of topping Gartner's first Magic Quadrant for AI Governance Platforms (2026) describes visibility into AI use cases, centralised AI asset inventory and lineage, and use-case onboarding — the program layer — not runtime enforcement [10]. So the three questions that define the discipline — what AI is running, what is it allowed to do, what evidence proves it — go unanswered, because nothing is connected to production. Meanwhile roughly one in eight reported AI breaches now involves an autonomous agent [8]: exactly the layer the paper registry cannot see.

Values

Eight affirmations. Each states what we build toward and, by naming it, what we build away from. Not all are new: values 1, 5 and 7 — governance as code, machine-readable evidence and measured risk reduction — are inherited

from GRC engineering; values 2 and 4 — evals that fail the build, and agent identity and scope — are what AI forces us to add. Chapter 03 expands each with an in-practice example and the anti-pattern it rejects.

1. Governance is code, not a document. A policy in a PDF is a statement of intent a human must remember and apply; a policy as code is a control that executes, versioned in a repository and enforced without anyone remembering to. The document describes the rule; the code is the rule, and only what runs can be measured.

2. Evals fail builds; reviews only recommend. A review produces a recommendation someone may act on, later, or not. An eval produces a verdict with consequences: the model or agent passed or failed a defined test, and a failure blocks the release. We prefer controls that bite.

3. Evidence comes from runtime, not from a point-in-time attestation. An attestation says a control was in place when someone looked. Runtime evidence shows it working continuously, emitted by the system as it runs — because an AI system changes between reviews, and evidence gathered once decays immediately.

4. Every agent carries its own identity and scope. An actor on shared credentials is ungovernable: you cannot attribute its actions, revoke its access precisely, or bound what it may do. Identity is the precondition of accountability; scope is the precondition of containment, and both are established before the actor is allowed to act.

5. Evidence is machine-readable or it is not evidence. Evidence a human must produce, format and file by hand cannot be queried, diffed or verified at speed. Machine-readable artefacts — OSCAL, structured eval results, signed logs — turn the audit into a query and feed continuous assurance instead of a one-off binder.

6. Tooling must be inspectable and composable. You cannot trust a verdict you cannot trace. Tooling whose reasoning and data path you can open — bought or built — lets you follow a decision to the rule that produced it and the evidence it emitted, and wire it into the pipeline you already run rather than export into someone else's.

7. Success is measured in realised risk reduction, not framework coverage. Mapping every control to a framework proves you have read it, not that any risk fell. We measure the thing itself: did the rate of the failure mode drop, did the blast radius shrink, did the incident get caught earlier? Coverage is an input; realised risk reduction is the outcome.

8. Governance is owned with engineering, not enforced from outside. Governance that sits apart and grants or denies passage is a bottleneck engineers route around. Owned jointly with engineering — built into the paved path, adopted because it is the easiest way to ship — it becomes part of how things are made, not a meeting one side dreads.

Principles

The values say what we prefer; these principles say what we commit to *do*. They are rules of action, not restatements of the preferences above.

Build the control at the earliest point it can block. Put every control where it can still stop the thing going wrong, and no later — in the repository, the build and the runtime, not in a review after the fact. The earliest enforceable point is the cheapest and the strongest, so that is where we put it.

Give every control teeth, or call it a signal. A control has to be able to change what happens next — block a merge, fail a deploy, revoke access. Anything that can only inform a committee is a signal, and we label it honestly as one rather than dress it up as a control.

Register and bound every actor before it acts. Nothing — human or non-human — gets to act until it has an owner, a declared scope and a way to be stopped. Autonomy is granted only where it can be attributed, contained and withdrawn, never by default.

Instrument the build to produce its own proof. Wire each control to emit its own record as it runs, so assurance falls out of the system instead of being assembled by hand. If demonstrating a control needs a screenshot, we have not finished building it.

Start from a named failure mode or a named harm. Design each control against a specific way the system fails — prompt injection, tool misuse, agent identity abuse, data exfiltration — or a specific harm to a person's rights. If we cannot name the risk it answers, we do not build it.

Make the governed path the easiest path. Ship governance as tooling, templates and paved paths engineers adopt without asking permission, and measure adoption. If routing around the governance is easier than using it, we fix the product, not the people.

What AI governance engineers build

Not decks. Working artefacts, versioned in a repository and running in production:

- **Policy-as-code** — governance rules as executable policy (`OPA/Rego` , Cedar, Policy Cards) that evaluate in CI/CD and at runtime.
- **An agent registry** — the runtime-aware inventory of every model, service and agent, each with an owner, a scope and a status.
- **AIBOM and model/data cards** — the bill of materials for an AI system (`CycloneDX ML-BOM` , `SPDX 3.0 AI` profile) and structured transparency documentation.
- **Eval gates in CI** — adversarial and capability evals (Inspect, promptfoo, Garak, Giskard) wired into the pipeline so a failing eval blocks the release.
- **Runtime guardrails and kill switches** — input/output controls, tool-call mediation and a tested way to stop an agent, at the point of action.
- **Continuous assurance telemetry** — tracing and monitoring (`OpenTelemetry` , agent observability) that turns production behaviour into a live control signal.
- **Machine-readable evidence** — `OSCAL` and signed, structured artefacts that make the audit a query instead of a scramble.
- **Incident pipelines** — the plumbing to detect, triage and report serious incidents on the clock, including the EU AI Act's Article 73 reporting for high-risk systems.
- **FRIA and DPIA templates as code** — fundamental-rights and data-protection impact assessments maintained as versioned, reviewable artefacts, not one-off documents.

These artefacts map, layer by layer, onto the five-layer AI governance engineering stack: Govern-as- Code, Inventory & Transparency, Evals & Red Teaming as Evidence, Runtime Controls & Observability, and Assurance & Continuous Compliance.

We concede the inheritance plainly, because it is the honest defence against "this is just GRC with AI words." Three of the five layers — Govern-as-Code, machine-readable evidence, and Assurance & Continuous Compliance — are inherited from GRC engineering and carried across almost unchanged. Two are what AI forces us to add: evals and red-teaming *as controls* (layer 03), because the thing being governed is a model whose behaviour can only be established by testing it; and agent identity and runtime control (layer 04), because an autonomous actor has no analogue in classic GRC. The new work of the discipline concentrates in those two layers.

A discipline, distinct from its neighbours

AI governance engineering is not AI safety research, MLOps, model risk management, AI compliance or legal work, or Responsible AI ethics; it is the engineering that turns all of those into running controls and readable evidence. It is the AI-era sibling of AI security engineering, the direct descendant of GRC engineering. One name clash is worth flagging: some vendors use the same words, "AI governance engineering", for the reverse problem — governing the AI tools that engineers use inside their own workflows [9]. That is governed AI engineering, not the discipline described here. Chapter 01 draws every one of these lines in full.

Authors

Jorge García Aibar (v0.1–v0.4.0) — AI Governance & Privacy Engineer. LinkedIn: <https://www.linkedin.com/in/jorgara>

Aurélié Pols (v0.1–v0.4.0) — Responsible AI (EU/Global), Privacy & Data Governance. LinkedIn: <https://www.linkedin.com/in/aureliepols>

Co-authors wanted. This is version 0.4.0: a public draft, deliberately incomplete. It was started by one practitioner and it needs many. If you build governance for AI systems — policy-as- code, agent registries, eval gates, runtime guardrails, continuous assurance — and you can bring a verified fact, a pattern that worked, or a sharper argument, you are invited to co-author. The discipline is a capability anyone can develop, and this text belongs to everyone who does the work.

Sign / get involved

- **Read it** at <https://aigovernanceengineer.com/thesis> and the Body of Knowledge at <https://aigovernanceengineer.com/bok>
- **Sign the Thesis** by opening a pull request that adds your name to `bok/CONTRIBUTORS.md` (SIGNATORIES section) in the repository, github.com/losanchos5/aige .

- **Contribute a chapter or a pattern** following `STYLEGUIDE.md` ; every factual claim needs a sourced, verified citation.
- **Discuss it** on LinkedIn with Jorge García Aibar (<https://www.linkedin.com/in/jorgara>), naming the discipline, not the person.

Licence

This work is licensed under **CC BY 4.0**. It may be freely copied and shared, but only in its entirety and including this notice, so that it continues to identify *AI Governance Engineering: The Thesis & Body of Knowledge* as the source.

Attribution: Jorge García Aibar and contributors.

Sources

[1] GRC Engineering Manifesto. grcengineering. ~2024. <https://grc.engineering/> (verified: primary) [2] "What is GRC Engineering" (Ayoub Fandi). GRC Engineer. 2025. <https://grcengineer.com/what-is-grc-engineering/> (verified: primary) [3] AI Governance Profession Report 2025. IAPP (with Credo AI). 2025-04-16. <https://iapp.org/resources/article/ai-governance-profession-report/> (verified: primary) [4] "Gartner Predicts Over 40% of Agentic AI Projects Will Be Canceled by End of 2027". Gartner. 2025-06-25. <https://www.gartner.com/en/newsroom/press-releases/2025-06-25-gartner-predicts-over-40-percent-of-agentic-ai-projects-will-be-canceled-by-end-of-2027> (verified: primary) [5] "Gartner Forecasts the Market for Securing AI Will Reach Almost \$5 Billion in 2027". Gartner. 2026-08-26. <https://www.gartner.com/en/newsroom/press-releases/2026-08-26-gartner-forecasts-the-market-for-securing-ai-will-reach-almost-5-billion-in-2027> (verified: primary) [6] CEN-CENELEC JTC 21 standards tracker / CSA research note (no harmonised standard cited in the OJ; ISO/IEC 42001 not a harmonised standard). 2026. <https://kla.digital/blog/jtc-21-standards-tracker> (verified: secondary) [7] "Best AI Governance Platforms 2026" (runtime data path critique). Kosmoy. 2026. <https://www.kosmoy.com/resources/blog/best-ai-governance-platforms-2026/> (verified: secondary) [8] Threat Report 2026 (~1 in 8 reported AI breaches involve autonomous agents). HiddenLayer. 2026. <https://www.hiddenlayer.com/report-and-guide/threatreport2026> (verified: reported) [9] "AI Governance Engineering" (governing AI used inside engineering workflows). Visure Solutions. 2026. <https://visuresolutions.com/ai-engineering/ai-governance-engineering/> (verified: primary) [10] "IBM recognized as a Leader in the Gartner Magic Quadrant for AI Governance Platforms" (first-ever MQ; visibility into AI use cases, AI asset inventory and lineage, use-case onboarding — the program layer). IBM. 2026. <https://www.ibm.com/new/announcements/ibm-recognized-as-a-leader-in-gartner-magic-quadrant-for-ai-governance-platforms> (verified: primary)

00. Preface

Why this book exists, who it is for, and how to use it.

Why this exists

There is a manifesto for GRC engineering. There is a manifesto for agile software. There is a pattern catalogue for responsible AI and a twelve-factor guide for cloud applications. There is nothing that tells you how to *engineer* the governance of AI systems — how to turn an AI Act obligation or an ISO 42001 control into policy-as-code, an eval gate, an agent registry and machine-readable evidence that an auditor can read. This book is the first attempt to write that down.

It exists because the gap is now expensive. The thing being governed — models that retrain, prompts that change, agents that act on their own — moves faster than any document can follow. Governance written as PDFs and spreadsheets is stale before it is signed. The discipline that closes the gap is engineering, applied to governance. This book is its founding text and its working reference.

Who wrote it, and from what

The Thesis and Body of Knowledge were written by Jorge García Aibar, an AI Governance & Privacy Engineer — drawing on two and a half years designing and operating an AI governance framework inside a large telco, sitting between Legal, Security and Engineering, and covering the governance, security, compliance, business and model-performance dimensions of AI risk — with Aurélie Pols, working in Responsible AI, Privacy and Data Governance, as co-author. Nothing in this book discloses any employer's internal detail; where practice is described it is generic ("in a large telco").

It is built from three things. First, that operating experience: what actually held up when a model changed on a Friday and an agent gained a new tool over the weekend. Second, the **GRC Engineering** precedent — the community, manifesto and body of practice that, over roughly the last two years, turned governance, risk and compliance into a product built with code [1][2]. Third, the public record: the EU AI Act and its Digital Omnibus reform, ISO/IEC 42001, the NIST AI RMF, OWASP's GenAI and Agentic work, CSA, CSIRO's Responsible AI Pattern Catalogue, and the frontier labs' own safety frameworks. Every factual claim in the book carries a sourced, verified citation.

Who should read this

- **AI governance leads** who want to stop shipping documents and start shipping controls.
- **Security engineers and AI security engineers** extending their threat models to models and agents.
- **Privacy engineers and DPOs** who want FRIA and DPIA to live as code, not as one-off PDFs.
- **MLOps and platform engineers** who are being asked to make governance a property of the pipeline.
- **Lawyers and compliance professionals who want to build** — to see the obligation turned into an executable control and readable evidence, and to help specify it.

You do not need to write production code to use this book, but you should be comfortable near a pipeline. The discipline is a capability anyone close to the build can develop.

What this is not

This is not a compliance checklist, and it is not legal advice. It does not tell you whether your system is compliant; it tells you how to build the controls and the evidence that let someone qualified make that call. It is not an AI safety research agenda, an MLOps handbook, or a vendor buyer's guide — tools are named only as illustrative examples of a category, never as endorsements. And it is not finished. Version 0.4.0 is a public draft with deliberate gaps for co-authors to fill.

How to cite

García Aibar, J., Pols, A. and contributors. *AI Governance Engineering: The Thesis & Body of Knowledge*, v0.4.0. 2026. <https://aigovernanceengineer.com>. Licensed CC BY 4.0.

Cite a specific chapter by its number and title (for example, "chapter 01, The definition"). The canonical home of the Thesis is <https://aigovernanceengineer.com/thesis> and of the Body of Knowledge <https://aigovernanceengineer.com/bok>. Each chapter carries its own numbered source list; the consolidated table lives in `sources/SOURCES.md`.

Versioning

This is **v0.4.0** — a public draft. Versioning is semantic in spirit: patch releases fix facts and typos, minor releases add chapters or patterns, and a 1.0 will mark the point at which the core chapters (00–10) are complete and reviewed. Every change is recorded in `bok/CHANGELOG.md`. Because the regulatory and standards landscape moves — the Digital Omnibus reform, harmonised standards under JTC 21, the OWASP and CSA releases — chapters carry a "current as of" date and are expected to be revised.

How to contribute

This book is meant to be co-authored. To contribute:

1. Read `STYLEGUIDE.md` and follow the chapter or pattern template exactly.
2. Source every factual claim. Use the `[n]` citation format, tag each source `primary`, `secondary` or `reported`, and add the row to `sources/SOURCES.md` under your chapter's section.
3. Open a pull request. To sign the Thesis, add your name to `bok/CONTRIBUTORS.md`.

The rules exist so that many hands produce one coherent book. Everything else — the arguments, the patterns, the mappings — is open for you to improve.

Maps to: this preface makes no normative claim; the standards it names are treated in full in chapters 04, 05 and 08.

Sources

[1] GRC Engineering Manifesto. grcengineering. ~2024. <https://grc.engineering/> (verified: primary) [2] "What is GRC Engineering" (Ayoub Fandi). GRC Engineer. 2025. <https://grcengineer.com/what-is-grc-engineering/> (verified: primary)

01. The definition

AI governance engineering is the application of engineering practice — systems thinking, product thinking and code — to the governance of AI systems; a capability, not a job title, measured by realised risk reduction and audit-ready evidence.

The definition

AI governance engineering is the application of engineering practice — systems thinking, product thinking and code — to the governance of AI systems.

Read the sentence in three parts. *Engineering practice* means we build, run and measure governance the way engineers build, run and measure anything else: as versioned systems with tests, telemetry and owners, not as documents.

Systems thinking and product thinking mean we treat governance as a whole that spans data, model, pipeline, runtime and organisation, delivered as a product to the engineers who are its users. *The governance of AI systems* is the subject: the whole span of governance, risk and assurance for AI — including autonomous agents — not one narrow slice of it.

The framing is borrowed, deliberately. GRC engineering defines itself as "the application of software engineering practice, systems thinking and product thinking to governance, risk and compliance" [1]. AI governance engineering is that same move, aimed at the governance of AI. It is the parent discipline pointed at a faster, stranger target.

Three clarifiers

It covers governance, risk and assurance of AI systems — including agents. The scope is not "compliance." It runs from setting the rules (governance), through identifying and reducing what can go wrong (risk), to producing the evidence that the controls work (assurance). It explicitly includes autonomous and agentic AI, because that is where the hardest governance problems now live: an agent that browses, executes code, calls APIs and acts under delegated authority is the object that legacy governance can least see.

It is a capability, not a job title. You do not need "AI governance engineer" on your business card to do this work, and having the title does not mean you are doing it. It is a set of practices — policy-as-code, eval gates, agent registries, continuous assurance — that a security engineer, a privacy engineer, an MLOps engineer or a governance lead can each develop. The market is forming the role (technical AI-governance roles in the tech sector report a median near USD 221,000, the highest band in IAPP's survey [2], and the AI governance platform market is forecast to grow from tens of millions of dollars to well over a billion by 2030 [3]) — but the discipline is defined by the capability, not by the vacancy.

It is measured by realised risk reduction and audit-ready evidence. There are exactly two tests. Did the risk actually fall — measurably, in production, not on a maturity slide? And can a regulator or auditor read the proof — as machine-readable evidence, not a reassembled screenshot? A control that passes neither test is theatre. Framework coverage, number of policies written and committees held are inputs at best; they are never the measure.

The disambiguation cluster

The discipline is defined as much by what it is not as by what it is. Eight neighbours are routinely confused with it. Each shares a border; none is the same thing.

Neighbour	What it does	How AI governance engineering differs
AI safety research	Studies whether powerful models are safe in principle (alignment, dangerous capabilities).	Engineers the controls and evidence for AI systems in production; consumes safety research, does not conduct it.
MLOps / LLOps	Builds, deploys and serves models and pipelines reliably.	Governs what MLOps ships: adds policy, evals-as-evidence, registry and assurance as gates on the same pipeline.
Model risk management (SR 11-7 style)	Validates models, checks for conceptual soundness and back-tests, in the banking tradition.	Extends beyond model validation to runtime behaviour, agents, rights impact and continuous, machine-readable evidence.
AI compliance / legal	Interprets obligations (EU AI Act, GDPR) and advises on them.	Turns the obligation into an executable control and readable evidence; needs legal, does not replace it.

Neighbour	What it does	How AI governance engineering differs
Responsible AI / AI ethics	Sets the values and principles (fairness, transparency, accountability).	Implements those values as running controls; ethics sets the target, engineering hits it and proves it.
GRC engineering (the parent)	Applies engineering practice to governance, risk and compliance generally.	Same method, specialised to AI: models, agents, evals, AIBOM, runtime AI controls.
AI security engineering (the sibling)	Secures AI systems against attack (prompt injection, model theft, agent abuse); its deliverable is a defended system.	Overlaps heavily — often the same person — but its deliverable is a <i>governed and evidenced</i> system: the rights-and-obligations record and continuous assurance, not only defence.
Visure's "AI governance engineering"	Governs the AI that engineers <i>use inside</i> engineering workflows (requirements, MBSE).	The opposite direction: our subject is governing AI <i>systems</i> , not governing AI-assisted engineering [4].

In prose: the frontier lies where a discipline stops. **AI safety research** asks whether a model is safe; we ask whether the deployed system is governed, and prove it. **MLOps** answers "is the model serving?"; we answer "is it allowed to serve, and what evidence says so?" — governing the very pipeline MLOps runs. **Model risk management** in the SR 11-7 tradition validates a model at points in time; we govern the system continuously, including agents that have no analogue in a credit model. **AI compliance and legal** tell you what the law requires; we build the control that meets it and the evidence that shows it, and we depend on lawyers to tell us we got the obligation right. **Responsible AI and AI ethics** set the values; without engineering, those values stay on a poster. **GRC engineering** is the parent method, and we are its AI specialisation. We inherit three of the five stack layers almost unchanged — govern-as-code, machine-readable evidence and continuous assurance — along with the "green dashboard over a broken control is theatre" test; what AI forces us to add is the other two, evals and red-teaming as controls and agent identity and runtime control, because a model whose behaviour must be tested and an autonomous actor that acts under delegated authority have no analogue in classic GRC. **AI security engineering** is the sibling we overlap with most, and the overlap is a feature, not a boundary dispute: the same person often wears both hats. The line is not "framing versus fences" but *deliverable*. Security engineering's deliverable is a defended system — it stops the attack. AI governance engineering's deliverable is a *governed and evidenced* system: the rights-and-obligations record (which control answers which article, with the proof attached) and continuous assurance as a product an auditor or regulator can query. A red-team eval is security work and governance work at once; it becomes governance when its result is filed as evidence against an obligation. Security asks "is it safe from attack?"; we ask "is it governed, and can we prove it?" — and we usually need the security answer as an input to ours. And **Visure's** use of the identical phrase points the other way entirely: governing the AI that assists engineering work, not engineering the governance of AI. We claim the second meaning and disambiguate the first on sight.

The object of governance

What, concretely, does this discipline govern? Five nested objects, each needing different controls:

- **Models.** The trained artefacts — foundation models, fine-tunes, classifiers — with their provenance, capabilities, evaluations and known failure modes. Governed with model cards, evals and AIBOM.
- **Systems.** The application around the model: prompts, retrieval, tools, orchestration, the human and machine users. Most risk is here, not in the raw model.
- **Agents.** Systems that act — browse, execute code, call APIs, move money, delegate to other agents. Governed with identity, bounded scope, tool mediation, runtime guardrails and kill switches. This is the hardest and newest object, and the one legacy governance cannot see.
- **Data.** Training data, retrieval corpora, prompts and outputs — with their lawful basis, rights, provenance and retention. Governed with data cards, DPIAs and lineage.
- **The organisation.** The roles, decision rights, escalation paths and accountability that surround all of the above. Governed with an operating model, RACI and an incident pipeline. A control with no owner is not a control.

The discipline is coherent only when it addresses all five. A model card with no agent registry, or an agent registry with no runtime data path, governs one object and leaves the others open.

The three questions

At any moment, an AI governance engineering function must be able to answer three questions about production — instantly, from live systems, not from a document last updated a quarter ago:

1. **What AI is running?** Which models, systems and agents are live, in which version, owned by whom. This is the job of the inventory and the agent registry — and it must be fed by a runtime data path, not typed into a spreadsheet.
2. **What is it allowed to do?** The scope, permissions, guardrails and policy that bound each system and agent. This is the job of governance-as-code and runtime controls — identity before autonomy, scope before action.
3. **What evidence proves it?** The machine-readable, audit-ready record that the controls fired and the risk fell. This is the job of evals-as-evidence and continuous assurance — evidence as a by-product of the build.

These three questions are the spine of the whole Body of Knowledge. The five-layer stack (chapter 04) is built to answer them: Govern-as-Code and Inventory & Transparency answer *what is running* and *what it may do*; Evals & Red Teaming and Assurance & Continuous Compliance answer *what evidence proves it*; Runtime Controls & Observability keeps all three answers true as the system changes. The threats those controls are built against — prompt injection, tool misuse, agent identity and privilege abuse, rogue agents — are catalogued in OWASP's Top 10 for Agentic Applications [5], and the patterns that answer them are in chapter 05.

The limits of the eval gate

This book leans hard on evals as controls, so it owes the reader the same "theatre" test it applies to everything else. An eval gate is necessary; it is not sufficient. Take it seriously and its limits follow directly:

- **It is point-in-time and sampling-bound.** An eval proves the system passed *these* cases at *this* version. It says nothing about the inputs it did not sample, and nothing about tomorrow's model.
- **It is Goodhartable.** The moment a threshold gates a release, there is pressure to tune the model to the suite or the threshold to the model. A gate optimised against becomes a number that rises while the risk it stood for does not move.
- **It catches regressions, not novelty.** A suite tests known failure modes. A novel jailbreak or an attack the suite never imagined passes green, because nothing in the gate was built to see it.

None of this argues against the gate; it argues for how the gate must be run. The eval suite is itself an artefact to be governed: its coverage measured, its cases maintained adversarially, its thresholds traced to named failure modes rather than round numbers. And a passing gate *obliges* runtime monitoring — layer 04 — rather than replacing it. An eval is the control at build time; the guardrail and the trace are the control at run time, against the inputs no eval anticipated. A discipline that treats a green gate as proof of safety has rebuilt framework theatre with a faster pipeline.

In practice Inside a large telco, the difference between "governed" and "documented" came down to those three questions. A model inventory maintained by hand answered question one on the day it was edited and was wrong within a week. Wiring the registry to the deployment pipeline — so a new model or agent registered itself, with an owner and a scope, at deploy time — was what made the three questions answerable on any given Tuesday. The document became a query.

Maps to: EU AI Act Art. 9 (risk management), Art. 11/49/71 (documentation and registration), Art. 55 (GPAI systemic-risk duties) · ISO/IEC 42001 (AI management system) · NIST AI RMF (Govern, Map, Measure, Manage) · OWASP Top 10 for Agentic Applications 2026. Mappings are illustrative, not a claim of conformity.

Sources

[1] "What is GRC Engineering" (Ayoub Fandi). GRC Engineer. 2025. <https://grcengineer.com/what-is-grc-engineering/> (verified: primary) [2] Salary & Jobs Report 2025-26. IAPP. 2025-08-03. <https://iapp.org/resources/article/salary-survey-summary/> (verified: primary) [3] "Global AI Regulations Fuel Billion-Dollar Market for AI Governance Platforms". Gartner. 2026-02-17. <https://www.gartner.com/en/newsroom/press-releases/2026-02-17-gartner-global-ai-regulations-fuel-billion-dollar-market-for-ai-governance-platforms> (verified: primary) [4] "AI Governance Engineering". Visure Solutions. 2026. <https://visuresolutions.com/ai-engineering/ai-governance-engineering/> (verified: primary) [5] Top 10 for Agentic Applications 2026. OWASP GenAI Security Project. 2025-12-09. <https://genai.owasp.org/resource/owasp-top-10-for-agentic-applications-for-2026/> (verified: primary)

02. Why now

AI governance engineering is forming now because the thing being governed changed shape, the market renamed the role before the profession named itself, and the law began asking for engineered evidence — all inside the same eighteen months.

Disciplines do not appear on a schedule. They appear when an old way of working visibly stops holding and enough people, in enough places, start building the replacement at once. That is happening to AI governance now. The Thesis states five fundamental problems with legacy AI governance; this chapter takes each in turn, attaches the evidence, and then lays out the market, regulatory and technical signals that together explain the timing. The claim is narrow and falsifiable: not that governance suddenly matters, but that the *engineering* of governance has become the constraint, and that the data now says so.

The five problems, with the evidence

1. Governance written for systems that no longer exist

Legacy governance describes an AI system as it was on the day it was reviewed, while models retrain, prompts change and agents gain tools by the day; the artefact is stale before it is signed. The structural tell is *where the function sits*. In IAPP's 2025 AI Governance Profession Report, the AI governance function is lodged mostly with privacy (22%), legal and compliance (22%) and IT (17%), and with only 5% in security [1] — far from the pipeline where the system changes. A function in the review layer, not the build layer, cannot keep its description of production true, because nothing wires it to the deploy.

2. Point-in-time review of a continuously changing thing

Annual assessments and committee sign-offs assume a system that holds still long enough to be judged. Frontier models and autonomous agents do not. Gartner expects more than 40% of agentic AI projects to be cancelled by the end of 2027, naming inadequate risk controls among the causes [2], and predicts that by 2029 more than half of successful attacks on AI agents will exploit access-control weaknesses and prompt injection [3]. Both are runtime failure modes — the system misbehaving between reviews — and a once-a-year assessment is structurally blind to them. This is not the same work as model risk management in the SR 11-7 tradition, which validates a model at points in time; the object here never stops moving.

3. Governance as a gate at the end, not a property of the build

Governance still tends to arrive after the model is trained, as a checkpoint to clear before launch, and nothing it produces is wired back into how the system is built. A policy that can only recommend cannot stop a bad release; an eval wired into an `eval gate` that can fail the build can — the difference between a control that describes risk and one that prevents it.

4. Framework theatre

Mapping to NIST AI RMF or ISO/IEC 42001 becomes the end state instead of the starting point, and a green mapping matrix is mistaken for a working control. Yet as of 2026-09-19 no harmonised standard is cited in the EU's Official Journal, so there is no Article 40 presumption of conformity for the AI Act [4]. Even EN ISO/IEC 42001, the AI management-system standard, is not the Article 17 quality management system the Act requires and confers no presumption of conformity on its own [5]. Coverage is not assurance. A crosswalk proves you have read the framework; it does not prove the control it points to actually fires.

5. No runtime data path

The registry does not know what is running. Most of the AI governance platform category "manages the program — inventories, assessments, framework mappings, evidence workflows — without any runtime data path" [6]. So the three questions that define the discipline — what AI is running, what is it allowed to do, what evidence proves it — go unanswered, because nothing is connected to production. Meanwhile roughly one in eight reported AI breaches now involves an autonomous agent [7]: exactly the layer the paper registry cannot see. A control with no telemetry is a claim, not a control.

The evidence

The profession is understaffed, and it knows it

The clearest signal that the work has outgrown its current form is that almost nobody thinks they have enough people to do it. In IAPP's 2025 report, of 671 respondents only 10 — 1.5% — said they will not need additional AI governance staff in the next 12 months [1]. Some 77% of organisations report working on AI governance, rising to roughly nine in ten among those already using AI [1]. Demand is near-universal; capacity is not. When a function is wanted everywhere, under-resourced, and lodged outside the build layer, the gap is not closed by hiring more reviewers. It is closed by turning the governance into a system that scales — which is an engineering problem.

The market renamed the role before the profession did

The labour market is already pricing this shift, and it is doing so in the vocabulary of engineering. An analysis of United States AI-governance postings found the most common skill demanded was observability and monitoring, in roughly 41% of postings, with Python in about 28% — ahead of several named frameworks [8]. These are build-and-run skills, not review skills. Live "AI Governance Engineer" postings ask for the same substance under the new title: an enterprise agent registry, threat modelling, red teams, non-human-identity controls and NIST AI RMF alignment, at a large family office [9]; AI intake and classification aligned to NIST AI RMF, at a global custodian bank [10]; security controls, threat models and incident response for LLM systems, at a mid-market employer [11]. And at the frontier, Anthropic has advertised an Engineering Manager for GRC tasked with building an "AI-forward GRC engineering function", translating "policies into policy-as-code", and deploying agentic workflows that use Claude "to serve as a virtual GRC analyst" [12]. The market has been describing AI governance engineering for a year; it simply had not agreed on the name.

Registry versus runtime

The tooling market tells the same story from the supply side. Gartner published its first Magic Quadrant for AI Governance Platforms in June 2026, and its inclusion criteria are revealing: AI discovery and registry, compliance risk management, policy management, dynamic risk scoring, evidence collection, workflow and approvals, and an audit trail [13]. Every one of those sits at the program layer. The market for these platforms is real and growing fast — Gartner puts it at roughly USD 65 million in 2024, rising to more than USD 1.4 billion by 2030 [14] — but growth in program-layer tooling does not close the runtime gap. As one review of the category puts it, most of it manages the program "without any runtime data path" [6]. The category is maturing around inventories and evidence workflows precisely while the risk is migrating to runtime and to agents. That mismatch is the space the discipline occupies.

The standards gap

The framework-theatre problem has a hard deadline attached to it. As of 2026-09-19, zero harmonised standards are cited in the Official Journal, so Article 40's presumption of conformity is not yet available to anyone [4]. EN 18286, the quality-management standard aimed at Article 17, was published in July 2026 — the first JTC 21 AI Act standard to reach publication — but it is not yet cited in the Official Journal, so it carries no presumption of conformity [15]. The Article 9 risk-management, Article 12 logging and Article 15 cybersecurity standards were still at the Enquiry stage, targeting the end of 2026 [15]. EN ISO/IEC 42001:2026, the European adoption of the AI management-system standard, is not the Article 17 QMS and does not by itself evidence conformity [5]. The practical consequence for a governance function: for the period covered by this edition, there is no standard you can certify against to buy a legal presumption. You have to build the controls and the evidence yourself and be ready to defend them on their merits.

The regulatory wave: the Omnibus and GPAI enforcement

The law moved twice in the same summer, in opposite directions, and both moves point at engineering. First, the Digital Omnibus — Regulation (EU) 2026/1744 — entered into force on 27 July 2026, six days before the 2 August high-risk deadline, and reset the clock: Annex III high-risk obligations moved from 2 August 2026 to 2 December 2027, and Annex I embedded high-risk from 2 August 2027 to 2 August 2028 [16]. The extra time is real, but it is not relief from the engineering; it is more runway to do it. Second, and unmoved by the Omnibus, GPAI enforcement went live on 2 August 2026: the AI Office can now demand documentation, evaluate models and require measures, and the Commission can fine providers of general-purpose models up to 3% of global annual turnover or EUR 15 million, whichever is higher, under Article 101 [17]. Trade press reported that the AI Office sent its first Requests for Information to frontier labs on 29 August 2026, covering model security, external evaluations and post-market monitoring; that specific report is not independently confirmed in a primary source and is carried here only as reported [18]. What the enforceable obligations ask for — model evaluations, adversarial testing, incident reporting, weight security — is an engineering programme, not a policy binder.

The agent shift

The last signal is the object of governance itself. Agents that browse, execute code, call APIs and act under delegated authority are now the hardest and newest thing to govern, and the evidence that legacy controls cannot see them is mounting. Gartner's forecast that by 2029 more than half of successful attacks on AI agents will exploit access-control weaknesses and prompt injection [3] names identity and injection — runtime concerns — as the dominant attack surface. The identity problem is concrete: NIST's National Cybersecurity Center of Excellence published a 2026 concept paper on identity and authorisation for software and AI agents, which is reported to flag that agents today are commonly run on generic, shared service accounts rather than distinct, attributable identities [19] (reported) — the exact condition that makes an agent's actions impossible to trace or revoke precisely. OWASP's Top 10 for Agentic Applications 2026 catalogues the failure modes that follow, from agent goal hijack (ASI01) through tool misuse (ASI02) and agent identity and privilege abuse (ASI03) to rogue agents (ASI10) [20]. And HiddenLayer's 2026 threat reporting puts roughly one in eight reported AI breaches at the feet of autonomous agents [7] (reported). The threat model has moved to the layer the paper registry cannot reach.

What changes when governance is engineered

The five problems share one root: governance that describes rather than runs. Engineering the governance changes the artefact, and changing the artefact changes what the function can promise. When the registry is fed by the deployment pipeline, "what AI is running?" is a live query, not a quarterly guess, and the staleness of problem 1 stops existing. When an `eval gate` fails the build on a dropped injection-resistance threshold and a policy compiled to code blocks an out-of-region deployment, the point-in-time review of problem 2 and the end-of-line gate of problem 3 give way to controls that fire where the system changes. When guardrail decisions, eval results and policy verdicts stream into an assurance store as machine-readable records, the framework theatre of problem 4 is answered by a measure that is a named failure mode's falling rate, not a count of green cells. And when identity precedes autonomy and telemetry becomes a live control signal, the runtime data path of problem 5 is the spine rather than an afterthought — so the three questions become answerable on any given Tuesday, from live systems.

None of this is a claim that engineered governance guarantees compliance; no artefact does, and no standard yet confers a presumption. The claim is narrower and more useful: measured against realised risk reduction and audit-ready evidence, a governance function that runs beats one that is written, and the market, the regulator and the threat model have all, in the same window, started asking for the version that runs. That is why now.

In practice Inside a large telco, the shift from "documented" to "engineered" was visible in a single quarter's incident drill. The paper version answered "which agents can reach the payments API?" with a spreadsheet that was a week stale and missing two services stood up since the last review. After the registry was wired to the deploy pipeline and every agent was issued a scoped identity, the same question was a query that returned owners, scopes and last-seen timestamps in seconds, and a misbehaving agent could be revoked without breaking the others. The evidence for the drill was not assembled afterwards; it was already in the assurance store.

Maps to: EU AI Act Art. 4/4a (literacy, bias-detection data), Art. 15 (robustness, cybersecurity), Art. 17 (QMS), Art. 53/55 (GPAI), Art. 101 (GPAI fines) · ISO/IEC 42001 · NIST AI RMF (Govern, Map, Measure, Manage) · OWASP Top 10 for Agentic Applications 2026 (ASI01–ASI03, ASI10) · five-layer stack, all layers. Mappings are illustrative, not a claim of conformity.

Sources

[1] AI Governance Profession Report 2025 (with Credo AI; 671 respondents; only 1.5% will not need more staff; 77% working on AI governance; 5% of the function in Security). IAPP. 2025-04-16. <https://iapp.org/resources/article/ai-governance-profession-report/> (verified: primary) [2] "Gartner Predicts Over 40% of Agentic AI Projects Will Be Canceled by End of 2027". Gartner. 2025-06-25. <https://www.gartner.com/en/newsroom/press-releases/2025-06-25-gartner-predicts-over-40-percent-of-agentic-ai-projects-will-be-canceled-by-end-of-2027> (verified: primary) [3] "Gartner Forecasts the Market for Securing AI Will Reach Almost \$5 Billion in 2027" (by 2029, >50% of successful attacks on AI agents exploit access control and prompt injection). Gartner. 2026-08-26. <https://www.gartner.com/en/newsroom/press-releases/2026-08-26-gartner-forecasts-the-market-for-securing-ai-will-reach-almost-5-billion-in-2027> (verified: primary) [4] CEN-CENELEC JTC 21 standards tracker (no harmonised standard cited in the OJ → no Art. 40 presumption). CEN-CENELEC JTC 21 (via kla.digital). 2026. <https://kla.digital/blog/jtc-21-standards-tracker> (verified: secondary) [5] "ISO/IEC 42001 and the AI Act: why certification is not yet a presumption of conformity" (ISO 42001 AIMS ≠ Art. 17 QMS). Law & Technology. 2026. <https://lawandtechnology.eu/en/iso-iec-42001-and-the-ai-act-why-certification-is-not-yet-a-presumption-of-conformity/> (verified: secondary) [6] "Best AI Governance Platforms 2026" (category manages the program "without any runtime data path"). Kosmoy. 2026.

<https://www.kosmoy.com/resources/blog/best-ai-governance-platforms-2026/> (verified: secondary) [7] Threat Report 2026 (~1 in 8 reported AI breaches involve autonomous agents). HiddenLayer. 2026.

<https://www.hiddenlayer.com/report-and-guide/threatreport2026> (verified: reported) [8] "AI Governance Jobs" insights (US postings: observability/monitoring ~41%, Python ~28%). Axial Search. 2026-08-04.

<https://axialsearch.com/insights/ai-governance-jobs> (verified: secondary) [9] "AI Governance Engineer" posting, Sept 2026 (enterprise agent registry, threat modelling, red teams, non-human-identity controls, NIST AI RMF). Dalio Family Office (via SimplyHired). 2026-09.

<https://www.simplyhired.com/job/vKibNHPKReLf8h5PgCpXyJWZOx1ctDsmI8sjzdKbSpd6ZoNoNvP-Q> (verified: secondary) [10] "AI Governance VP" posting, Sept 2026 (AI intake and classification aligned to NIST AI RMF). State Street (via SimplyHired). 2026-09.

https://www.simplyhired.com/job/iy04VeXdd9DR3vffvKZvupPCsoZtHi405zOLc_RMSJXOSRJvqLbhmw (verified: secondary) [11] "AI Governance Engineer" posting, Sept 2026 (security controls, threat models, incident response for LLM systems). Bright Vision (via SimplyHired). 2026-09.

https://www.simplyhired.com/job/s2v6mqCI9KO8Xldf5enD0Z46iDqIBLdwS0jqZlgtO8YhD-DU_4XDog (verified: secondary) [12] "Engineering Manager, GRC" (build an "AI-forward GRC engineering function"; "translate policies into policy-as-code"; Claude "as a virtual GRC analyst"). Anthropic (via General Catalyst jobs). 2026.

<https://jobs.generalcatalyst.com/companies/anthropic/jobs/78167778-engineering-manager-grc> (verified: primary) [13] "IBM recognized as a Leader in the Gartner Magic Quadrant for AI Governance Platforms" (first MQ, June 2026; inclusion criteria: discovery & registry, compliance risk mgmt, policy management, dynamic risk scoring, evidence collection, workflow & approvals, audit trail). IBM / Gartner. 2026-06. <https://www.ibm.com/new/announcements/ibm-recognized-as-a-leader-in-gartner-magic-quadrant-for-ai-governance-platforms> (verified: secondary) [14] "Global AI Regulations Fuel Billion-Dollar Market for AI Governance Platforms" (~\$65M in 2024 → >\$1.4B by 2030). Gartner. 2026-02-17. <https://www.gartner.com/en/newsroom/press-releases/2026-02-17-gartner-global-ai-regulations-fuel-billion-dollar-market-for-ai-governance-platforms> (verified: primary) [15] EN 18286:2026 (Art. 17 QMS) published July 2026, not yet cited in the OJ; risk/logging/cybersecurity standards at Enquiry (Q4 2026 target). CEN-CENELEC news / JTC 21 tracker. 2026-07-30. <https://www.cenelec.eu/news-events/news/2026/en-in-the-spotlight/2026-07-30-ai-quality-management/> (verified: secondary) [16] "AI Omnibus enters into force" (Reg. (EU) 2026/1744, in force 2026-07-27; Annex III high-risk → 2 Dec 2027; Annex I → 2 Aug 2028). European Commission. 2026-07-27. <https://digital-strategy.ec.europa.eu/en/news/ai-omnibus-enters-force> (verified: primary) [17] Commission enforcement powers over GPAI providers apply from 2 August 2026 (fines up to 3% of worldwide turnover or EUR 15M under Art. 101). European Commission — AI Act Service Desk. 2026-08-02. <https://ai-act-service-desk.ec.europa.eu/en/ai-act/faq/commissions-enforcement-powers-related-ai-act-obligations-providers-most-advanced-models> (verified: primary) [18] Reported first AI Office Requests for Information to GPAI providers on 29 Aug 2026 (model security, external evaluations, post-market monitoring); not independently confirmed in a primary source. Trade press. 2026-08-29. <https://tokenstead.ai/guides/eu-ai-act-first-enforcement-security-rfis> (verified: reported) [19] Concept paper "Software and AI Agent Identity and Authorization" (agents commonly run on generic/shared service accounts rather than distinct identities). NIST NCCoE. 2026-02-05. <https://www.nccoe.nist.gov/news-insights/new-concept-paper-identity-and-authority-software-agents> (verified: reported) [20] Top 10 for Agentic Applications 2026 (ASI01 Agent Goal Hijack, ASI02 Tool Misuse, ASI03 Agent Identity & Privilege Abuse, ... ASI10 Rogue Agents). OWASP GenAI Security Project. 2025-12-09. <https://genai.owasp.org/resource/owasp-top-10-for-agentic-applications-for-2026/> (verified: primary)

03. Values and principles

The eight values and six principles of the Thesis, each expanded with what it means in practice and the anti-pattern it rejects.

The Thesis states eight values and six principles in one line each. This chapter expands them. The two are deliberately different kinds of thing, and the difference is what keeps them from being the same list told twice. A **value** is a preference — a trade-off, what we lean toward when we cannot have both; it is stated as an affirmation, and naming what we build *toward* also names what we build *away from*. A **principle** is a commitment to *act* — a rule of method, phrased as something we do, that holds regardless of preference. So the values name the artefacts and outcomes we build toward; the principles name how we work, and they carry no vocabulary borrowed from the values. Read the values to know which way to lean; read the principles to know what to do on Monday.

Not all of the values are new to AI. Governance as code (1), machine-readable evidence (5) and measured risk reduction (7) are inherited from GRC engineering, the parent discipline. Evals that fail the build (2) and agent identity and scope (4) are what AI forces us to add — the model that must be tested and the autonomous actor that must be bounded have no analogue in classic GRC.

The eight values

Each value is stated as an affirmation — what we build toward, which by naming it also names what we build away from. The values-and-principles form follows the GRC Engineering Manifesto that is this book's precedent [1].

1. Governance is code, not a document

A policy in a PDF is a statement of intent that a human must read, remember and apply. A policy as code is a control that executes: it evaluates a pull request, a deployment or a runtime call and returns a decision. The document describes the rule; the code *is* the rule, versioned in a repository, tested, and enforced without anyone remembering to. When the rule changes, you change one artefact and every enforcement point updates. When an auditor asks what the policy was on a given date, you show the commit.

This is not "delete all documents." A policy still needs a human-readable statement of why it exists. The value is that the authoritative, enforced version is the executable one, and the prose is its documentation — not the other way around.

In practice A data-residency rule written as `OPA/Rego` blocks any deployment that would route inference outside the permitted region, in CI and at admission. The same rule, as a PDF, was "communicated" quarterly and violated monthly. **Anti-pattern** The "policy library" that is a folder of Word documents nobody can query, whose enforcement is an email and whose compliance is self-attested.

2. Evals fail builds; reviews only recommend

A review produces a recommendation; someone may act on it, later, or not. An eval produces a verdict with consequences: the model or agent passed or failed a defined test, and a failure blocks the release. In AI systems the most honest control is a test the system must pass, run automatically, whose result changes what happens next. We prefer controls that bite.

A review still has its place — for questions no test can settle. But when a property *can* be tested (a jailbreak resistance threshold, a PII-leakage bound, a tool-scope check), turning it into a review instead of an eval gate is a choice to be able to recommend but not to stop.

In practice A red-team eval suite (Inspect, Garak) runs in CI; if injection resistance drops below the agreed threshold, the pipeline fails and the release does not ship until it is fixed. **Anti-pattern** A "model risk review board" that meets monthly, writes findings rated low/medium/high, and has no mechanism to stop a launch already scheduled.

3. Evidence comes from runtime, not from a point-in-time attestation

An attestation says a control was in place when someone looked. Runtime evidence shows the control working continuously, from the system itself. AI systems change between reviews — a model is retrained, an agent gains a tool

— so evidence gathered once decays immediately. We prefer evidence that is emitted as the system runs, so "is the control working?" is answered by live telemetry, not by a signature dated last quarter.

In practice Guardrail decisions, eval results and policy verdicts stream into an assurance store with timestamps; the control's status is a live query, not an annual sign-off. **Anti-pattern** A SOC-style attestation binder assembled the week before an audit, describing controls as they were imagined to be, not as production actually behaved.

4. Every agent carries its own identity and scope

An agent acting on a shared service account or a static key is ungovernable: you cannot attribute its actions, revoke its access precisely, or bound what it may do. We prefer every non-human actor to have its own identity, an owner, and a scope of permitted actions — established *before* it is allowed to act. Identity is the precondition of accountability; scope is the precondition of containment.

In practice Each agent is issued a distinct workload identity, registered with an owner and a declared scope; a misbehaving agent is traced to its identity and its access revoked without touching the others. **Anti-pattern** A fleet of agents sharing one API key and one privileged service account, where an incident means rotating one secret and breaking everything, and attribution is impossible.

5. Evidence is machine-readable or it is not evidence

Evidence a human must produce, format and file by hand does not scale and cannot be verified at speed. Machine-readable evidence — `OSCAL`, structured eval results, signed logs — can be queried, diffed, aggregated and checked automatically. We prefer evidence a machine can read, because the audit then becomes a query and the same evidence feeds continuous assurance rather than a one-off binder.

In practice Control results are emitted as `OSCAL` component and assessment artefacts; an auditor's question is answered by running a query against the evidence store. **Anti-pattern** A shared drive of screenshots and exported spreadsheets, re-collected from scratch for every audit, unverifiable and out of date the moment it is saved.

6. Tooling must be inspectable and composable

The point is not who built the tool or whether it is open source; it is whether you can see inside it. You cannot trust a verdict you cannot trace. Inspectable tooling lets you follow a decision to the rule that produced it, the input it saw and the evidence it emitted; composable tooling lets you wire that decision into your own pipeline rather than export into someone else's. We prefer tooling whose reasoning and data path we can open — bought or built — because governance you cannot see inside is a control you cannot trust. Much of the AI-governance platform category stops at the program layer, "managing inventories, assessments and framework mappings without any runtime data path" [3]; the objection is not that it is commercial but that its verdict cannot be audited.

This is a lean, not an absolute. Closed and commercial tools have a place, including capable platforms. But the default is tooling the team can inspect and compose, over a black box the team must take on faith.

In practice The eval harness, policy library and registry expose how a verdict is reached — the rule, the input and the emitted evidence — and compose into the pipeline the team already runs, whether the components are open-source or a platform with an open data path. **Anti-pattern** A six-figure governance platform whose "compliance score" cannot be traced to a single running control, and whose data path stops at the spreadsheet import.

7. Success is measured in realised risk reduction, not framework coverage

Mapping every control to NIST AI RMF and ISO 42001 proves you have read the frameworks; it does not prove any risk fell. We prefer to measure the thing itself: did the rate of the failure mode drop, did the blast radius shrink, did the incident get caught earlier? Coverage is an input; realised risk reduction is the outcome. A green mapping matrix over a broken control is theatre with extra steps [2].

In practice Each control declares the failure mode it addresses and a metric for it (injection success rate, time-to-detect, unauthorised tool-calls blocked); the control is judged by the metric moving, not by the framework cell turning green. **Anti-pattern** A 300-row traceability matrix mapping controls to five frameworks, presented as maturity, with no measurement of whether any of the mapped controls actually reduces risk.

8. Governance is owned with engineering, not enforced from outside

Governance that sits apart and grants or denies passage is a bottleneck engineers route around. Governance owned jointly with engineering — built into the paved path, adopted because it is the easiest way to ship — becomes part of how things are made. We prefer shared ownership: the governance function builds the tooling, engineering builds on it, and the gate is a stage in a pipeline both own, not a meeting one side dreads.

In practice The eval gate and policy checks ship as part of the standard pipeline template; engineers adopt them because the paved path is also the fastest path, and governance co-maintains them. **Anti-pattern** A governance team that reviews and sign-offs releases from the outside, measured by how many it stops, while engineering builds a shadow process to avoid it.

We still use the practices each value builds away from; we build toward the affirmation.

The six principles

Build the control at the earliest point it can block

Put every control where it can still stop the thing going wrong, and no later. The earliest place a risk can be caught is the cheapest to fix and the strongest place to enforce — catching it in the repository beats catching it in production, which beats explaining it to a regulator. So a control is sited at the first gate that can refuse the change: a policy in CI, an eval before deploy, an identity check at admission, a guardrail at the point of action. The commitment is about *placement*: whatever the control, it belongs at the earliest enforceable point, not bolted on at the end.

In practice A new agent cannot be deployed until it has registered an owner and a scope and passed its eval gate; the pipeline enforces this at the earliest gate that can refuse it, not a person at the end. **Anti-pattern** A pre-launch "governance review" that happens after the system is built, changes nothing about how it was built, and can only delay or wave through.

Give every control teeth

A control must be able to change what happens next — block a merge, fail a deploy, revoke access. Anything that can only inform a committee is a *signal*, and we say so rather than dress it up as a control. This is the commitment that turns an eval from research into a gate, a policy from a PDF into a check, a threshold from a dashboard number into a release blocker. When a property can be tested, turning it into a review instead of a gate is a choice to be able to recommend but not to stop, and we make that choice consciously or not at all.

The teeth are not the whole animal. A gate that can fail the build is necessary and not sufficient: an eval is point-in-time and sampling-bound, it can be gamed by tuning the model to the suite or the threshold to the model, and it catches regressions against known cases, not the novel attack the suite never imagined. So a control with teeth carries obligations of its own — its coverage measured, its cases maintained adversarially, its thresholds traced to named failure modes — and a passing gate *obliges* the runtime monitoring of layer 04, it does not replace it. A green gate treated as proof of safety is framework theatre with a faster pipeline.

In practice Capability and adversarial evals are versioned alongside the model; the deploy depends on the eval job passing, its coverage is tracked as its own metric, and a runtime guardrail carries the same threshold into production against the inputs no eval sampled. **Anti-pattern** A one-off model evaluation run before launch, its results pasted into a slide, never re-run when the model or its prompts change — or a gate whose threshold is quietly lowered until the build goes green.

Register and bound every actor before it acts

Nothing — human or non-human — gets to act until it has an owner, a declared scope and a way to be stopped. Autonomy on shared credentials is ungovernable by construction: you cannot attribute, contain or revoke what you cannot trace to an actor. So registration is a precondition, not a follow-up: every actor is enrolled, scoped and given a kill switch *before* it does anything on its own. Autonomy is earned by being governable, not granted by default.

In practice The agent registry is the gate: an agent with no owner, no scope or no kill switch is denied a workload identity and cannot reach production. **Anti-pattern** Agents spun up ad hoc on a shared key, discovered only after one of them takes an action no one can explain or undo.

Instrument the build to produce its own proof

Wire each control to emit its own record as it runs, so assurance falls out of the system instead of being assembled by hand before an audit. If demonstrating a control needs a screenshot, we have not finished building it. The commitment is to instrumentation: every gate, guardrail and check writes a structured, signed record as it fires, so the audit is a query and the same records drive continuous assurance and incident response.

In practice Every gate and guardrail writes a structured, signed record; the audit trail builds itself, and the evidence store answers both the auditor and the on-call engineer. **Anti-pattern** An evidence-collection sprint before each audit, re-creating after the fact a record the systems never actually produced.

Start from a named failure mode or a named harm

Design each control against a specific way the system fails or a specific harm it can do to a person, and start there — not from a framework checklist. Threat models (prompt injection, tool misuse, agent identity abuse, data exfiltration) and fundamental-rights impact assessments are the inputs to design, not paperwork produced afterward. If we cannot name the risk a control answers, we do not build it.

In practice The agent's design starts from its OWASP Agentic threat model [4] and its FRIA; the controls that ship are exactly the ones those two documents demanded, and they map back to them. **Anti-pattern** A control catalogue assembled from a framework checklist, addressing risks the system does not have while missing the injection path an attacker actually uses.

Make the governed path the easiest path

Ship governance as tooling, templates and paved paths engineers adopt without asking permission — a policy library, a registry with an API, a gate they can run locally before they push — and measure adoption. The people governed are our users; the paved path has to be the fastest path or it will be routed around. If using the governance is harder than avoiding it, the governance is designed wrong, and we fix the product, not the people.

In practice An engineer scaffolds a new AI service from a template that already includes the registry hook, the policy checks and the eval gate; compliance is the default, not a request. **Anti-pattern** A governance intranet of forms and ticket queues, where doing the right thing takes a week and a meeting, so teams quietly do the fast thing instead.

Maps to: the values and principles are realised through the five-layer stack (chapter 04) and the pattern catalogue (chapter 05); the failure modes they target are OWASP Top 10 for Agentic Applications; the assurance they demand maps to ISO/IEC 42001, NIST AI RMF and EU AI Act Arts. 9, 15, 55 and 72. Mappings are illustrative, not a claim of conformity.

Sources

[1] GRC Engineering Manifesto (values and principles precedent). grcengineering. ~2024. <https://grc.engineering/> (verified: primary) [2] "What is GRC Engineering" (Ayoub Fandi; "theatre with extra steps"). GRC Engineer. 2025. <https://grcengineer.com/what-is-grc-engineering/> (verified: primary) [3] "Best AI Governance Platforms 2026" (runtime data path critique). Kosmoy. 2026. <https://www.kosmoy.com/resources/blog/best-ai-governance-platforms-2026/> (verified: secondary) [4] Top 10 for Agentic Applications 2026. OWASP GenAI Security Project. 2025-12-09. <https://genai.owasp.org/resource/owasp-top-10-for-agentic-applications-for-2026/> (verified: primary)

04. The stack (five layers)

The reference architecture of AI governance engineering: five layers that answer the three questions, where evidence is produced at the bottom and proven at the top.

How to read the stack

The stack is not an org chart and not a maturity ladder. It is a build order. Each layer produces an artefact that the layer above consumes, so that governance stops being a set of parallel documents and becomes a single system with a data path from policy to proof.

Read it along one spine: **Policy** → **Inventory** → **Evals** → **Runtime** → **Assurance**. You write the rule as code (layer 01). You cannot enforce a rule against a system you cannot see, so you inventory what is running (layer 02). You cannot claim a system meets the rule without a test that can fail, so you run evals as evidence (layer 03). Rules and tests decay the moment the system changes, so you hold the line at runtime (layer 04). And none of it is worth anything to an auditor unless the evidence is emitted, signed and queryable, so you close with continuous assurance (layer 05).

Evidence flows up. A policy verdict from layer 01, a registry entry from layer 02, an eval result from layer 03 and a guardrail decision from layer 04 are not four disconnected records. Each is a structured artefact with a timestamp and an owner, and layer 05 is where they are aggregated into audit-ready evidence. The test of the whole stack is the test from the Thesis: as Ayoub Fandi puts it, a green dashboard over a broken control is "theatre with extra steps" [1]. The stack is the plumbing that makes the dashboard mean something — every green cell traces to a running control and the evidence it emitted.

The five layers, named exactly and in order, are: **01 Govern-as-Code** · **02 Inventory & Transparency** · **03 Evals & Red Teaming as Evidence** · **04 Runtime Controls & Observability** · **05 Assurance & Continuous Compliance**. They map onto the three questions the discipline must answer at any moment. Layers 01 and 02 answer *what AI is running* and *what it is allowed to do*. Layers 03 and 05 answer *what evidence proves it*. Layer 04 keeps all three answers true as the system changes underneath them. The threats the controls are built against — goal hijack, tool misuse, agent identity and privilege abuse, rogue agents — are catalogued in OWASP's Top 10 for Agentic Applications 2026 [2].

Three of these layers are inherited, not invented. Govern-as-Code (01), machine-readable evidence (part of 02 and 05) and Assurance & Continuous Compliance (05) come almost unchanged from GRC engineering. Layers 03 and 04 — evals and red-teaming as controls, and agent identity and runtime control — are what AI forces the discipline to add, because a model whose behaviour must be tested and an autonomous actor that acts under delegated authority have no analogue in classic GRC. The new work concentrates there; the rest is a specialisation of a method that already works.

Every tool named below is an example of a category, not a recommendation. The categories are the substance; the brands are illustrative and interchangeable.

Layer 01 — Govern-as-Code

What it proves. That a governance rule exists as an executable artefact, not a paragraph, and that it evaluated a specific change and returned a decision. The proof is a policy verdict tied to a commit, a pull request or a deploy — machine-readable and reproducible.

The artefacts. An AI governance engineer ships policy-as-code: rules written in a policy language that a pipeline evaluates. Typical artefacts are a policy library under version control, a set of crosswalks that map each policy to the frameworks it serves, and the CI/CD wiring that runs the policy at the right gate. A policy in this layer is small and testable: "inference for this data class may not route outside the permitted region", "no model deploys without a registered owner and a passed eval gate", "an agent may not be granted a tool scope it did not declare". Each rule carries a human-readable statement of intent as its documentation, but the enforced version is the code.

Reference tools and standards (illustrative). Policy engines express allow/deny logic. **OPA/Rego** is the general default for non-authorization policy — data residency, deploy gating, configuration constraints — while **Cedar** is narrower: an authorization language, strongest for "may this principal take this action on this resource?" and a poor fit for policy that is not an access decision. Choose by the shape of the rule, not the brand. Beyond general-purpose engines, one proposed approach is Policy Cards: a research proposal for a machine-readable, deployment-layer artefact that encodes operational, regulatory and ethical constraints for an agent and links them to enforcement and audit pipelines [3]; it is a single preprint, promising but not yet a standard, and the point it makes — that the rule should travel with

the agent as data — stands whichever format wins. Crosswalks reference the frameworks themselves: ISO/IEC 42001 (the AI management system), NIST AI RMF and its four functions — Govern, Map, Measure, Manage [4] — the EU AI Act, and the CSA AI Controls Matrix (AICM) v1.1, which offers 247 control objectives across 18 domains as a control vocabulary to map against [5]. Vendors are illustrative; the standards are not.

Definition of done.

- Every policy exists as code in a repository, with a test that proves it fires on a violating input and passes a clean one.
- Each policy declares the framework clauses it maps to, so the crosswalk is generated from the code, not maintained beside it.
- At least one policy runs pre-merge and blocks the merge on failure; at least one runs at deploy or admission and blocks the release.
- A change to a policy is a reviewable diff with an owner and an effective date.
- The policy engine emits a structured verdict (allow/deny, rule id, input hash, timestamp) for every evaluation.

Anti-patterns.

- The "policy library" that is a folder of Word documents nobody can query, enforced by email and self-attested in a spreadsheet.
- The crosswalk maintained as a 300-row matrix in a separate tool, drifting from the policies it claims to describe — coverage presented as control.

Evidence to the next layer. The policy verdict is the first evidence artefact. But a verdict is only meaningful against a known object: "deny deploy of model X" presumes the stack knows model X exists, who owns it and what it is. That object is what layer 02 supplies.

In practice For `csa-01`, a customer-service assistant in a large telco, a data-residency rule written as `OPA/Rego` ran at both CI and admission and blocked any deployment routing its inference outside the permitted region. The value was not the rule but that the enforced version emitted a verdict every engineer could see before merge.

Maps to: EU AI Act Art. 9 (risk management) · ISO/IEC 42001 · NIST AI RMF (Govern) · CSA AICM · OWASP Agentic ASI02/ASI03. Mappings are illustrative, not a claim of conformity.

Layer 02 — Inventory & Transparency

What it proves. That the organisation knows what AI is running — which models, systems and agents are live, in which version, owned by whom — and that each has the transparency documentation an obligation demands. The proof is a registry entry and its attached documents, ideally written by a deployment pipeline rather than typed by hand.

The artefacts. The core artefact is the **agent registry**: the runtime-aware inventory of every model, service and agent, each with an owner, a scope and a status. Around it sit the transparency documents — model cards and data cards — and the **AIBOM**, the bill of materials for an AI system. FRIA and DPIA references hang off the registry entry for systems that need them, so the inventory is also the index of which system has which impact assessment.

Reference tools and standards (illustrative). Registries range from ITSM and governance suites (for example ServiceNow, Credo AI) to agent-discovery tools (for example Zenity) that find AI the inventory did not know about. The AIBOM has standard formats: CycloneDX ML-BOM and the SPDX 3.0 AI profile, with the OWASP AIBOM generator producing CycloneDX output [6]. The distinction from a classic SBOM matters: an AIBOM records models, datasets, weights and their provenance, not only software dependencies. The registry is the object that layer 01's policies evaluate and layer 04's runtime controls attach to; without a runtime data path feeding it, it is a spreadsheet that was true on the day it was edited.

Definition of done.

- Every model, system and agent in production has a registry entry with an owner, a version and a declared scope; an unregistered artefact cannot reach production.
- The registry is fed by the deployment pipeline, not by manual entry — a new deploy registers itself.
- Each high-risk system links to its model card, data card and, where required, its FRIA/DPIA.
- An AIBOM is generated at build for each AI system and stored with the registry entry.
- A discovery mechanism periodically reconciles the registry against what is actually running and flags drift.

Anti-patterns.

- The hand-maintained model inventory that answers "what is running?" correctly on the day it is edited and is wrong within a week.
- Transparency documents written once at launch and never regenerated when the model, prompt or dataset changes — a model card describing a model that no longer exists.

Evidence to the next layer. The registry entry names the object under test. An eval in layer 03 is run *against a registered version* and its result is filed *against that entry*, so the question "was this model tested?" is answered by a join, not a search. The AIBOM tells the eval layer what to test — which model, which datasets, which provenance claims need adversarial probing.

In practice Wiring the registry to the deployment pipeline — so `csa-01` registered itself, with an owner and a scope, at deploy time — turned "documented" into "governed". Its registry entry became a query answered from production, not from a slide.

Maps to: EU AI Act Art. 11 (technical documentation), Art. 49/71 (registration and the EU database), Art. 50 (transparency) · ISO/IEC 42001 · NIST AI RMF (Map) · CSA AICM · OWASP Agentic ASI10.

Layer 03 — Evals & Red Teaming as Evidence

What it proves. That the model or agent passed a defined test whose failure has consequences. This is the layer where the principle "give every control teeth" becomes concrete for evals: a benchmark that only informs a committee is not a control; an eval wired into an **eval gate** that can fail the build is. The proof is a structured eval result — pass or fail against a threshold, versioned alongside the model it tested.

The artefacts. An engineer ships eval suites and the gate that runs them. Three families recur: capability and quality evals (does the system do its job — groundedness, regression against a golden set), adversarial and red-team evals (does it resist jailbreaks, prompt injection, tool misuse), and safety-threshold evals tied to a policy from layer 01 (a PII-leakage bound, an injection-resistance floor). The gate is a pipeline stage: if the eval drops below the agreed threshold, the pipeline fails and the release does not ship.

Reference tools and standards (illustrative). Evaluation frameworks such as Inspect — maintained on behalf of the UK AI Security Institute [7] — promptfoo and DeepEval run capability and regression suites; Garak, Mindgard and Giskard run adversarial and vulnerability probes; Ragas covers retrieval-augmented quality. Adversarial testing is not only good practice: the GPAI Code of Practice lists adversarial testing and red-teaming among the evaluation approaches expected of GPAI models with systemic risk, though the Code is voluntary and scoped to those models [8]. The eval result is the evidence artefact that layer 05 will aggregate — which is why it must be machine-readable, not a screenshot pasted into a slide.

Definition of done.

- Each model or agent has an eval suite versioned in the same repository and updated when the system changes.
- At least one adversarial eval and one capability eval run in CI, with a documented threshold.
- A failing eval blocks the deploy — the gate has teeth, not just a report.
- Every eval run emits a structured result (suite id, model version, score, threshold, pass/fail, timestamp) filed against the registry entry.
- Thresholds trace to a named failure mode or obligation, not to a round number chosen for comfort.

Anti-patterns.

- The one-off pre-launch evaluation whose results are pasted into a slide and never re-run when the model or its prompts change.
- The "risk review board" that rates findings low/medium/high monthly but has no mechanism to stop a launch already scheduled — recommendation without consequence.

Evidence to the next layer. An eval proves the system was safe *at test time*. The system then meets inputs no eval anticipated. Layer 04 carries the same thresholds into production as runtime guardrails, and the eval result becomes the baseline that runtime telemetry is compared against — a drop in live injection-resistance against the tested baseline is a signal, not a surprise.

In practice A red-team eval suite combining Inspect and Garak ran in CI for `csa-01`; a release that dropped its injection resistance below the agreed floor failed the pipeline until fixed. The eval's output — not a reviewer's opinion — was the assurance evidence filed against the model version.

Maps to: EU AI Act Art. 15 (accuracy, robustness, cybersecurity), Art. 55 (GPAI systemic-risk evaluation) · ISO/IEC 42001 · NIST AI RMF (Measure) · CSA AICM · OWASP Agentic ASI01/ASI02.

Layer 04 — Runtime Controls & Observability

What it proves. That the controls hold while the system is acting, and that its behaviour is observed. Policies and evals are point-in-time; agents act continuously, on inputs no one reviewed. This layer proves that a guardrail mediated a real call, that an agent acted under its own identity and scope, and that there is a tested way to stop it. The proof is a stream of runtime decisions and traces.

The artefacts. Three groups. Guardrails: input/output filters, tool-call mediation, the enforcement point where a policy from layer 01 fires against a live request. Observability: tracing and monitoring that turn agent behaviour into a control signal. Agent runtime identity: every non-human actor with its own workload identity, a bounded scope, and a **kill switch** — a tested way to revoke access and stop an agent without breaking the fleet. Register and bound every actor before it acts: no agent acts before it has an identity, an owner and a scope.

Reference tools and standards (illustrative). Guardrail frameworks such as NVIDIA NeMo Guardrails, Meta LlamaFirewall and Lakera enforce input/output and tool-call policy; observability tools such as Langfuse and Arize Phoenix build on OpenTelemetry to trace agent runs. Agent identity has two distinct questions the tools should not be allowed to blur. The first is **channel authentication**: how a client authenticates to a tool server on one hop. The Model Context Protocol specification of 2026-07-28 tightened exactly this — deprecating Dynamic Client Registration in favour of Client ID Metadata Documents and binding credentials to their issuer [9]. That hardens the MCP connection; it is not the identity model for the agent. The second is **agent workload identity**: a durable, attributable identity the agent carries across every hop, tool and protocol, under which its actions are logged and its access revoked. That is the job of a workload-identity system — SPIFFE/SPIRE, or first-class agent identities from enterprise providers (for example Microsoft Entra Agent ID, Okta Agent SSO) — not of MCP, which secures one channel. Conflating the two leaves an agent well-authenticated on the MCP hop and still unattributable everywhere else. NIST's NCCoE set out the open questions in its February 2026 concept paper on software and AI agent identity and authorization: how identification, authentication and authorization apply so each agent is "known, trusted, and properly governed", including non-repudiation and tamper-proof logging [10]. Gartner expects that by 2029 more than half of successful attacks on AI agents will exploit access-control weaknesses and prompt injection [11] — the failure modes this layer exists to contain.

Definition of done.

- Every agent runs under its own identity with a declared scope; no agent shares a service account or a static key across functions.
- A guardrail mediates tool calls and input/output for each agent, enforcing the scope declared in the registry.
- A kill switch exists and has been tested — revoking one agent's access does not break the others.
- Every agent run is traced, and traces carry the agent identity, the tools called and the policy verdicts that fired.
- Runtime signals are compared against the eval baseline from layer 03, and a regression raises an alert.

Anti-patterns.

- A fleet of agents sharing one API key and one privileged service account: an incident means rotating one secret and breaking everything, and attribution is impossible.
- Guardrails that log but never block — observability mistaken for control, so the system watches itself fail in high resolution.

Evidence to the next layer. Every guardrail decision, trace and identity event is a timestamped, structured record. Layer 05 does not re-collect this evidence; it subscribes to it. Runtime is where continuous assurance gets its continuity — the difference between an attestation that a control existed and evidence that it fired, on a specific call, at a specific time.

In practice `csa-01` was issued a distinct workload identity with an owner and a declared scope, as was every other agent in the telco; when it misbehaved it was traced to its identity and revoked without touching the others. The kill switch was tested on a schedule — an untested kill switch is a claim, not a control.

Maps to: EU AI Act Art. 14 (human oversight), Art. 15 (robustness, cybersecurity), Art. 12 (logging) · ISO/IEC 42001 · NIST AI RMF (Manage) · CSA AICM · OWASP Agentic ASI02/ASI03/ASI10.

Layer 05 — Assurance & Continuous Compliance

What it proves. That the controls below are working, continuously, and that the proof is machine-readable and audit-ready. This is where evidence stops being a by-product and becomes the product: the audit is a query, not a project. The proof is a live assurance store that any of the lower layers writes into and an auditor can read from.

The artefacts. Machine-readable evidence in a standard format; framework mappings generated from that evidence rather than maintained beside it; and the incident and reporting plumbing that turns a runtime signal into an obligation met on the clock. The organising format is `OSCAL`. Its stable substrate is NIST's native model: a control layer (`catalog` , `profile`), an implementation layer (`component-definition` , `system-security-plan`) and an assessment layer (`assessment-plan` , `assessment-results` , `POA&M`), with traceability from an assessment result back to the control it tested [15]. That model is the part to build on; the AI-specific additions on top are still forming. One proposed approach — a single 2026 preprint, not a standard — extends OSCAL with sixteen property extensions for lifecycle phase, enforcement semantics and risk traceability, in a three-layer compliance-as-code architecture that generates OSCAL assessment results automatically [13]. Treat it as an early answer to a real gap the authors name well: frameworks "such as the EU AI Act, ISO/IEC 42001, and NIST AI RMF specify what to assure but provide no executable format for how" [12]. The gap is what layer 05 closes; the native OSCAL assessment models close most of it today, with or without the extensions.

Reference tools and standards (illustrative). Evidence is emitted as OSCAL component and assessment artefacts; GRC and AI-governance suites (for example Vanta, Drata, OneTrust; `watsonx.governance`, Holistic AI, Saidot) aggregate and present it. Incident reporting maps to EU AI Act Art. 73 (serious incidents) and Art. 72 (post-market monitoring). No harmonised standard is yet cited in the EU's Official Journal, so an ISO/IEC 42001 certificate supports the Art. 17 quality management system but does not by itself satisfy it, and confers no presumption of conformity under Art. 40 [13].

Definition of done.

- Control results from layers 01–04 are emitted as machine-readable evidence (for example OSCAL) with timestamps and owners, continuously.
- Framework mappings are generated from the evidence, so a mapping cell that turns green points at a control that actually fired.
- An auditor's question is answered by a query against the evidence store, not an evidence-collection sprint.
- A serious-incident pipeline can detect, triage and report on the clock, with the Art. 73 timelines encoded, not remembered.
- The assurance store measures realised risk reduction — the failure-mode rate, the time-to-detect, the blast radius — not framework coverage.

Anti-patterns.

- The attestation binder assembled the week before an audit, describing controls as they were imagined to be, not as production behaved.
- A "compliance score" from a closed platform that cannot be traced to a single running control, whose data path stops at the spreadsheet import.

Evidence closes the loop. Assurance is not the top of a one-way ladder. A drop in a live metric feeds back to layer 03 as a new eval, to layer 01 as a tightened policy, and to layer 02 as a registry flag. The stack is a loop that happens to be drawn as a ladder.

In practice `csa-01` 's guardrail decisions, eval results and policy verdicts streamed into an assurance store with timestamps, so a control's status was a live query, not an annual sign-off. When an auditor asked what its data-residency control did in the second quarter, the answer was a filter over emitted evidence, produced in minutes.

Maps to: EU AI Act Art. 17 (quality management), Art. 72 (post-market monitoring), Art. 73 (serious incident reporting) · ISO/IEC 42001, ISO/IEC 42005 · NIST AI RMF (Govern, Manage) · CSA AICM.

Data governance across the stack

The five layers govern models and agents; they are only as sound as the data underneath them, and data governance is not one layer but a thread through all five. Training data, fine-tuning sets, retrieval corpora, prompts and outputs each carry a lawful basis, a provenance, a retention limit and a set of rights, and each is an object the stack must be able to name. In layer 02 this is the **data card** and the lineage record — where a dataset came from, what it may be used for, when it must be deleted — attached to the registry entry beside the model card. In layer 01 it is retention-and-residency policy as code. In layer 03 it is data-quality and bias tests run against the set, not assumed of it. The EU AI Act treats this as a first-class duty: Article 10 requires representative, relevant and error-checked datasets for high-risk systems, and the post-Omnibus Article 4a gives a narrow lawful basis to process special-category data *for bias detection*, conditioned on pseudonymisation and deletion once bias is corrected (see chapter 08). The engineering commitment is that a RAG corpus is governed like a model: versioned, its provenance and licence recorded in the AIBOM, its snapshot tied to the eval that tested the system on it, so "what was in the corpus when this answer was produced?" is a query, not a guess. Data with no card, no lineage and no retention rule is the ungoverned object that makes every layer above it unprovable.

Designing human oversight (Article 14)

Human oversight is a control to be engineered, not a reassurance to be asserted. EU AI Act Article 14 requires that high-risk systems be designed so a person can *effectively* oversee them — understand the output, decide against it, and stop the system — and the hard part is that undifferentiated oversight fails in both directions. Human review of every action destroys the value of automation; nominal oversight of a firehose of actions is a rubber stamp, and a rubber stamp is worse than none because it launders the decision. Two failure modes have to be designed against explicitly. **Automation bias**: a reviewer who sees a confident machine output will tend to confirm it, so oversight that only offers "approve/reject" on the model's proposal is oversight in name only. **Oversight that degrades**: a gate that a person can clear in two seconds under load will be cleared in two seconds, and its quality falls silently as volume rises. The engineering answer is to classify actions by consequence and place a designed checkpoint only where the stakes justify the latency — the **Human-in-the-loop Gate** pattern (chapter 05) — giving the reviewer enough context to disagree, logging the approver and the decision as evidence, and monitoring the oversight itself (approval rate, time-to-decide, override rate) as a signal that can degrade. Oversight you do not measure is oversight you cannot claim.

Third-party and procured AI

Most organisations do not train the models they run. They buy SaaS with an embedded LLM, call an API-only foundation model, or inherit an agent inside a vendor's product — and for those, the parts of the stack that assume you own the model degrade. You cannot red-team weights you cannot reach, and an **eval gate** (layer 03) can only test the vendor's system as a black box, at its boundary, not its internals. Runtime control (layer 04) narrows to what the integration exposes: the tool scopes you grant, the identity you issue the vendor's agent, the traffic you can observe — not the model's own behaviour. The layers do not disappear, but layer 03 shrinks to boundary evals and reliance on the vendor's own evidence, and layer 04 shrinks to the perimeter you control. What grows to compensate is inventory and assurance: the vendor's system still needs a registry entry, an owner and a scope; its supplier documentation, model card and any AIBOM become evidence you collect rather than produce; and the due-diligence itself becomes a gate. This is the **Vendor / Model Due-Diligence Gate** pattern (chapter 05), anchored in ISO/IEC 42001 Annex A.10 (third-party and customer relationships) and the EU AI Act's split of duties between provider and deployer (see chapter 08). The rule of thumb: the less of the model you own, the more of your control budget moves from testing it to bounding it and evidencing the supplier.

The cost of the stack

None of this is free, and a FinOps line is part of governing it honestly. The recurring costs are compute for eval suites run on every change (adversarial suites are the expensive ones, and running them on each commit rather than each release is a real bill), storage and egress for traces and the evidence store (agent traces are verbose, and continuous assurance means keeping them long enough to answer an audit), and the engineering time to maintain suites, thresholds and integrations as the systems move. The costs scale with change frequency and trace volume, so the levers are obvious once named: sample or tier expensive evals by risk, set retention by obligation rather than by default, and push the cheapest controls (a policy check, an identity gate) to the front where they catch failures before an expensive eval runs. A stack whose running cost no one tracks is a stack that will be cut in the first budget round, which is its own governance failure.

The minimum viable stack for a team of one

Most AI governance functions are small, and many are a single person: in the adjacent GRC discipline, roughly half of teams are four people or fewer and about one in six is a team of one [14]. A team of one cannot build all five layers at depth, but it can build the spine thinly, end to end — one vertical slice that touches every layer beats one layer built out and four left on paper. Start where the leverage is highest and the cost is lowest:

- **Layer 02 first, minimally.** A registry that a deploy writes to, with an owner and a scope per entry. If you can answer "what is running and who owns it?" from a live source, you have more than most.
- **One policy in layer 01 with teeth.** A single rule that matters — no deploy without a registered owner, or a data-residency check — as code, in the pipeline, blocking on failure. One control that bites beats a hundred that recommend.
- **One eval gate in layer 03.** One adversarial eval against your highest-risk agent, wired so a regression fails the build. Reuse an open framework; do not write your own harness.
- **Identity and a kill switch in layer 04.** Every agent under its own identity with a scope, and a tested way to stop it. This is the cheapest control with the largest blast-radius reduction.
- **Evidence as a by-product in layer 05.** Have each of the above emit a structured, timestamped record into one store. You are not building an OSCAL pipeline yet; you are refusing to rely on screenshots.

The order is deliberate: see it, rule it, test it, contain it, prove it. A thin vertical slice answers all three questions for one system today and widens as the team grows. The alternative — a thick layer 01 of policies with no inventory beneath them — answers none of the three questions, and is exactly the framework theatre the discipline exists to end.

One system through the five layers

`csa-01`, the customer-service assistant from the boxes above, is one system, not five. Below is the single artefact it produces at each layer — short excerpts of the schemas defined in chapter 05, each illustrative.

Layer 01 — Govern-as-Code — a Policy Card verdict (illustrative):

```
{ "rule_id": "residency.eu-only.v3", "decision": "deny",  
  "input_hash": "sha256:9f2b...", "timestamp": "2026-09-18T14:07:11Z" }
```

Layer 02 — Inventory & Transparency — its registry entry (illustrative):

```
{ "id": "csa-01", "owner": "team-support-platform",  
  "scope": ["refunds:read", "orders:read"], "expiry": "2026-12-17" }
```

Layer 03 — Evals & Red Teaming as Evidence — an eval-gate result (illustrative):

```
{ "suite_id": "injection-resistance.v4", "model_version": "csa-01@2026-09-18",  
  "score": 0.982, "threshold": 0.95, "result": "pass" }
```

Layer 04 — Runtime Controls & Observability — a guardrail event (illustrative):

```
{ "agent": "csa-01", "direction": "output", "rule_id": "output.pii.v2",  
  "decision": "block", "timestamp": "2026-09-18T14:31:52Z" }
```

Layer 05 — Assurance & Continuous Compliance — the evidence record it emits (illustrative):

```
{ "control_id": "guardrail.output.pii.v2", "subject": "csa-01@2026-09-18",  
  "decision": "alert", "obligation": "EU AI Act Art. 15",  
  "timestamp": "2026-09-18T14:31:52Z" }
```

The five excerpts are one data path from policy to proof, keyed on the same registry id.

Sources

[1] "What is GRC Engineering" (Ayoub Fandi; "theatre with extra steps"). GRC Engineer. 2025. <https://grcengineer.com/what-is-grc-engineering/> (verified: primary) [2] Top 10 for Agentic Applications 2026. OWASP GenAI Security Project. 2025-12-09. <https://genai.owasp.org/resource/owasp-top-10-for-agentic-applications-for-2026/> (verified: primary) [3] Policy Cards: Machine-Readable Runtime Governance for Autonomous AI Agents (arXiv 2510.24383). 2025-10. <https://arxiv.org/abs/2510.24383> (verified: primary) [4] AI Risk Management Framework (AI RMF 1.0). NIST. 2023-01-26. <https://www.nist.gov/itl/ai-risk-management-framework> (verified: primary) [5] AI Controls Matrix (AICM) v1.1 (247 control objectives across 18 domains). Cloud Security Alliance. 2026-06-22. <https://cloudsecurityalliance.org/artifacts/ai-controls-matrix-v1-1> (verified: primary) [6] "Evolving AI Transparency: the AIBOM generator's new home at OWASP" (CycloneDX output). OWASP GenAI Security Project. 2025-12-18. <https://genai.owasp.org/2025/12/18/evolving-ai-transparency-the-journey-of-the-aibom-generator-and-its-new-home-at-owasp/> (verified: primary) [7] Inspect — a framework for large language model evaluations (UK AI Security Institute). GitHub. 2026. https://github.com/UKGovernmentBEIS/inspect_ai (verified: primary) [8] GPAI Code of Practice, Safety & Security chapter (adversarial testing / red-teaming for systemic-risk models; voluntary; published 10 Jul 2025). European Commission. 2025-07-10. <https://artificialintelligenceact.eu/code-of-practice-overview/> (verified: primary) [9] Model Context Protocol specification 2026-07-28 (DCR deprecated in favour of CIMD; issuer-bound credentials). MCP. 2026-07-28. <https://blog.modelcontextprotocol.io/posts/2026-07-28/> (verified: primary) [10] "Accelerating the Adoption of Software and AI Agent Identity and Authorization" (concept paper; "known, trusted, and properly governed"). NIST NCCoE. 2026-02-05. <https://www.nccoe.nist.gov/news-insights/new-concept-paper-identity-and-authority-software-agents> (verified: primary) [11] "Gartner Forecasts the Market for Securing AI Will Reach Almost \$5 Billion in 2027" (>50% of agent attacks exploit access-control and prompt injection by 2029). Gartner. 2026-08-26. <https://www.gartner.com/en/newsroom/press-releases/2026-08-26-gartner-forecasts-the-market-for-securing-ai-will-reach-almost-5-billion-in-2027> (verified: primary) [12] "Making AI Compliance Evidence Machine-Readable" (OSCAL + 16 property extensions; three-layer compliance-as-code) (arXiv 2604.13767). UC3M. 2026-04-15. <https://arxiv.org/abs/2604.13767> (verified: primary) [13] JTC 21 standards tracker / CSA research note (no harmonised standard cited in the OJ; ISO/IEC 42001 not a harmonised standard, supports but does not satisfy Art. 17). CEN-CENELEC JTC 21 (via kla.digital) / CSA. 2026. <https://kla.digital/blog/jtc-21-standards-tracker> (verified: secondary) [14] State of GRC 2026 (≈51% of GRC teams ≤4 people; ≈18.5% solo). GRC Engineer. 2026. <https://grcengineer.com/report/> (verified: primary) [15] OSCAL native model (control layer: catalog, profile; implementation: component-definition, system-security-plan; assessment: assessment-plan, assessment-results, POA&M). NIST. 2026. <https://pages.nist.gov/OSCAL/learn/concepts/layer/> (verified: primary)

05. Patterns

A catalogue of reusable AI governance engineering patterns, each named to a layer of the stack, in the CSIRO Responsible AI Pattern Catalogue structure.

This chapter is the catalogue. Each pattern is a reusable solution to a problem that recurs when you engineer the governance of AI systems. The structure follows the CSIRO Responsible AI Pattern Catalogue, which applies software-engineering design patterns to responsible AI across governance, process and product levels [1]. We keep its fields — summary, objectives, target users, impacted stakeholders, relevant principles, context, problem, solution, consequences, related patterns — and add a **Maps to** line naming the standards, articles and stack layer (1–5) each pattern serves.

Every pattern names one of the five layers (chapter 04) so the catalogue and the stack stay consistent, and realises one or more of the six principles (chapter 03): *build the control at the earliest point it can block · give every control teeth · register and bound every actor before it acts · instrument the build to produce its own proof · start from a named failure mode or harm · make the governed path the easiest path*. Each **Maps to** line draws its threat IDs from the OWASP Top 10 for Agentic Applications 2026 [3] and its function labels from the NIST AI RMF [18]. Each pattern carries a short example labelled **(illustrative)**: a plausible, de-identified sketch, not a claim about any named system. Mappings to the EU AI Act are illustrative, not a claim of conformity, and no harmonised standard is yet cited in the Official Journal.

Pattern: Policy Card

Summary — Express a governance rule as a machine-readable artefact that travels with the model or agent and is evaluated in the pipeline and at runtime, rather than as prose a human must apply. The Policy Card encodes permitted and prohibited actions, obligations and evidentiary requirements for one AI system, and links to the enforcement and audit pipelines that act on it [2].

Objectives

Turn a policy from a statement of intent into an executing control, versioned and testable, so that a rule change is a reviewable diff and every evaluation leaves a verdict.

Target users

AI governance engineer, platform team, policy owner.

Impacted stakeholders

Model owners, deployers, auditors, regulators.

Relevant principles

Build the control at the earliest point it can block; make the governed path the easiest path.

Context

An organisation with more than a handful of AI systems and a governance function that cannot review every change by hand. Rules exist but live in documents no pipeline can read.

Problem

Prose policy cannot be enforced automatically, drifts from the systems it governs, and leaves no evidence that it was applied. A rule that can only be remembered is violated the moment someone forgets.

Solution

Write each rule as a Policy Card: a structured, machine-readable artefact (for example expressed for an **OPA/Rego** or **Cedar** engine, or as a Policy Cards document) that states allow/deny logic, the failure mode it addresses and the framework clauses it maps to. Store it with the system it governs. Evaluate it pre-merge, at deploy, and — where the rule is a runtime constraint — at the point of action. Emit a verdict (rule id, input hash, decision, timestamp) on every evaluation.

Illustrative schema for the verdict:

```
{
  "rule_id": "residency.eu-only.v3",
  "decision": "deny",
  "input_hash": "sha256:9f2b...",
  "timestamp": "2026-09-18T14:07:11Z"
}
```

Example (illustrative) A Policy Card for a customer-service agent declares that it may call the refunds tool only up to a bounded amount and never outside business hours; the same card is evaluated in CI against the agent's declared scope and at runtime by the tool-call guardrail.

Consequences

Rules become enforceable and auditable, and the crosswalk generates itself. The cost is authoring and maintaining cards, and the discipline to keep the executable version authoritative over the prose.

Related patterns

Framework Crosswalk; Eval Gate in CI; Runtime Guardrail; Machine-Readable Evidence (OSCAL); Agent Identity & Scoped Credentials.

Maps to: EU AI Act Art. 9 · ISO/IEC 42001 · NIST AI RMF (Govern) · CSA AICM · OWASP Agentic ASI02/ASI03 · Layer 01 Govern-as-Code.

Pattern: Eval Gate in CI

Summary — Wire an evaluation suite into the CI/CD pipeline so that a model or agent must pass a defined test, above a documented threshold, before it can ship. The eval run is the control and its result is the evidence; a failing eval blocks the build.

Objectives

Make "give every control teeth" concrete: give a testable property a consequence, so failure stops a release instead of filing a finding.

Target users

AI governance engineer, ML engineer, platform team.

Impacted stakeholders

Model owners, users exposed to the system, auditors.

Relevant principles

Give every control teeth; build the control at the earliest point it can block.

Context

A model or agent that changes — retrained, re-prompted, given a new tool — and a pipeline that already runs tests for functional correctness.

Problem

Evaluations run once before launch and pasted into a slide prove nothing after the next change. A review board that can only rate findings cannot stop a scheduled launch. Without a gate, evaluation is research, not control.

Solution

Version an eval suite alongside the model. Run at least one capability eval and one adversarial eval in CI (for example with Inspect, promptfoo, Garak or Giskard — illustrative). Set a threshold that traces to a named failure mode or obligation. Fail the pipeline below the threshold. Emit a structured result (suite id, model version, score, threshold, pass/fail, timestamp) filed against the registry entry.

Illustrative schema for the result:

```
{
  "suite_id": "injection-resistance.v4",
  "model_version": "csa-01@2026-09-18",
  "score": 0.982,
  "threshold": 0.95,
  "result": "pass",
  "timestamp": "2026-09-18T14:22:03Z"
}
```

Example (illustrative) An internal coding agent must clear an injection-resistance floor and a regression suite before deploy; a release that drops resistance below the floor fails the pipeline and does not ship until fixed.

Consequences

Regressions are caught before production and evidence accrues automatically. The trade-off is eval maintenance, run-time cost in CI, and the need to tune thresholds to avoid flaky gates.

Related patterns

Policy Card; Adversarial Red-Team Suite; Continuous Assurance Telemetry; Machine-Readable Evidence (OSCAL); Model Card as Control Evidence.

Maps to: EU AI Act Art. 15, Art. 55 · ISO/IEC 42001 · NIST AI RMF (Measure) · OWASP Agentic ASI01/ASI02 · Layer 03 Evals & Red Teaming as Evidence.

Pattern: Adversarial Red-Team Suite

Summary — Maintain a versioned adversarial test suite, built from a threat taxonomy and run in CI or on a schedule, whose findings are triaged into fixes or accepted risks, recorded as evidence, and fed back into the suite. Where an Eval Gate proves a threshold still holds, the red-team suite is the standing adversary that keeps finding the inputs the threshold never anticipated.

Objectives

Turn adversarial testing from a one-off exercise into a maintained, versioned control that discovers failure modes before an attacker does and leaves a triaged, auditable record of every finding.

Target users

AI governance engineer, security engineer, ML engineer, red-team lead.

Impacted stakeholders

Model owners, users exposed to the system, incident responders, auditors, regulators.

Relevant principles

Start from a named failure mode or harm; give every control teeth.

Context

A model or agent whose exposure grows as it gains tools, prompts and reach, in an organisation that already runs an Eval Gate for regression and wants a standing adversary rather than a single pre-launch penetration test.

Problem

A one-off red-team is out of date the moment the system changes, and its findings — a slide of jailbreaks — leave no trace that they were fixed or accepted. Without a versioned suite and a triage record, the same attack is rediscovered every quarter and no one can prove which findings were closed.

Solution

Build the suite from a threat taxonomy rather than intuition: draw techniques from MITRE ATLAS's adversarial tactics and techniques for AI systems [25] and the agentic attack classes in the OWASP Top 10 for Agentic Applications [3], so each test traces to a named technique. Version the suite alongside the model and run it in CI or on a schedule against

the registered version. Route each finding through triage — fix, or accept with a recorded rationale and owner — and file the outcome as a structured evidence record against the registry entry. Feed every confirmed finding back into the suite as a regression test, so a closed attack stays closed. The suite complements the Eval Gate: the gate enforces a threshold on each release, the suite is the adversary that generates the next one.

Example (illustrative) A red-team suite for a customer-service assistant runs a versioned set of prompt-injection and tool-abuse cases drawn from ATLAS and the OWASP agentic classes; a new tool-exfiltration finding is triaged, fixed and added to the suite, so the next release must pass it.

Consequences

Adversarial coverage grows over time instead of resetting each launch, and the triage record shows what was found, fixed or accepted. The cost is maintaining the taxonomy and suite, the compute to run adversarial cases often, and the discipline to triage every finding rather than let it lapse.

Related patterns

Eval Gate in CI; Runtime Guardrail; Continuous Assurance Telemetry; Incident Pipeline; Vendor / Model Due-Diligence Gate.

Maps to: EU AI Act Art. 9, Art. 15, Art. 55 (GPAI) · ISO/IEC 42001 · NIST AI RMF (Measure) · OWASP Agentic ASI01/ASI02 · Layer 03 Evals & Red Teaming as Evidence.

Pattern: Agent Registry

Summary — Maintain a runtime-aware inventory of every model, service and agent, each entry carrying an owner, a scope and an expiry, fed by the deployment pipeline rather than typed by hand. The registry is the object that policies evaluate and runtime controls attach to.

Objectives

Answer "what AI is running, and what is it allowed to do?" from a live source, and make registration a precondition of reaching production.

Target users

AI governance engineer, platform team, security engineer.

Impacted stakeholders

Model owners, deployers, auditors, incident responders.

Relevant principles

Register and bound every actor before it acts; make the governed path the easiest path.

Context

An organisation deploying models and agents across teams, where no single source knows what is live.

Problem

A hand-maintained inventory is correct on the day it is edited and wrong within a week. Without owner, scope and expiry, an action cannot be attributed, a scope cannot be enforced, and a stale agent lingers with standing access no one revisits.

Solution

Make the registry an API the deploy pipeline writes to: a new model or agent registers itself at deploy with an owner, a declared scope and an expiry after which the entry must be renewed or is deactivated. Deny production access to unregistered artefacts. Reconcile periodically against what is actually running (see Shadow-AI Discovery) and flag drift.

Illustrative schema for a registry entry:

```
{  
  "id": "csa-01",
```

```
"version": "2026-09-18",
"owner": "team-support-platform",
"scope": ["refunds:read", "orders:read"],
"expiry": "2026-12-17"
}
```

Example (illustrative) Each agent's registry entry expires after 90 days; an owner who does not renew loses the agent's workload identity, so abandoned agents fall out of production automatically.

Consequences

Attribution, scope enforcement and lifecycle control become possible, and every other layer gets an object to anchor to. The cost is pipeline integration and the governance to enforce the expiry.

Related patterns

Agent Identity & Scoped Credentials; AIBOM; Shadow-AI Discovery; Kill Switch / Circuit Breaker.

Maps to: EU AI Act Art. 49/71, Art. 11 · ISO/IEC 42001 · NIST AI RMF (Map) · CSA AICM · OWASP Agentic ASI10 · Layer 02 Inventory & Transparency.

Pattern: AIBOM

Summary — Generate an AI bill of materials at build for each AI system, recording models, datasets, weights and their provenance in a standard format, and store it with the registry entry. The AIBOM is what the transparency and eval layers read to know what to document and what to test.

Objectives

Make the composition and provenance of an AI system machine-readable, so supply-chain risk and transparency obligations can be answered from an artefact, not reconstructed.

Target users

AI governance engineer, ML engineer, security engineer.

Impacted stakeholders

Model owners, downstream deployers, auditors, procurement.

Relevant principles

Instrument the build to produce its own proof; start from a named failure mode or harm.

Context

AI systems assembled from foundation models, fine-tunes, third-party datasets and libraries, where the classic SBOM captures software dependencies but not models or data.

Problem

Without a bill of materials for models and data, an organisation cannot answer which model version, from which provenance, trained on which data, is inside a given system — so it cannot assess supply-chain risk or produce transparency documentation on demand.

Solution

Emit an AIBOM at build in a standard format — CycloneDX ML-BOM or the SPDX 3.0 AI profile, for example with the OWASP AIBOM generator [14] (illustrative) — covering models, datasets, weights, and their provenance and licences. Attach it to the registry entry and regenerate it on each build so it never drifts from the deployed system.

Example (illustrative) A retrieval-augmented assistant's AIBOM lists the base model, the embedding model, the corpus snapshot and their licences; when a corpus licence changes, the diff surfaces in the next build's AIBOM.

Consequences

Supply-chain and provenance questions become queries; transparency documents can be generated from the AIBOM. The cost is toolchain integration and keeping provenance metadata accurate.

Related patterns

Agent Registry; Model Card as Control Evidence; Machine-Readable Evidence (OSCAL).

Maps to: EU AI Act Art. 11, Art. 53 (GPAI documentation) · ISO/IEC 42001 · NIST AI RMF (Map) · CSA AICM · Layer 02 Inventory & Transparency.

Pattern: Model Card as Control Evidence

Summary — Treat the model card and data card not as launch documentation written once, but as structured evidence regenerated from the pipeline, so that transparency documents describe the system as it is now and feed the assurance layer.

Objectives

Convert transparency documentation from a static PDF into a versioned artefact that is both human-readable and machine-consumable, and that counts as control evidence.

Target users

AI governance engineer, ML engineer, DPO.

Impacted stakeholders

Model owners, users, auditors, data subjects.

Relevant principles

Instrument the build to produce its own proof; make the governed path the easiest path.

Context

A system subject to transparency obligations, where cards are traditionally written at launch and never touched again.

Problem

A model card written once decays into fiction as the model, prompts and datasets change. A card that is not regenerated cannot be trusted as evidence and misleads the very auditor it was meant to satisfy.

Solution

Template the card and populate it from the pipeline: intended use, evaluation results (from the Eval Gate), datasets (from the AIBOM), known limitations and owner. Regenerate on each significant change and version it with the model. Store the card as structured data so it can be both read by a person and consumed by the assurance layer.

Example (illustrative) A classifier's card is rebuilt on every deploy, pulling its latest fairness-eval scores and dataset provenance automatically, so the card an auditor reads is the card production produced.

Consequences

Transparency stays true and doubles as evidence. The cost is templating and pipeline wiring, and agreeing what "significant change" triggers a regeneration.

Related patterns

AIBOM; Eval Gate in CI; Machine-Readable Evidence (OSCAL); FRIA-as-Code.

Maps to: EU AI Act Art. 11, Art. 13 (transparency) · ISO/IEC 42001, ISO/IEC 42005 · NIST AI RMF (Map, Measure) · Layer 02 Inventory & Transparency.

Pattern: Continuous Assurance Telemetry

Summary — Stream control decisions — policy verdicts, eval results, guardrail actions, identity events — into an assurance store as they happen, so the status of a control is a live query rather than a point-in-time attestation. Trustworthiness becomes a continuously generated signal, not a static certificate [5].

Objectives

Replace periodic attestation with evidence emitted as the system runs, so "is the control working?" is answered by telemetry.

Target users

AI governance engineer, SRE/platform team, auditor.

Impacted stakeholders

Model owners, auditors, regulators, incident responders.

Relevant principles

Instrument the build to produce its own proof; give every control teeth.

Context

A stack whose lower layers already emit structured records, and an assurance function tired of assembling binders before each audit.

Problem

An attestation says a control existed when someone looked; it says nothing about the weeks in between, during which the model was retrained and an agent gained a tool. Point-in-time evidence decays immediately.

Solution

Have each control write a timestamped, structured record to a common assurance store, on one schema. Fix the schema first: a minimum useful evidence record is `{control_id, subject (model/agent/system id + version from the registry), decision (pass/fail/allow/deny/alert), metric + value + threshold, failure_mode/obligation ref, input_hash, actor, timestamp, signature}`. Normalise every tool's output into that shape on ingest, so heterogeneous sources compose into one queryable store keyed on the registry id.

Illustrative schema for the evidence record:

```
{
  "control_id": "guardrail.output.pii.v2",
  "subject": "csa-01@2026-09-18",
  "decision": "alert",
  "metric": "pii_leak_rate", "value": 0.004, "threshold": 0.0,
  "obligation": "EU AI Act Art. 15",
  "input_hash": "sha256:1c7d...",
  "actor": "csa-01",
  "timestamp": "2026-09-18T14:31:52Z",
  "signature": "ed25519:5a..."
}
```

Two research proposals point at the same idea and are worth watching, not adopting whole: TAIP treats NIST TEVV outputs as reusable AI Assurance Objects that compose across systems [5], and AAGATE operationalises a control plane aligning the NIST AI RMF functions for agents in production [6]; both are single preprints. Expose the current status of each control as a query over the store.

Example (illustrative) A dashboard tile for the data-residency control is backed by a live query over emitted guardrail decisions; if the control stops firing, the tile goes red within minutes, not at the next audit.

Consequences

The audit becomes a query and drift is visible in near real time. The cost is building the pipeline and storage, and defining a common evidence schema across tools.

Related patterns

Machine-Readable Evidence (OSCAL); Eval Gate in CI; Runtime Guardrail; Incident Pipeline; Kill Switch / Circuit Breaker.

Maps to: EU AI Act Art. 72 · ISO/IEC 42001 · NIST AI RMF (Manage, Govern) · CSA AICM · Layer 05 Assurance & Continuous Compliance.

Pattern: Runtime Guardrail

Summary — Place input and output guardrails on the model or agent's runtime path that enforce its Policy Card on every live request and emit a decision to telemetry and to the circuit breaker. The guardrail is where a policy written in layer 01 and a threshold tested in layer 03 become an action taken on a real call, not a claim about one.

Objectives

Enforce policy at the point of action, on inputs and outputs no eval anticipated, and make each enforcement a structured event the assurance and incident layers can consume.

Target users

AI governance engineer, ML engineer, security engineer, platform team.

Impacted stakeholders

Users, model owners, affected persons, incident responders, auditors.

Relevant principles

Give every control teeth; instrument the build to produce its own proof.

Context

An agent or model in production, acting on live inputs, whose Policy Card and eval thresholds exist but have no runtime enforcement point — so a rule proven in CI is unguarded the moment the system meets an input no test covered.

Problem

Policies and evals are point-in-time; the system then meets prompt injection, unsafe outputs and tool calls no one reviewed. A guardrail that only logs is observability mistaken for control — the system watches itself fail in high resolution. Without an enforcement point that can block and emit, the runtime is the gap between a tested control and an uncontrolled action.

Solution

Put a guardrail on both sides of the model/agent path. The input guardrail screens prompts and retrieved context for injection and policy-violating requests before they reach the model; the output guardrail screens generations and tool calls for unsafe content, data leakage and out-of-scope actions before they take effect. Enforce the same Policy Card evaluated in CI [2], so the runtime decision and the pipeline decision share one rule. On every call emit a structured event — `{agent, direction (input/output), rule_id, decision (allow/block/redact), timestamp}` — to the assurance store (Continuous Assurance Telemetry) and, on a defined breach, signal the circuit breaker (Kill Switch / Circuit Breaker). Guardrail frameworks realise this as a category; the OWASP Agent Control Standard names the runtime-control surface [11]. This is distinct from the kill switch: the guardrail decides one call at a time and stays in the request path; the breaker withdraws the agent's autonomy wholesale when the guardrail's signals cross a threshold.

Example (illustrative) A customer-service assistant's input guardrail blocks a prompt-injection attempt and its output guardrail redacts an account number the model was about to return; both decisions are emitted to the assurance store, and a burst of blocks trips the circuit breaker.

Consequences

The tested control holds on live traffic and every enforcement leaves evidence; the guardrail is also the sensor the breaker and the incident pipeline read. The cost is per-call latency, false positives to tune, and keeping the runtime rule in sync with the Policy Card and eval thresholds.

Related patterns

Policy Card; Kill Switch / Circuit Breaker; Agent Identity & Scoped Credentials; Continuous Assurance Telemetry; Human-in-the-loop Gate; Eval Gate in CI.

Maps to: EU AI Act Art. 14, Art. 15 · ISO/IEC 42001 · NIST AI RMF (Manage) · OWASP Agentic ASI02/ASI03 · Layer 04 Runtime Controls & Observability.

Pattern: Kill Switch / Circuit Breaker

Summary — Provide a tested mechanism to stop an agent or class of agents at the point of action — revoking access and halting tool calls — without breaking the rest of the fleet. Autonomy is granted only where it can be withdrawn.

Objectives

Bound the blast radius of a misbehaving or compromised agent, and make "stop it" a control that has been exercised, not a claim.

Target users

AI governance engineer, security engineer, SRE.

Impacted stakeholders

Model owners, users, incident responders, affected third parties.

Relevant principles

Register and bound every actor before it acts; start from a named failure mode or harm.

Context

Agents that act autonomously — calling tools, moving data or money — where a single failure can cascade. Gartner expects that by 2029 more than half of successful attacks on AI agents will exploit access-control weaknesses and prompt injection [17].

Problem

An agent that cannot be stopped precisely can only be stopped by breaking everything. A fleet on shared credentials means an incident forces a choice between leaving the agent running and rotating a secret that halts the whole fleet.

Solution

Bind each agent to its own identity (see Agent Identity & Scoped Credentials) so access can be revoked per agent. Implement a circuit breaker at the tool-call boundary that trips on a defined signal — a threshold breach, an anomaly, a manual pull. Test the kill switch on a schedule; an untested kill switch is not a control.

Example (illustrative) A payments agent's circuit breaker trips automatically when its unauthorised-tool-call rate crosses a threshold, revoking just that agent's scope while the rest of the fleet keeps running; the pull is drilled monthly.

Consequences

Incidents are contained to one agent and recovery is fast. The cost is per-agent identity plumbing and the engineering to make revocation instant and safe.

Related patterns

Agent Identity & Scoped Credentials; Agent Registry; Human-in-the-loop Gate; Incident Pipeline.

Maps to: EU AI Act Art. 14, Art. 15 · ISO/IEC 42001 · NIST AI RMF (Manage) · CSA AICM · OWASP Agentic ASI02/ASI10 · Layer 04 Runtime Controls & Observability.

Pattern: Incident Pipeline

Summary — Build the plumbing to detect, triage and report serious AI incidents on the clock, with reporting timelines and templates encoded rather than remembered. For high-risk systems this includes EU AI Act Article 73 serious-

incident reporting; for GPAI models it includes the systemic-incident reporting the Code of Practice expects.

Objectives

Turn a runtime signal into a reported obligation within the legal window, and produce the incident record as structured evidence.

Target users

AI governance engineer, security/incident responder, legal/compliance.

Impacted stakeholders

Regulators, affected persons, deployers, model owners.

Relevant principles

Instrument the build to produce its own proof; start from a named failure mode or harm.

Context

A high-risk or GPAI system in production, subject to serious-incident reporting duties, where detection lives in engineering and reporting lives in legal, with no wire between them.

Problem

When an incident is detected, the clock starts. If detection, triage and reporting are disconnected manual steps, the deadline is missed and the evidence of what happened is reconstructed after the fact.

Solution

Connect runtime detection (from observability and guardrails) to a triage workflow that classifies severity and, on a reportable event, drafts the report against the required template and starts the statutory timer. Encode the EU AI Act Art. 73 timelines and the Art. 72 post-market monitoring feed [12]; hold the incident record as machine-readable evidence.

Example (illustrative) A guardrail flags a data-exfiltration attempt via an agent tool; the pipeline classifies it, opens an incident with the Art. 73 timer running, and pre-fills the report from the trace and registry entry.

Consequences

Reporting happens on time and the record is audit-ready. The cost is cross-functional integration and keeping the severity criteria and templates current with the law.

Related patterns

Continuous Assurance Telemetry; Runtime Guardrail; Kill Switch / Circuit Breaker; Machine-Readable Evidence (OSCAL).

Maps to: EU AI Act Art. 72, Art. 73, Art. 55 (GPAI) · ISO/IEC 42001 · NIST AI RMF (Manage) · Layer 05 Assurance & Continuous Compliance.

Pattern: FRIA-as-Code

Summary — Maintain the fundamental-rights impact assessment as a versioned, reviewable artefact cross-referenced to the data-protection impact assessment, so that a rights assessment is an input to design that updates with the system, not a document produced once and filed.

Objectives

Keep the FRIA and its DPIA cross-reference live and linked to the controls they demand, so a change in the system triggers a review of its rights impact.

Target users

AI governance engineer, DPO, legal/compliance.

Impacted stakeholders

Data subjects, affected persons, deployers, regulators.

Relevant principles

Start from a named failure mode or harm; instrument the build to produce its own proof.

Context

A deployer of a high-risk AI system subject to the EU AI Act Art. 27 fundamental-rights impact assessment, where a DPIA under GDPR Art. 35 may already exist and overlap.

Problem

A FRIA written once as a Word document describes rights impact at a single moment and is never revisited when the system changes. Duplicated effort between FRIA and DPIA wastes work and leaves the two out of sync.

Solution

Template the FRIA as structured data covering intended use, affected groups, risks to rights, and the mitigating controls — each mitigation linked to the control that implements it (a Policy Card, an Eval Gate, a guardrail). Cross-reference the DPIA so shared elements are written once. Store it with the registry entry and re-open it on significant change. The EU AI Act Art. 27 FRIA and its DPIA cross-reference define the scope [13].

Example (illustrative) A benefits-eligibility system's FRIA links each identified risk to people's rights to a specific eval and guardrail; when the model is retrained, the FRIA flags which mitigations need re-verification.

Consequences

Rights assessment stays current and traceable to controls, and overlaps with the DPIA are not duplicated. The cost is templating and the discipline to treat the assessment as living.

Related patterns

Model Card as Control Evidence; Policy Card; Runtime Guardrail; Machine-Readable Evidence (OSCAL); Human-in-the-loop Gate.

Maps to: EU AI Act Art. 27 (FRIA), Art. 9 · GDPR Art. 35 (DPIA) · ISO/IEC 42005 · NIST AI RMF (Map) · Layer 01 Govern-as-Code / Layer 02 Inventory & Transparency.

Pattern: Framework Crosswalk

Summary — Maintain a mapping from each control to the framework clauses it serves, generated from the controls themselves, as an index for navigation and reuse — never as the end state. A crosswalk proves you have read the framework; it does not prove the control fires.

Objectives

Let one control satisfy many frameworks and make coverage navigable, while refusing to mistake coverage for assurance.

Target users

AI governance engineer, compliance lead, auditor.

Impacted stakeholders

Auditors, regulators, model owners.

Relevant principles

Make the governed path the easiest path; instrument the build to produce its own proof.

Context

An organisation answering to several overlapping frameworks — EU AI Act, ISO/IEC 42001, NIST AI RMF, CSA AICM — that would otherwise implement the same control several times.

Problem

The crosswalk is where framework theatre begins. A green mapping matrix is mistaken for a working control; a 300-row spreadsheet mapping controls to five frameworks is presented as maturity while nothing measures whether any mapped control reduces risk. **The anti-pattern is treating coverage as control: a mapping cell is not evidence.**

Solution

Generate the crosswalk from the controls, not beside them: each Policy Card and Eval Gate declares the clauses it maps to, and the crosswalk is the aggregation. Use it to find gaps and reuse controls, not to report compliance. Every mapping cell must resolve to a running control and its emitted evidence; a cell with no evidence behind it is flagged, not counted. Reference vocabularies such as the CSA AICM (247 control objectives across 18 domains) [15] and the OWASP Agent Control Standard [11] anchor the mapping.

Example (illustrative) Clicking a green cell for "logging" opens the guardrail and the OSCAL evidence it emitted this week; a cell with no evidence renders amber, not green.

Consequences

Controls are reused across frameworks and gaps are visible, without inflating a matrix into false assurance. The trade-off is the discipline to keep cells honest and to resist reporting coverage as outcome.

Related patterns

Policy Card; Runtime Guardrail; Machine-Readable Evidence (OSCAL); Continuous Assurance Telemetry.

Maps to: EU AI Act (cross-cutting) · ISO/IEC 42001 · NIST AI RMF (Govern) · CSA AICM · OWASP Agent Control Standard · Layer 01 Govern-as-Code / Layer 05 Assurance & Continuous Compliance.

Pattern: Machine-Readable Evidence (OSCAL)

Summary — Emit control evidence in a machine-readable, standard format so that the audit is a query and the same evidence feeds continuous assurance. OSCAL, extended with properties for AI, is the organising format: frameworks specify what to assure but provide no executable format for how, and this pattern supplies it [4].

Objectives

Make evidence queryable, diffable and aggregatable, and eliminate the screenshot as an evidence artefact.

Target users

AI governance engineer, auditor, platform team.

Impacted stakeholders

Auditors, regulators, model owners.

Relevant principles

Instrument the build to produce its own proof; give every control teeth.

Context

A stack whose controls already produce structured records, and an assurance function that must answer auditors repeatedly and at speed.

Problem

Evidence a human must format and file by hand does not scale, cannot be verified quickly, and is out of date the moment it is saved. Every audit re-collects it from scratch.

Solution

Emit control results as OSCAL component-definition and assessment-results artefacts. OSCAL's native model — a control layer (`catalog` , `profile`), an implementation layer (`component-definition` , `system-security-plan`) and an assessment layer (`assessment-plan` , `assessment-results` , `POA&M`), with traceability from a result back to the control it tested [21] — is the stable substrate; build on it first. AI-specific extensions are still forming: one proposed approach, a single 2026 preprint, adds sixteen property extensions for lifecycle phase, enforcement semantics and risk traceability in

a three-layer policy/evidence/enforcement architecture that generates OSCAL assessment results automatically and validates them against the NIST JSON schema [4]. Adopt the extensions if they fit, but the native assessment models carry most of the load today. Store the evidence so an auditor's question is answered by a query.

Example (illustrative) An eval gate writes an OSCAL assessment result on every run; the auditor's request for "all robustness evidence in Q3" is a filter over the store, returned in minutes.

Consequences

The audit becomes a query and evidence composes across tools and jurisdictions. The cost is adopting the schema and instrumenting controls to emit it.

Related patterns

Continuous Assurance Telemetry; Framework Crosswalk; Eval Gate in CI; Incident Pipeline.

Maps to: EU AI Act Art. 12, Art. 17, Art. 72 · ISO/IEC 42001 · NIST AI RMF (Manage, Govern) · Layer 05 Assurance & Continuous Compliance.

Pattern: Agent Identity & Scoped Credentials

Summary — Give every agent its own identity, an owner, a bounded scope and an expiry, established before it acts, so that its actions can be attributed, its access revoked precisely and its scope contained. Identity is the precondition of accountability; scope is the precondition of containment.

Objectives

Make every non-human actor governable by construction: attributable, scorable, revocable, expiring.

Target users

AI governance engineer, security engineer, IAM/platform team.

Impacted stakeholders

Model owners, security operations, auditors, affected third parties.

Relevant principles

Register and bound every actor before it acts; start from a named failure mode or harm.

Context

Agents that act under delegated authority — calling APIs, tools and other agents — where the default is a shared service account or a static key.

Problem

An agent on borrowed credentials cannot be attributed, contained or revoked. NIST's NCCoE frames the open question directly: how do identification, authentication and authorization apply so each agent is "known, trusted, and properly governed", with non-repudiation and tamper-proof logging [10].

Solution

Issue each agent a distinct workload identity with a declared scope, an owner and an expiry, recorded in the Agent Registry. Keep two questions separate. **Channel authentication** secures one hop — how a client authenticates to a tool server; the MCP specification of 2026-07-28 tightened exactly this, deprecating Dynamic Client Registration in favour of Client ID Metadata Documents and binding credentials to their issuer [7]. That hardens the MCP connection but is not the agent's identity. **Agent workload identity** is the durable, attributable identity the agent carries across every hop and protocol, under which its actions are logged and its access revoked — the job of a workload-identity system (SPIFFE/SPIRE) or a first-class agent identity from an enterprise provider (for example Microsoft Entra Agent ID [8] or Okta Agent SSO [9] — illustrative), recorded in the registry, not of the transport protocol. Secure the channel *and* issue the workload identity; scope its credentials to the least privilege the agent's declared function needs.

Example (illustrative) A data-analysis agent authenticates with an issuer-bound credential and a scope limited to read-only access to one dataset; its every action is logged under its own identity, and its credential expires with its registry entry.

Consequences

Attribution, containment and precise revocation become possible, and the kill switch has something to act on. The cost is IAM integration and managing non-human identities at scale.

Related patterns

Agent Registry; Kill Switch / Circuit Breaker; Policy Card; Human-in-the-loop Gate.

Maps to: EU AI Act Art. 12, Art. 14, Art. 15 · ISO/IEC 42001 · NIST AI RMF (Manage) · CSA AICM · OWASP Agentic ASI03 · Layer 04 Runtime Controls & Observability.

Pattern: Human-in-the-loop Gate

Summary — Require human approval at a defined, high-consequence decision point before an agent's action takes effect, so that autonomy is bounded by a person exactly where the stakes justify the latency. Oversight is a designed checkpoint, not an afterthought.

Objectives

Insert meaningful human oversight where an action is irreversible or high-impact, and record the decision as evidence.

Target users

AI governance engineer, product owner, risk owner.

Impacted stakeholders

Affected persons, users, model owners, regulators.

Relevant principles

Start from a named failure mode or harm; register and bound every actor before it acts.

Context

An agent whose actions include some that are irreversible or affect people's rights — a payment, a denial, a publication — alongside many that are routine.

Problem

Full autonomy over a high-consequence action removes the human oversight the law and the risk both require; full manual review over every action destroys the value of the agent. Undifferentiated oversight fails in both directions.

Solution

Classify actions by consequence. For the high-consequence class, gate the action behind a human approval step with enough context to decide, and block the action until approval. Log the approver, the context and the decision as evidence. Keep the routine class autonomous under guardrails. This realises the EU AI Act Art. 14 human-oversight requirement [19] at the point of action.

Example (illustrative) An agent can draft and queue refunds autonomously, but any refund above a threshold is held for a named human approver, whose decision is logged against the transaction.

Consequences

Oversight lands where it matters without throttling routine work, and the approval is auditable. The cost is designing the consequence classification and the latency it adds to gated actions.

Related patterns

Kill Switch / Circuit Breaker; Runtime Guardrail; Policy Card; Agent Identity & Scoped Credentials; FRIA-as-Code.

Maps to: EU AI Act Art. 14 · ISO/IEC 42001 · NIST AI RMF (Manage) · OWASP Agentic ASI02 · Layer 04 Runtime Controls & Observability.

Pattern: Shadow-AI Discovery

Summary — Continuously discover AI systems and agents that are running but not registered, and reconcile them against the registry, so the inventory reflects reality rather than only what teams remembered to declare. You cannot govern what you cannot see.

Objectives

Close the gap between the registry and production by finding unregistered models, agents and AI-enabled tools, and bringing them under governance.

Target users

AI governance engineer, security engineer, platform team.

Impacted stakeholders

Model owners, security operations, auditors.

Relevant principles

Register and bound every actor before it acts; make the governed path the easiest path.

Context

An organisation where teams adopt AI tools and spin up agents faster than any central inventory can track, and where much of the AI-governance platform category "manages the program ... without any runtime data path" [20].

Problem

A registry fed only by voluntary declaration is always behind. Unregistered agents — the shadow fleet — are exactly the layer a paper inventory cannot see, and roughly one in eight reported AI breaches now involves an autonomous agent [16] (reported).

Solution

Run discovery against the environments where AI appears — identity providers, cloud accounts, network egress, code repositories, SaaS integrations — using discovery tooling (illustrative) to find models and agents. Reconcile findings against the registry, open an entry for each unknown with an owner to claim it, and escalate the unclaimed. Feed the result back into the Agent Registry's drift check.

Example (illustrative) A weekly discovery sweep finds an agent calling an external API from a team's cloud account with no registry entry; it is auto-registered as unclaimed, its owner is notified, and its scope is frozen until claimed.

Consequences

The inventory converges on reality and the blind spot shrinks. The cost is discovery integration and the process to triage and claim what it finds.

Related patterns

Agent Registry; Agent Identity & Scoped Credentials; Continuous Assurance Telemetry.

Maps to: EU AI Act Art. 49/71 · ISO/IEC 42001 · NIST AI RMF (Map) · CSA AICM · OWASP Agentic ASI10 · Layer 02 Inventory & Transparency.

Pattern: Vendor / Model Due-Diligence Gate

Summary — Gate the procurement or integration of a third-party AI system — SaaS with an embedded LLM, an API-only foundation model, a vendor's agent — on a structured due-diligence assessment, so a model you do not own still

enters through a control that records what you can and cannot verify about it. When you do not own the model, this gate is what replaces the red-team you cannot run.

Objectives

Bring bought and API-only AI under the same registry and assurance discipline as systems you build, and make the limits of your verification explicit rather than assumed away.

Target users

AI governance engineer, procurement, security engineer, DPO.

Impacted stakeholders

Deployers, model providers, data subjects, auditors, regulators.

Relevant principles

Register and bound every actor before it acts; start from a named failure mode or harm.

Context

An organisation that consumes far more AI than it trains: SaaS features with an embedded LLM, hosted foundation models reached only by API, agents shipped inside a vendor's product. The weights, training data and internal guardrails belong to someone else.

Problem

The parts of the stack that assume you own the model degrade when you do not. You cannot red-team weights you cannot reach, so an eval gate (layer 03) can only test the vendor's system as a black box at its boundary; runtime control (layer 04) narrows to the tool scopes, identity and traffic the integration exposes, not the model's own behaviour. Left ungoverned, procured AI becomes the shadow fleet with a contract — in production, unassessed, and outside the registry.

Solution

Make due diligence a gate a procured or integrated AI system must pass before it reaches production, and structure the assessment on a template rather than an ad-hoc questionnaire — the CSIRO Responsible AI Pattern Catalogue's supplier-assessment fields are a usable starting point [1]. Assess, at minimum: the provider's own evaluations and red-team evidence (what they will share, and its independence); the model card, supplier documentation and any AIBOM you can obtain; the lawful basis and data flows, including whether your inputs train their model; the tool scopes and identity you will grant the vendor's agent; the provider's incident-reporting commitments; and the contractual right to audit and to be notified of material change. Record the result as a registry entry with an owner and a scope, and re-open the gate on renewal or on a material model change. Anchor the assessment in ISO/IEC 42001 Annex A.10 (third-party and customer relationships) [22], the EU AI Act's allocation of duties along the value chain — provider obligations versus deployer obligations under Articles 25, 26 and 27 [23] — and, for general-purpose models, the transparency and documentation the GPAI Code of Practice expects providers to supply [24]. Where you cannot verify a control, record that you cannot, and compensate by bounding the integration: least-privilege scopes, boundary evals, and tighter runtime observation of the traffic you do control.

Example (illustrative) A team integrating an API-only foundation model cannot test its weights, so the gate captures the provider's published evaluations, restricts the model to a scoped service identity with no standing data access, adds a boundary eval on the team's own prompts, and files the whole assessment as the system's registry entry — flagged "provider-attested" where the team relied on the vendor's evidence rather than its own.

Consequences

Procured AI is inventoried and bounded, and the reliance on provider-supplied evidence is explicit rather than hidden. The cost is real: layers 03 and 04 give less assurance over a model you do not own, and the gate depends on provider cooperation and contract terms you may not fully win.

Related patterns

Agent Registry; AIBOM; Adversarial Red-Team Suite; Model Card as Control Evidence; Agent Identity & Scoped Credentials; Shadow-AI Discovery.

Maps to: EU AI Act Art. 25 (value-chain responsibilities), Art. 26 (deployer duties), Art. 27 (FRIA), Art. 53 (GPAI documentation) · ISO/IEC 42001 Annex A.10 · GPAI Code of Practice · NIST AI RMF (Map, Govern) · Layer 02 Inventory & Transparency / Layer 05 Assurance & Continuous Compliance.

Sources

[1] Responsible AI Pattern Catalogue (template: summary, objectives, target users, impacted stakeholders, relevant principles, context, problem, solution, consequences; governance/process/product patterns). CSIRO. 2023–2024. <https://research.csiro.au/ss/science/projects/responsible-ai-pattern-catalogue/> (verified: primary) [2] Policy Cards: machine-readable, deployment-layer governance artefacts for AI agents, linked to enforcement and audit pipelines (arXiv 2510.24383). 2025–10. <https://arxiv.org/abs/2510.24383> (verified: primary) [3] Top 10 for Agentic Applications 2026 (ASI IDs). OWASP GenAI Security Project. 2025–12–09. <https://genai.owasp.org/resource/owasp-top-10-for-agentic-applications-for-2026/> (verified: primary) [4] "Making AI Compliance Evidence Machine-Readable" (OSCAL + 16 property extensions; three-layer policy/evidence/enforcement; "specify what to assure but provide no executable format for how") (arXiv 2604.13767). UC3M. 2026–04–15. <https://arxiv.org/abs/2604.13767> (verified: primary) [5] TAIP: NIST TEVV outputs as reusable AI Assurance Objects; trustworthiness as a continuously generated signal (arXiv 2603.03340; submitted 15 Feb 2026). 2026–02. <https://arxiv.org/abs/2603.03340> (verified: primary) [6] AAGATE: NIST AI RMF-aligned, Kubernetes-native governance control plane for agentic AI (arXiv 2510.25863). 2025–10. <https://arxiv.org/abs/2510.25863> (verified: primary) [7] Model Context Protocol specification 2026-07-28 (DCR deprecated in favour of CIMD; issuer-bound credentials). MCP. 2026-07-28. <https://blog.modelcontextprotocol.io/posts/2026-07-28/> (verified: primary) [8] Microsoft Entra Agent ID (first-class agent identity; OAuth 2.0, MCP, A2A). Microsoft Learn. 2026-04. <https://learn.microsoft.com/en-us/entra/agent-id/what-is-microsoft-entra-agent-id> (verified: primary) [9] "Okta brings first-class identity to AI agents with Agent SSO" (GA 24 Aug 2026; Cross App Access as MCP EMA extension). Okta. 2026-08-24. <https://www.okta.com/newsroom/press-releases/okta-brings-first-class-identity-to-ai-agents-with-agent-sso/> (verified: primary) [10] "Accelerating the Adoption of Software and AI Agent Identity and Authorization" (concept paper; "known, trusted, and properly governed"; non-repudiation, tamper-proof logging). NIST NCCoE. 2026-02-05. <https://www.nccoe.nist.gov/news-insights/new-concept-paper-identity-and-authority-software-agents> (verified: primary) [11] OWASP GenAI Security Project unveils the Agent Control Standard (ACS) and 2026 Top 10 for LLM Applications. OWASP GenAI Security Project. 2026-09-01. <https://genai.owasp.org/2026/09/01/owasp-genai-security-project-unveils-2026-top-10-for-llm-applications-new-agent-control-standard-and-sponsors-as-community-tops-30000-members/> (verified: primary) [12] EU AI Act Art. 72 (post-market monitoring) and Art. 73 (reporting of serious incidents). artificialintelligenceact.eu. Reg. (EU) 2024/1689. <https://artificialintelligenceact.eu/article/73/> (verified: primary) [13] EU AI Act Art. 27 (fundamental-rights impact assessment for high-risk AI systems; DPIA cross-reference). artificialintelligenceact.eu. Reg. (EU) 2024/1689. <https://artificialintelligenceact.eu/article/27/> (verified: primary) [14] "Evolving AI Transparency: the AIBOM generator's new home at OWASP" (CycloneDX output). OWASP GenAI Security Project. 2025–12–18. <https://genai.owasp.org/2025/12/18/evolving-ai-transparency-the-journey-of-the-aibom-generator-and-its-new-home-at-owasp/> (verified: primary) [15] AI Controls Matrix (AICM) v1.1 (247 control objectives across 18 domains). Cloud Security Alliance. 2026-06-22. <https://cloudsecurityalliance.org/artifacts/ai-controls-matrix-v1-1> (verified: primary) [16] Threat Report 2026 (~1 in 8 reported AI breaches involve autonomous agents). HiddenLayer. 2026. <https://www.hiddenlayer.com/report-and-guide/threatreport2026> (verified: reported) [17] "Gartner Forecasts the Market for Securing AI Will Reach Almost \$5 Billion in 2027" (>50% of agent attacks exploit access-control and prompt injection by 2029). Gartner. 2026-08-26. <https://www.gartner.com/en/newsroom/press-releases/2026-08-26-gartner-forecasts-the-market-for-securing-ai-will-reach-almost-5-billion-in-2027> (verified: primary) [18] AI Risk Management Framework (AI RMF 1.0; Govern, Map, Measure, Manage). NIST. 2023-01-26. <https://www.nist.gov/itl/ai-risk-management-framework> (verified: primary) [19] EU AI Act Art. 14 (human oversight of high-risk AI systems). artificialintelligenceact.eu. Reg. (EU) 2024/1689. <https://artificialintelligenceact.eu/article/14/> (verified: primary) [20] "Best AI Governance Platforms 2026" (most of the category "manages the program ... without any runtime data path"). Kosmoy. 2026. <https://www.kosmoy.com/resources/blog/best-ai-governance-platforms-2026/> (verified: secondary) [21] OSCAL native model (control layer: catalog, profile; implementation: component-definition, system-security-plan; assessment: assessment-plan, assessment-results, POA&M). NIST. 2026. <https://pages.nist.gov/OSCAL/learn/concepts/layer/> (verified: primary) [22] ISO/IEC 42001:2023 Annex A.10 (third-party and customer relationships; supplier controls). ISO/IEC. 2023. <https://www.iso.org/standard/81230.html> (verified: secondary) [23] EU AI Act Arts. 25 (value-chain responsibilities), 26 (deployer obligations), 27 (FRIA) — allocation of duties between provider and deployer. AI Act (Reg. (EU) 2024/1689). 2024. <https://artificialintelligenceact.eu/article/25/> (verified: primary) [24] GPAI Code of Practice (published 10 Jul 2025; voluntary; transparency documentation providers

supply to downstream deployers). European Commission / AI Act Explorer. 2025-07-10. <https://artificialintelligenceact.eu/introduction-to-code-of-practice/> (verified: primary) [25] MITRE ATLAS: adversarial tactics and techniques knowledge base for AI systems (incl. agents). MITRE. 2026. <https://atlas.mitre.org/> (verified: primary)

06. The role

The AI governance engineer as a concrete role: a capability first and a job title second, defined by the workflows it owns and the evidence it produces, not by the certifications on its holder.

Capability first, title second

The Thesis insists that AI governance engineering is a capability, not a job title — the same claim the parent discipline makes for GRC engineering, a capability anyone close to the build can develop [11]. This chapter makes it concrete without contradicting that: a capability still lives in someone's week — the tickets they own, the pipelines they maintain, the incidents they are paged for. So we describe the **AI governance engineer** as the person, on whatever org chart, who holds that capability and is accountable for the three questions in production: what AI is running, what it is allowed to do, and what evidence proves it.

The distinction matters because the title is still forming. The same work is advertised as "AI Governance Engineer", "AI Risk Engineer", "AI Evaluation & Governance Engineer" [1], "Responsible AI Engineer" and, inside GRC teams building the AI-forward version of their function, "GRC Engineer" with an AI mandate [2]. A security engineer who writes the eval gate, a privacy engineer who turns a FRIA into code, an MLOps engineer who wires the registry to the deploy — each is doing AI governance engineering under a different title. We define the role by what it owns, not by what HR called the requisition.

One line separates it from its analyst neighbour, and the rest of the chapter earns that line: the AI governance analyst describes the system from the outside and files the description; the AI governance engineer reads the system directly and ships the control that changes what it does.

What the role owns, by workflow

The engineer owns workflows, not documents. Each workflow below is a running system with inputs, artefacts and evidence — and each maps to one of the five layers of the stack (chapter 04). Grouping responsibilities this way keeps the role honest: you own a workflow when you can be paged for it, not when your name is on a policy.

Intake and classification

Every AI system, model and agent enters through an intake that classifies it — by risk tier, by regulatory exposure (EU AI Act high-risk, GPAI, out of scope), by data sensitivity and by autonomy. The engineer builds intake as a form-plus-code path, not a meeting: a request that scaffolds a registry entry, triggers the right impact assessment (FRIA, DPIA), and routes the system to the controls its class requires. Live postings name exactly this — one VP role is scoped to "AI intake, classification and approval aligned to NIST AI RMF" [3]. **Maps to** Inventory & Transparency.

Inventory and registry

The engineer owns the inventory of models and the **agent registry** — the runtime-aware record of every non-human actor, each with an owner, a declared scope, a status and a kill switch. The capability that distinguishes the engineer here is the runtime data path: the registry is fed by the deployment pipeline and by discovery against production, not typed into a spreadsheet. Registry work appears verbatim in senior postings that ask for an "enterprise agent registry" and non-human identity governance [4]. **Maps to** Inventory & Transparency.

Evals and red teaming as evidence

The engineer builds and maintains the eval suites — capability, safety and adversarial — and wires them into an **eval gate** so a failing eval blocks the release. This is the workflow that most sharply separates the engineer from the analyst: the analyst reviews a model report; the engineer writes the test the model must pass and owns the harness that runs it (Inspect AI, promptfoo, Garak, Giskard, DeepEval, Ragas as category examples, illustrative not endorsed). The "AI Evaluation & Governance Engineer" role is defined around "automated testing harnesses and safety guardrails" and adversarial red-teaming [1]. **Maps to** Evals & Red Teaming as Evidence.

Policy-as-code and gates

The engineer expresses governance rules as executable policy (OPA/Rego, Cedar, Policy Cards) that evaluate in CI/CD and at admission, and maintains the gates that enforce them. A public GRC engineering-manager posting frames the mandate as "translate policies into policy-as-code" [5]. The engineer's output here is a merge that is blocked or allowed, with a logged reason — not a recommendation in a review. **Maps to** Govern-as-Code.

Runtime monitoring and incidents

The engineer instruments runtime: guardrail decisions, tool-call mediation, drift signals and agent behaviour stream into observability (Langfuse, Arize Phoenix over OpenTelemetry as examples). They own the detection-to-report path for serious incidents, including the EU AI Act's Article 73 clock for high-risk systems, and they own the tested **kill switch** for agents. Postings scope this as "threat modelling, red teams, MCP security" and "incident response for LLM systems" [4][6]. **Maps to** Runtime Controls & Observability.

Assurance and audit evidence

The engineer emits **audit-ready evidence** as a by-product of the build — **OSCAL** component and assessment artefacts, signed logs, structured eval results — so the audit is a query, not a project. This is **continuous assurance**: the control's status is a live signal, not a point-in-time attestation. **Maps to** Assurance & Continuous Compliance.

Regulatory translation

The engineer reads the obligation well enough to build the control that meets it: turning an AI Act article, an ISO/IEC 42001 control or a NIST AI RMF subcategory into a gate, a registry field or an evidence artefact — and back, so an auditor can trace the control to the obligation. This is translation, not legal advice; the engineer depends on Legal to confirm the obligation is read right. **Maps to** all five layers; it is the spine that chapter 08 indexes.

Skills, by workflow

Skills are grouped by the workflow they serve, not by the certificate that teaches them. The table is a capability map: it says what you must be able to do, in rough order of how load-bearing it is for each workflow. It is illustrative, not a checklist to pass.

Workflow	Core skills	Supporting skills
Intake & classification	Risk taxonomy design; reading the EU AI Act risk tiers; requirements analysis	Form/workflow tooling; light data modelling
Inventory & registry	Non-human identity and scoped access; API integration to CI/CD; data modelling	Cloud IAM; discovery tooling; SPIFFE/SPIRE concepts
Evals & red teaming	Eval harness engineering; adversarial prompting; statistical literacy; Python	LLM/agent internals; benchmark design; threat modelling (STRIDE/PASTA)
Policy-as-code & gates	OPA/Rego or Cedar; CI/CD pipeline engineering; Git	Policy schema design (Policy Cards); admission control
Runtime monitoring & incidents	Observability/OpenTelemetry; guardrail configuration; incident response	Detection engineering; MCP and agent-protocol security
Assurance & audit evidence	OSCAL and machine-readable evidence; logging and signing; audit fluency	Cryptographic attestation; evidence-store design
Regulatory translation	Reading regulation and standards (AI Act, ISO/IEC 42001, NIST AI RMF); mapping	Legal English; DPIA/FRIA methodology

Two cross-cutting skills sit under all seven: enough **Python** to glue systems together (postings put Python in roughly one in four AI-governance listings [7]), and enough **law-reading** to parse an article without mistaking it for advice. Neither is optional; neither is the whole job.

Analyst versus engineer

The clearest way to define the role is against the analyst it grows from. The contrast below is written for AI governance and modelled on the analyst-vs-engineer table that the GRC Engineer literature uses for its parent discipline [2]. Both roles are needed; the engineer is not "better", but does different work and is measured differently.

Dimension	AI governance analyst	AI governance engineer
Evidence artefact	A point-in-time artefact — an attestation, a questionnaire, an exported report — compiled for a review	A continuously emitted artefact — a query result, an eval run, a signed log — produced as the pipeline runs

Dimension	AI governance analyst	AI governance engineer
Primary source	Works from the system's reported description: documentation, summaries and vendor answers	Works from the running system: the registry and production telemetry, the same signals the build emits
Toolset	Spreadsheets, a GRC/AI-governance platform, ticketing	Python, OPA/Rego , Git, CI/CD, eval harnesses, OSCAL — plus the platform
Cadence	Periodic: quarterly reviews, annual assessments	Continuous: every commit, deploy and runtime call
Output	A report, a mapping matrix, a risk rating	A merged-or-blocked build, a registered agent, a machine-readable evidence artefact
Success metric	Audit passed, framework coverage complete	Control failures caught before the auditor arrives, and realised risk measurably reduced

The distinction is cadence and artefact, not competence or clearance. It is not that the analyst cannot read a registry or the engineer cannot write a report; it is that the analyst's deliverable is a periodic description and the engineer's is a continuous control. A good analyst reads systems closely; a good engineer writes clearly. The failure modes differ too, and naming them keeps both honest. The analyst's failure mode is compliance theatre: documentation that runs ahead of reality. The engineer's failure mode is over-engineering: automating a control for a process nobody agreed to fix, or building a gate so brittle that engineers route around it. Neither role is safe from its own failure mode by title alone.

The career ladder

The role has observable rungs, each defined by what the person can be trusted to own end to end, not by years served — and verifiable by looking at the systems, not a self-assessment.

1. **Associate.** Runs existing controls: adds an eval to a suite, registers an agent correctly, produces evidence from a control someone else built.
2. **AI governance engineer.** Builds a control end to end — one obligation into a gate, an eval suite for a system class, wired into CI/CD — and owns at least one workflow for a product area.
3. **Senior.** Owns a full workflow across the organisation and designs the paved path others adopt; the registry, eval gate or evidence pipeline they built is the standard template.
4. **Staff / principal.** Owns the reference architecture — how the five layers fit together — and the cross-cutting decisions (identity model, evidence format, incident path).
5. **Head of AI governance engineering.** Owns the function and its shared ownership with engineering, measured by realised risk reduction, not by controls stopped. A public engineering-manager posting scoped to "build an AI-forward GRC engineering function" sits at this rung [5].

A person can hold the capability at rung two while the title lags at "analyst", or hold the title without the capability. The ladder describes the work, and the work is visible in the systems.

Three ways in

Nobody starts as an AI governance engineer; everyone converts from an adjacent discipline, keeping its strength and adding what it lacks.

- **From Legal or privacy.** Your edge is regulatory translation; your gap is the build. Turn one assessment into a versioned, executable artefact (a FRIA-as-code template, a policy in [OPA/Rego](#)), and learn enough pipeline to see where the control fires. Start at policy-as-code and intake.
- **From Security or GRC.** Your edge is the control mindset; your gap is the model layer. GRC engineering already taught the parent moves — policy-as-code, continuous assurance, evidence as a by-product [2]; add the AI-specific objects: evals as controls, agent identity and scope, and the model and agent failure modes of the OWASP Agentic Top 10 [8]. Start at evals-as-evidence and the agent registry.
- **From MLOps or ML engineering.** Your edge is the runtime data path everyone else lacks; your gap is the obligation. Add the eval *gate* rather than the eval report, the registry field for owner and scope, the evidence artefact the audit needs. Start at eval gates in CI and runtime monitoring.

The market

The role is defined by the workflows above, not by the vacancies — but the market is forming and the evidence is public, and it corroborates the shape of the work. Treat every figure as sourced; salary points are survey medians or posting ranges, not guarantees.

Survey bands. The IAPP Salary & Jobs Report 2025-26 (1,600+ respondents, 60+ countries) puts technical AI-governance roles in the tech sector at a median of USD 221,000 — its highest band — against USD 151,800 for AI-governance work generally and USD 169,700 for combined privacy-and-AI- governance roles [9]. The premium is for the technical, build-the-control end of the discipline, which is exactly the role this chapter describes.

Skills in demand. Analysis of US postings since January 2026 (Axial Search) reports observability/monitoring in 41-42% of AI-governance listings, Python in 27-28%, NIST frameworks in about 27%, foundation-model familiarity in 25.6% and cloud in 18.2%; median advertised pay was USD 169,000 and median required experience five years [7]. And the demand signal is broad: LinkedIn's 2026 Skills on the Rise lists governance and responsible-AI skills among its fastest-rising clusters, alongside the technical AI capabilities [10]. The load-bearing signal is the skills mix — observability, Python, the runtime data path — not the salary headline.

Postings (footnote). Individual September 2026 postings corroborate the bands across seniorities — an "AI Governance Engineer" at USD 175,000-260,000 [4], a governance VP at USD 120,000-202,500 [3], another engineer role at USD 100,000-150,000 [6], and a GRC engineering-manager role at USD 405,000 [5] — but named vacancies are a lagging, noisy signal, kept here as corroboration and out of the argument. The role is the workflows, not the requisition.

What employers get wrong in the job description

Reading the postings against the workflows above, three mistakes recur.

- **Certifications as a proxy for capability.** Descriptions list AIGP, CIPP, CISSP and CISM as if a certificate produced a control. The Axial data shows certs appear in under 11% of postings each [7]; the load-bearing skills — eval harnesses, policy-as-code, the runtime data path — are the ones the JD under-specifies. Ask for the workflow, then the cert if it helps.
- **Analyst work under an engineer title.** Many "AI Governance Engineer" postings describe intake, mapping and reporting — analyst work — at engineer pay. The tell is the absence of any build: no eval gate, no registry integration, no evidence pipeline.
- **Everything, in one hire.** A single posting asks for policy-as-code, red teaming, identity, observability, incident response, regulatory translation and stakeholder management. That is a function, not a person: the seven workflows are owned across a team, and a first hire owns two or three and builds the paved path for the rest.

In practice In a large telco, the role arrived before the title. The first version of the work sat inside a privacy team and looked like DPIAs and reviews. What turned it into AI governance engineering was owning two workflows outright: wiring the model and agent registry to the deployment pipeline so it was true on any Tuesday, and putting an eval gate in CI so a regression in injection resistance failed the build. The job description caught up a year later. The capability was visible in the systems long before it was visible on the org chart.

Maps to: EU AI Act Art. 9 (risk management), Art. 26/27 (deployer duties, FRIA), Art. 72 (post-market monitoring), Art. 73 (serious-incident reporting) · ISO/IEC 42001 (roles, responsibilities and competence) · NIST AI RMF (Govern) · OWASP Top 10 for Agentic Applications 2026. Mappings are illustrative, not a claim of conformity.

Sources

[1] "How the AI Engineer role is unbundling in 2026" (names the AI Evaluation & Governance Engineer). AI Journal. 2026-08-26. <https://aijourn.com/how-the-ai-engineer-role-is-unbundling-in-2026/> (verified: secondary) [2] "The GRC Engineer role" (analyst-vs-engineer table; career paths). GRC Engineer. 2025. <https://grcengineer.com/grc-engineer/> (verified: primary) [3] "AI Governance VP" posting (intake, classification, approval aligned to NIST AI RMF), USD 120,000-202,500. State Street (via SimplyHired). 2026-09. https://www.simplyhired.com/job/iy04VeXdd9DR3vffvKZvupPCsoZtHi405zOLc_RMSJXOSRJvQLbhmw (verified: secondary) [4] "AI Governance Engineer" posting (agent registry, threat modelling, red teams, MCP security), USD 175,000-260,000. Dalio Family Office (via SimplyHired). 2026-09. <https://www.simplyhired.com/job/vKibNHPKReLf8h5PgcPxYjWZOx1ctDsmI8sjzydKbSpd6ZoNoNvP-Q> (verified: secondary) [5] "Engineering Manager, GRC" posting (AI-forward GRC engineering function; policies into policy-as-

code), USD 405,000. Anthropic (via General Catalyst jobs). 2026.
<https://jobs.generalcatalyst.com/companies/anthropic/jobs/78167778-engineering-manager-grc> (verified: secondary)
 [6] "AI Governance Engineer" posting (security controls, threat models, incident response for LLM systems), USD 100,000-150,000. Bright Vision (via SimplyHired). 2026-09.
https://www.simplyhired.com/job/s2v6mqCI9KO8Xldf5enD0Z46iDqIBLdwS0jqZlgtO8YhD-DU_4XDog (verified: secondary) [7] AI governance jobs analysis (US postings since Jan 2026: observability 41-42%, Python 27-28%, NIST ~27%, foundation models 25.6%, cloud 18.2%; median pay USD 169,000; median 5 yrs). Axial Search. 2026-08-04.
<https://axialsearch.com/insights/ai-governance-jobs> (verified: secondary) [8] Top 10 for Agentic Applications 2026. OWASP GenAI Security Project. 2025-12-09. <https://genai.owasp.org/resource/owasp-top-10-for-agentic-applications-for-2026/> (verified: primary) [9] Salary & Jobs Report 2025-26 (technical AI-gov in tech median USD 221,000; AI governance only 151,800; privacy + AI governance 169,700). IAPP. 2025-08-03.
<https://iapp.org/resources/article/salary-survey-summary/> (verified: primary) [10] LinkedIn 2026 Skills on the Rise (governance and responsible-AI skills among the fastest-rising clusters; no per-skill percentage published). LinkedIn, via EdTech Innovation Hub. 2026. <https://www.edtechinnovationhub.com/news/linkedins-2026-skills-on-the-rise-shows-global-ai-driving-hiring-shifts> (verified: secondary) [11] "What is GRC Engineering" (capability, not a job title). GRC Engineer. 2025. <https://grcengineer.com/what-is-grc-engineering/> (verified: primary)

07. Maturity model (five levels)

A ladder from paper to production — Documented, Inventoried, Tested, Enforced, Continuous — where each level is proven by what the running systems can show, not by what a document claims.

Why a maturity model, and how to read this one

Maturity models fail when they measure paperwork. This one measures the systems. A level is not a score you award yourself; it is a state you can demonstrate by querying the registry, running the gate and reading the evidence store. The ladder runs from governance that exists only on paper to governance that runs continuously off the runtime data path.

The five levels answer the three questions with rising confidence. **Documented** and **Inventoried** answer *what AI is running* — first on paper, then from a live inventory. **Tested** and **Enforced** answer *what it is allowed to do* — first by measuring, then by blocking. **Continuous** answers *what evidence proves it* — continuously, from telemetry. Each level is assessed across all five layers of the stack (chapter 04); you are at a level only when every layer has reached it, because a chain is as strong as its weakest layer.

The five levels

Level 1 — Documented. Governance exists as artefacts a human maintains: a policy PDF, a spreadsheet inventory, a risk register, a review that happens before launch. The rules are written and someone is accountable, but nothing executes. Typical evidence: policy documents, a populated spreadsheet, meeting minutes. Typical failure that moves you back: the document was last edited a quarter ago and no longer matches production — the artefact is stale before it is signed.

Level 2 — Inventoried. There is a real inventory of models and an **agent registry**, and it is fed by a runtime data path rather than typed by hand: a deploy registers a system with an owner, a scope and a status. You can answer *what is running* on any given day. Typical evidence: a registry with an owner and a class for every system; a discovery job reconciling registry against production. Typical failure: **shadow AI** — a system or agent reaches production without registering, so the inventory is complete only for the honest.

Level 3 — Tested. Systems are evaluated against defined tests — capability, safety and adversarial evals — and the results are recorded as evidence. Failures are visible, but a failing eval does not yet stop anything. You know which systems fall short; you have not yet made falling short have consequences. Typical evidence: versioned eval suites; stored, timestamped eval results; red-team findings. Typical failure: the eval is run once before launch, pasted into a slide, and never re-run when the model or its prompts change.

Level 4 — Enforced. The tests bite. **Policy-as-code** and **eval gates** run in CI/CD and at admission, and a failing control blocks the merge or the deploy. Identity precedes autonomy: an agent with no owner, scope or **kill switch** is denied a workload identity. Governance is now a property of the build, not a checkpoint after it. But a blocking gate is only as good as the test behind it, so Level 4 has a second condition that is easy to skip: the eval suite's own *quality* is assessed, not just its existence and its teeth. A gate that blocks on a trivial or stale suite is Level 4 by the letter and framework theatre in fact — a green build that proves nothing. So the enforcement claim requires that coverage is measured, adversarial cases are maintained against current threats, and thresholds trace to named failure modes rather than round numbers (see chapter 01, "the limits of the eval gate"). Typical evidence: pipeline logs showing blocked releases with reasons; admission-control denials; the registry acting as a deploy gate; a tracked coverage or adversarial-quality metric for the suites that gate. Typical failure: brittle gates that engineers route around; a gate maintained by governance alone that engineering does not own; or a gate whose suite is trivial or unmaintained, so the block is real but the assurance is not.

Level 5 — Continuous. Assurance is produced continuously from the runtime data path. Guardrail decisions, tool-call mediation, drift and agent behaviour stream into observability; **continuous assurance** turns production behaviour into a live control signal; evidence is emitted as **machine-readable artefacts** (**OSCAL** , signed logs) as the pipeline and runtime operate. The audit is a query. This is the end state most of the AI-governance platform category does not reach, because it "manages the program ... without any runtime data path" [1]. Typical evidence: a live assurance store; streaming eval and guardrail telemetry; an audit answered by running a query. Typical failure: telemetry that is collected but never wired to a decision — observability without enforcement decays back to Level 3 dressed up as Level 5.

Observable criteria, by layer and level

Read each row as one layer maturing left to right. You are at a level only when every row has reached its column.

Layer	1 Documented	2 Inventoried	3 Tested	4 Enforced	5 Continuous
1 Govern-as-Code	Policies written as prose	Policies indexed, mapped to systems	Policy checks run and report, non-blocking	Policy-as-code blocks merge/deploy	Policy verdicts stream to assurance, versioned
2 Inventory & Transparency	Spreadsheet inventory	Registry fed by deploy; owner + scope per system	Registry reconciled against production	Registry gates deployment ; no entry, no identity	Registry live off runtime discovery ; drift auto-flagged
3 Evals & Red Teaming as Evidence	Evals described in a plan	Eval suites exist and are versioned	Evals run, results stored, non-blocking	Eval gate fails the build on regression; suite coverage and adversarial quality assessed	Evals run continuously; results are live evidence
4 Runtime Controls & Observability	Guardrails named in a design	Guardrails deployed, not measured	Guardrail decisions logged	Kill switch tested; tool-calls mediated and enforced	Runtime signals drive control decisions in real time
5 Assurance & Continuous Compliance	Evidence gathered by hand for audit	Evidence templated per control	Structured evidence produced per run	Evidence required to pass the gate	Machine-readable evidence emitted continuously; audit = query

Partial maturity is the normal state. Almost no real function sits at one clean level across all five layers; the usual picture is a ragged line — inventory at Level 4, evals at Level 2, assurance at Level 3. That is not a failure of the model, it is the point of reading it by layer. The single overall level is the weakest layer, and it is a *floor* for planning, not a verdict on the whole function. Two readings follow: report the per-layer profile, not just the floor, because it shows where the leverage is; and expect the profile to stay ragged, because layers mature at the speed of the work they gate, not in lockstep. A function that is Level 4 on identity and Level 2 on evals is doing better than the overall "Level 2" suggests — and its next move is obvious from the profile.

Metrics per level

Each level has metrics you can read off the systems. Track the trend, not the single number.

- **Level 1 → 2:** percentage of AI systems and agents in the registry with a named owner and a class; registry-to-production reconciliation gap (systems in production but not registered).
- **Level 2 → 3:** percentage of registered systems with a versioned eval suite; percentage with a recorded, timestamped eval result in the last release.
- **Level 3 → 4:** percentage of releases passing through an **eval gate** (versus bypassing it); percentage of agents with a tested kill switch and a scoped, non-shared identity; number of releases blocked with a logged reason.
- **Level 4 → 5:** mean time to detect an unauthorised agent action (an agent doing something outside its declared scope — OWASP Agentic ASI03/ASI10 territory [2]); **evidence freshness** (age of the most recent evidence artefact per control); percentage of controls whose status is answerable by a live query rather than a manual pull.

The single most telling cross-level metric is evidence freshness. At Level 1 the freshest evidence is a quarter old; at Level 5 it is as old as the last pipeline run. If your evidence ages in months, you are not yet continuous, whatever the dashboard says.

Self-assessment checklist

Answer each with the system, not the intention. A "no" caps you at the level below.

- **Documented:** Is every AI system covered by a written policy with a named owner? Is there a risk register that a person maintains?
- **Inventoried:** Does the registry get an entry automatically at deploy, with owner, scope and status? Can you list every model and agent running today, from the system of record, in under a minute?

- **Tested:** Does every registered system have a versioned eval suite? Are results stored with timestamps? Do you run red-team evals against your agents?
- **Enforced:** Does a failing eval or policy check actually block a release? Is an agent without an owner, scope and kill switch prevented from reaching production? Can you show a release that was blocked, with the reason logged?
- **Continuous:** Is runtime telemetry wired to control decisions, not just dashboards? Is evidence emitted as machine-readable artefacts continuously? Would an audit question be answered by a query rather than a collection sprint?

If you can say yes to a whole level and to every layer within it, you are at that level. The first "no" is your next piece of work — and the smallest step to the next level is almost always to close the weakest layer, not to add a sixth control to the strongest one.

How this relates to certification and other assessments

This maturity model is not a certification and does not confer one. It relates to three external schemes; the relationship is one of support and overlap, not equivalence.

ISO/IEC 42001 certification. ISO/IEC 42001 certifies that an AI management system (AIMS) exists and is operated — a Level 1-2 proof of *process*: that governance is documented, owned and reviewed. It says little about whether an **eval gate** blocks a build or whether evidence is machine-readable, the Level 4-5 properties. And it is not a harmonised standard: the certificate confers no presumption of conformity with the EU AI Act, because none is yet cited in the Official Journal [3]. Reaching Level 5 supports a 42001 audit by producing evidence continuously; it does not replace the certificate, and the certificate does not prove you are past Level 2.

OWASP AI Maturity Assessment (AIMA). OWASP's GenAI Security Project publishes an AI Maturity Assessment reported at v1.0 (Aug 2025) [4]. It is complementary: where AIMA scores the *breadth* of an AI security programme, this model scores the *depth* of the runtime data path. Use AIMA to find gaps in coverage; use this ladder to find whether the covered controls actually fire.

CSA STAR for AI. CSA's STAR for AI is a certification programme built on the AI Controls Matrix (AICM), with a self-assessment tier, an automated "Valid-AI-ted" tier and a Level 2 combining third-party ISO/IEC 42001 certification with the validated assessment [1][5]. Its Level 2 aligns with the *Enforced* end of this ladder, but, like 42001, it attests a programme rather than measuring the freshness of runtime evidence — the property continuous assurance (Level 5) makes cheap to produce and hard to fake.

In practice A function in a large telco assessed itself honestly and landed at Level 2 for inventory but Level 1 for evals: the registry was live off the deploy pipeline, but evals were still run by hand before launch and pasted into slides. The chain was only as strong as its weakest layer, so the function was Level 1 overall. The smallest step was not a new framework mapping; it was versioning one eval suite and storing its timestamped results — moving the eval layer to Level 3 — before wiring it into a gate. Evidence freshness fell from a quarter to a release cycle within two sprints.

Maps to: EU AI Act Art. 9 (risk management), Art. 17 (quality management system), Art. 72 (post-market monitoring) · ISO/IEC 42001 (AIMS) and ISO/IEC 42005 (impact assessment) · NIST AI RMF (Govern, Measure, Manage) · OWASP Top 10 for Agentic Applications 2026 · CSA AICM / STAR for AI. Mappings are illustrative, not a claim of conformity.

Sources

[1] "Best AI Governance Platforms 2026" (most of the category "manages the program ... without any runtime data path"). Kosmoy. 2026. <https://www.kosmoy.com/resources/blog/best-ai-governance-platforms-2026/> (verified: secondary) [2] Top 10 for Agentic Applications 2026 (ASI03 Agent Identity & Privilege Abuse; ASI10 Rogue Agents). OWASP GenAI Security Project. 2025-12-09. <https://genai.owasp.org/resource/owasp-top-10-for-agentic-applications-for-2026/> (verified: primary) [3] ISO/IEC 42001 certification is not yet a presumption of conformity with the EU AI Act (no harmonised standard cited in the OJ). Law & Technology. 2026. <https://lawandtechnology.eu/en/iso-iec-42001-and-the-ai-act-why-certification-is-not-yet-a-presumption-of-conformity/> (verified: secondary) [4] OWASP AI Maturity Assessment (AIMA), reported at v1.0 (Aug 2025); Secure Governance initiative. OWASP GenAI Security Project. 2025. <https://genai.owasp.org/initiatives/> (verified: reported) [5] STAR for AI (three certification levels; Level 2 = third-party ISO/IEC 42001 + Valid-AI-ted; built on the AI Controls Matrix). Cloud Security Alliance. 2026. <https://cloudsecurityalliance.org/star/ai> (verified: primary)

08. Regulatory map (obligation → artefact → layer)

This chapter is the reverse index of every "Maps to" line in the book: for each obligation it names the engineering artefact that satisfies or supports it and the stack layer the artefact lives in.

Every other chapter maps *forward* — a capability, then the obligations it touches. This chapter maps *backward* — an obligation, then the artefact and the layer that answer it. The unit of the map is a row: an obligation, the engineering artefact that produces the evidence for it, and one of the five stack layers (chapter 04): **1 Govern-as-Code · 2 Inventory & Transparency · 3 Evals & Red Teaming as Evidence · 4 Runtime Controls & Observability · 5 Assurance & Continuous Compliance**. The map is a crosswalk for finding the artefact that answers a question, not a certificate that the artefact makes you compliant.

How to read this map

Read each row as a sentence: *this obligation is answered by this artefact, which lives in this layer*. Three cautions apply throughout.

- **Mappings are illustrative, not a claim of conformity.** No artefact in this book guarantees compliance, and no standard cited here confers a presumption of conformity (see "What is NOT harmonised yet"). An artefact *supports* and *evidences* an obligation; the legal judgement of conformity stays with lawyers, notified bodies and authorities.
- **Dates are the post-Omnibus dates.** Every EU AI Act date below reflects Regulation (EU) 2026/1744, the Digital Omnibus on AI, of 8 July 2026 (OJ L, 24 July 2026), in force 27 July 2026 [1][2][22]. Where the Omnibus moved a date, the moved date is shown; where it did not, the row says so.
- **The competent authority differs by regime.** For general-purpose AI (GPAI) the supervisor is the **AI Office**, and GPAI fines are formal Commission decisions under Article 101 [3][4]. For high-risk systems the supervisors are **national market-surveillance authorities**, whose penalties run under Article 99 [4]. The map states the authority per row so the reader knows who asks.

EU AI Act, post-Omnibus

High-risk obligations (Articles 9–15, 17, 25, 26, 27, 49, 71, 72, 73) apply to Annex III systems from **2 December 2027** and to Annex I embedded systems from **2 August 2028**, both deferred by the Omnibus from 2 August 2026 and 2 August 2027 respectively [1][2]. The GPAI obligations (Articles 53, 55) have applied since 2 August 2025, with Commission enforcement powers live since 2 August 2026 [3]. Transparency (Article 50) went live on 2 August 2026 and was not moved [5]. Legacy public-authority high-risk systems keep their original 2 August 2030 date [2].

The **Duty holder** column names who the obligation binds, which is a different axis from who enforces it. The high-risk design-and-build duties — Articles 9 to 15 and 17 — fall on the **provider**; Articles 26 and 27 fall on the **deployer**; Articles 4, 5 and 50 bind **both**; and Articles 53 and 55 bind the **GPAI provider**. Article 25 sits across the value chain: it sets the conditions under which a distributor, importer or deployer itself becomes a **provider** and inherits the provider duties. This matters for the engineer because the artefacts you can produce depend on which hat your organisation wears: a deployer cannot draw up the provider's technical documentation, but it must run the Article 26 monitoring and the Article 27 FRIA — and when the model is procured, most of the provider-side evidence becomes something you collect rather than produce (see the Vendor / Model Due-Diligence Gate in chapter 05).

Article 16 is the umbrella for the provider's high-risk duties. It adds no artefact of its own; it gathers the obligations the rows below break out article by article — the requirements of Articles 9 to 15 and 17, the conformity assessment (Article 43), the EU declaration of conformity (Article 47), registration (Articles 49 and 71), post-market monitoring (Article 72) and serious-incident reporting (Article 73) — so it reads here as a cross-reference, not a row [22].

Article	Obligation	Engineering artefact	Layer	Duty holder	Applies (post-Omnibus)	Authority
Art. 4	AI literacy: take measures to support the development of AI literacy among staff and operators	Literacy programme as code; role-based training records; onboarding gates	1	Provider + deployer	2026-07-27 (reworded, in force) [6]	Provider/deployer duty; national MSA

Article	Obligation	Engineering artefact	Layer	Duty holder	Applies (post-Omnibus)	Authority
Art. 4a	Lawful basis to process special-category data for bias detection in high-risk systems, with pseudonymisation and deletion once bias is corrected	Data governance controls; pseudonymisation and retention-as-code; data card noting basis and deletion	2	Provider	2026-07-27 (new, in force) [2]	National MSA / DPAs
Art. 5	Prohibited practices; new bans on AI-generated non-consensual intimate imagery (NCII) and CSAM	Policy-as-code blocklist; input/output guardrails; refusal and abuse detection	1 · 4	Provider + deployer	2026-12-02 (new bans); earlier prohibitions from 2025-02-02 [2]	National MSA
Art. 6	Classification rules for high-risk AI systems, incl. the Annex III (standalone) route and Annex I (safety-component) route	Risk-tiering as code; high-risk classification decision record; register entry flagging Annex III status	1 · 2	Provider	2027-12-02 (Annex III) [1] [22]	National MSA
Art. 9	Risk management system across the high-risk lifecycle	Risk register as code; threat models; linkage to FRIA and eval results	1 · 3	Provider	2027-12-02 (Annex III) [1]	National MSA
Art. 10	Data and data governance; representative, relevant, error-checked datasets	Data cards; lineage; bias and quality tests in CI	2 · 3	Provider	2027-12-02 (Annex III) [1]	National MSA
Art. 11	Technical documentation (Annex IV) drawn up and kept up to date	AIBOM (Cyc1oneDX ML-BOM , SPDX 3.0 AI); auto-generated technical documentation; model cards	2	Provider	2027-12-02 (Annex III) [1]	National MSA
Art. 12	Record-keeping: automatic logging of events over the system's lifetime	Structured, signed logs; OpenTelemetry traces; tamper-evident event store	4	Provider	2027-12-02 (Annex III) [1]	National MSA
Art. 13	Transparency and provision of information to deployers	Instructions for use as code; model and data cards; capability and limitation notes	2	Provider	2027-12-02 (Annex III) [1]	National MSA
Art. 14	Human oversight designed into the system	Human-in-the-loop checkpoints; kill switch; override and escalation paths	4	Provider	2027-12-02 (Annex III) [1]	National MSA
Art. 15	Accuracy, robustness and cybersecurity	Eval gate; adversarial red-team suite; robustness and security controls; regression evals	3 · 4	Provider	2027-12-02 (Annex III) [1]	National MSA
Art. 17	Quality management system	QMS-as-code; versioned policies; pipeline controls and change management	1 · 5	Provider	2027-12-02 (Annex III) [1]	National MSA

Article	Obligation	Engineering artefact	Layer	Duty holder	Applies (post-Omnibus)	Authority
Art. 25	Responsibilities along the AI value chain: when a distributor, importer or deployer becomes a provider, and the information a provider must pass to actors downstream	Value-chain due-diligence gate; provider/deployer responsibility allocation; AIBOM and model/data cards collected from upstream providers	2 · 5	Provider + value-chain actors	2027-12-02 (Annex III) [23]	National MSA
Art. 26	Deployer obligations for high-risk systems (use per instructions, monitoring, human oversight)	Deployment registry; monitoring hooks; assigned oversight and logging retention	2 · 4	Deployer	2027-12-02 (Annex III) [1]	National MSA
Art. 27	Fundamental Rights Impact Assessment (FRIA) for deployers of Annex III systems	FRIA-as-code from a template; cross-reference to a GDPR Art. 35 DPIA	1 · 2	Deployer	2027-12-02 (Annex III) [7]	National MSA
Art. 43	Conformity assessment before placing on the market (internal control, or a notified body for Annex III point 1 biometrics)	Conformity-assessment workflow; internal-control or notified-body evidence pack; traceability to Annex IV documentation	1 · 5	Provider	2027-12-02 (Annex III) [1] [22]	National MSA
Art. 47	EU declaration of conformity drawn up on completing the assessment	Auto-generated EU declaration of conformity from the evidence; CE-marking record	2 · 5	Provider	2027-12-02 (Annex III) [1] [22]	National MSA
Art. 49 / Art. 71	Registration of high-risk systems in the EU database	Agent/model registry with an API that feeds registration; owner and status per entry	2	Provider; public-authority deployer	2027-12-02 (Annex III) [1]	National MSA; Commission (database)
Art. 50	Transparency for certain AI systems: chatbot disclosure; marking and labelling of synthetic content	Content labelling and machine-readable marking (e.g. C2PA-style); chatbot disclosure banner	4 · 2	Provider + deployer	2026-08-02; marking grace for existing systems to 2026-12-02 [5]	National MSA
Art. 53	GPAI provider obligations, incl. a public summary of training content on an AI Office template	Model cards; training-content summary; AIBOM and dataset provenance	2	GPAI provider	Obligations from 2025-08-02; enforcement from 2026-08-02 [3]	AI Office
Art. 55	GPAI models with systemic risk: model evaluation incl. adversarial testing; Union-level risk assessment; serious-incident reporting; cybersecurity of the model	Eval and red-team suite; incident pipeline on the Commission serious-incident reporting template; weight-security controls; threat model	3 · 4 · 5	GPAI provider (systemic risk)	Obligations from 2025-08-02; enforcement from 2026-08-02 [3][26]	AI Office
Art. 60	Testing of high-risk (Annex III) AI systems in real-world conditions outside AI regulatory sandboxes	Real-world testing plan; Art. 61 informed-consent records; test monitoring, logging and incident hooks	3 · 4	Provider / prospective provider	2026-08-02 [22]	National MSA

Article	Obligation	Engineering artefact	Layer	Duty holder	Applies (post-Omnibus)	Authority
Art. 72	Post-market monitoring for high-risk systems	Continuous assurance telemetry; monitoring plan; drift and performance signals	5	Provider	2027-12-02 (Annex III) [1]	National MSA
Art. 73	Serious-incident reporting for high-risk systems (deadlines in the reporting-clock table below)	Incident detection and triage pipeline; reporting-clock automation; evidence capture	5 · 4	Provider	2027-12-02 (Annex III) [1]	National MSA

The Article 73 clock runs by incident class. The provider notifies the national market-surveillance authority on these deadlines, each running from the moment it becomes aware of the incident [22]:

Incident class	Reporting deadline	Who reports	Artefact
Serious incident (general)	No later than 15 days after establishing the causal link	Provider → national MSA	Incident triage pipeline; reporting-clock automation
Widespread infringement, or serious and irreversible disruption of critical infrastructure	No later than 2 days	Provider → national MSA	Same pipeline, escalated-severity path
Death of a person	No later than 10 days	Provider → national MSA	Same pipeline, priority path

Two cross-cutting provisions frame the penalties. Under **Art. 101**, the Commission may fine GPAI providers up to 3% of worldwide annual turnover or EUR 15 million, whichever is higher [3][4]. Under the Omnibus's new **Art. 75a – 75d**, the AI Office gains investigation powers backed by periodic penalty payments of up to 5% of average daily turnover per day for a continuing breach, in force since 27 July 2026 [8]. High-risk administrative fines run under **Art. 99** (ceilings of 7%, 3% and 1% of turnover depending on the breach), imposed by national authorities [4].

GPAI Code of Practice

The GPAI Code of Practice, published 10 July 2025, is the voluntary instrument providers use to demonstrate compliance with the GPAI obligations until harmonised standards exist. It has three chapters [9][10].

Chapter	What it asks for	Engineering artefact	Layer
Safety and Security (systemic-risk models only)	A Safety and Security Framework; model evaluations incl. adversarial testing; systemic-risk assessment and mitigation; serious-incident reporting; model and infrastructure security	Eval and red-team suite; adversarial testing harness; incident pipeline; weight-security controls	3 · 4 · 5
Transparency	Up-to-date model documentation for the AI Office and downstream deployers	Model cards; structured model documentation; AIBOM	2
Copyright	A policy to comply with Union copyright law, incl. respecting reservations of rights	Training-data provenance and licence records; policy-as-code for source filtering	1 · 2

The Code is voluntary; signing it is one route to demonstrating compliance, not a legal presumption of conformity [9]. Signatory status is dynamic and counted per the Commission's official list [10].

For serious-incident reporting specifically, the Commission published a reporting template on 4 November 2025 for serious incidents involving GPAI models with systemic risk. It aligns to the Article 55 reporting duty and to Commitment 9 of the Code's Safety and Security chapter, and is the concrete artefact the incident pipeline emits — the same template named in the Article 55 row above [26].

ISO/IEC 42001, 42005 and 42006

ISO/IEC 42001:2023 is the AI management-system (AIMS) standard; its Annex A groups control objectives into nine areas (A.2 – A.10). It is a management-system standard, not the Article 17 QMS, and its European adoption (EN ISO/IEC 42001:2026) confers no presumption of conformity [11][12].

Annex A area	Focus	Engineering artefact	Layer
A.2 Policies related to AI	AI policy set and its governance	Policy-as-code library; versioned policy repository	1
A.3 Internal organization	Roles, responsibilities, reporting	Operating model; RACI; ownership in the registry	1 · 2
A.4 Resources for AI systems	Data, tooling, compute, human resources documented	Resource inventory; AIBOM; environment manifests	2
A.5 Assessing impacts of AI systems	Impact assessment process	Impact assessment as code; FRIA/DPIA linkage (ISO/IEC 42005)	1 · 3
A.6 AI system life cycle	Responsible design, development, deployment	Pipeline controls; eval gates; change management	1 · 3 · 4
A.7 Data for AI systems	Data quality, provenance, preparation	Data cards; lineage; data quality tests	2 · 3
A.8 Information for interested parties	Transparency and reporting to stakeholders	Model/data cards; machine-readable disclosures	2
A.9 Use of AI systems	Responsible-use controls and monitoring	Runtime guardrails; usage telemetry	4
A.10 Third-party and customer relationships	Managing supplier and customer responsibilities	Supplier AIBOM; contractual and technical control mapping	2 · 5

ISO/IEC 42005:2025 provides guidance for AI system impact assessment and is the natural companion to Article 27 (FRIA) and Annex A.5 [13].

Two further ISO/IEC standards sit alongside the AIMS. **ISO/IEC 42006:2025** sets the requirements for the bodies that audit and certify AI management systems: building on ISO/IEC 17021-1, it is the answer to "who may credibly certify you to 42001", since it fixes the competence and consistency a certification body must show. **ISO/IEC 23894:2023** gives guidance on AI risk management, adapting ISO 31000 to AI; it is the risk-process companion to Article 9 and to the NIST AI RMF [27][28].

Standard	What it is	Engineering artefact	Layer
ISO/IEC 42006:2025	Requirements for bodies auditing and certifying AI management systems (who may credibly certify you to 42001)	Accredited certification scope; auditor-competence evidence; certificate register	5
ISO/IEC 23894:2023	Guidance on AI risk management (companion to ISO 31000)	Risk register as code; AI risk taxonomy; linkage to Art. 9 and NIST AI RMF	1 · 3

Maps to: these controls are realised through layers 1, 2, 3 and 5 of the stack; the impact-assessment control (A.5 / ISO 42005) supports EU AI Act Art. 27. Mappings are illustrative, not a claim of conformity.

NIST AI RMF

The NIST AI Risk Management Framework 1.0 (January 2023; there is no 2.0) organises risk work into four functions. It is voluntary and US-origin, and it maps cleanly onto the five-layer stack [14].

Function	What it asks for	Engineering artefact	Layer
GOVERN	A culture and structure for managing AI risk	Policy-as-code; operating model; registry ownership	1 · 2
MAP	Context and risk framing for each AI system	Threat models; use-case and impact mapping; data/model cards	2 · 3
MEASURE	Analyse, benchmark and monitor risk	Eval gates; adversarial red-team suite; metrics per failure mode	3
MANAGE	Prioritise, respond and recover	Runtime guardrails; incident pipeline; continuous assurance	4 · 5

Newer NIST AI work

Beyond the RMF, three newer NIST efforts bear on the stack. Two are still in draft, and the copy says so. The **AI Agent Standards Initiative**, launched by NIST's Center for AI Standards and Innovation (CAISI) on 17 February 2026, targets interoperable, secure standards for AI agents — identity, authentication, authorisation and agent security [29]. The draft **IR 8596 Cyber AI Profile** (initial public draft, 16 December 2025) is a Cybersecurity Framework (CSF 2.0) profile for AI, organised around Secure, Defend and Thwart [30]. The draft **AI 800-1** (Managing Misuse Risk for Dual-Use Foundation Models; second public draft, January 2025) is voluntary guidance for identifying, measuring and mitigating misuse risk across the AI lifecycle [31].

NIST item	What it is	Engineering artefact	Layer
AI Agent Standards Initiative (2026)	CAISI initiative on interoperable, secure AI agents: identity, authentication, agent security	Agent registry; non-human-identity controls; agent authentication and authorisation; adversarial agent evals	3 · 4
IR 8596 Cyber AI Profile (draft)	CSF 2.0 profile for AI (Secure / Defend / Thwart)	AI-system security controls; runtime observability; threat detection mapped to CSF 2.0	3 · 4
AI 800-1 (draft)	Managing Misuse Risk for Dual-Use Foundation Models — voluntary guidance	Misuse red-team suite; capability and dangerous-capability evals; safety framework	3

CSA AICM and STAR for AI

The Cloud Security Alliance's AI Controls Matrix (AICM) v1.1, published 22 June 2026, defines 247 control objectives across 18 domains, and the STAR for AI programme (with an agentic-certification track) provides the assurance scheme around it [15].

CSA artefact	What it is	Engineering artefact	Layer
AICM v1.1	247 control objectives across 18 domains, spanning governance, data, model and runtime	Control catalogue mapped to policy-as-code and evals; crosswalk to ISO 42001 / NIST AI RMF	1 · 3 · 5
STAR for AI	Assurance and certification programme, incl. proposed agent controls	Machine-readable evidence submission; continuous assurance telemetry	5
Agentic Control Supplement	A proposed set of agent-specific controls extending the AICM	Agent-specific control definitions; policy-as-code for agent scope and tools; runtime guardrails	1 · 4
Catastrophic Risk Annex	Enhanced AICM controls for high-autonomy systems with catastrophic-risk potential	Enhanced controls for high-autonomy systems; kill switch and oversight controls; pilot-audit evidence	4 · 5

Two extensions push the AICM toward agents and frontier risk. The **Agentic Control Supplement** adds a proposed set of agent-specific controls, and the **Catastrophic Risk Annex** adds a set of enhanced controls for high-autonomy systems, meant to be proven through pilot audits rather than asserted [32][33].

OWASP GenAI Security Project

OWASP's GenAI Security Project supplies the threat vocabulary the controls are built against, plus two formats — the Agent Control Standard and an AIBOM — that the stack consumes directly [16][17].

OWASP artefact	What it is	Engineering artefact	Layer
Top 10 for Agentic Applications 2026	Agent threat catalogue (ASI01 Agent Goal Hijack ... ASI10 Rogue Agents)	Agent threat model; adversarial evals; runtime guardrails; kill switch	3 · 4
Top 10 for LLM Applications 2026	LLM threat catalogue (incl. Excessive Agency at #3)	Prompt-injection and output-handling controls; eval gate	3 · 4
Agent Control Standard (ACS)	A standard for expressing agent controls	Machine-readable control definitions for agents	1 · 4
AIBOM	AI bill-of-materials format and generator	AIBOM at build (Cyc1oneDX ML-BOM , SPDX 3.0 AI)	2

US federal and state laws

The United States has no horizontal federal AI statute; the binding rules are state laws, and they differ in scope. Two bind only large frontier developers; two more — Texas and Colorado — reach ordinary developers and deployers.

Frontier-developer laws

Two US state laws bind only large frontier developers, not general deployers — a narrower scope than the EU AI Act's risk-tiering. They ask frontier developers to publish safety frameworks and report critical incidents to the state [18][19].

Law	Scope	Obligation	Engineering artefact	Layer
California SB 53 (TFAIA), in force 2026-01-01	Large frontier developers (models trained above $\sim 10^{26}$ FLOP; developer revenue over USD 500M)	Publish a safety and security framework; report critical safety incidents to the state; whistleblower protection; up to USD 1M per violation, AG-enforced	Published safety framework; incident pipeline reporting to the state; transparency artefacts	5 · 4
New York RAISE Act (S6953B), signed 2025-12-19	Large frontier developers (frontier models trained with over 10^{26} operations, cost over USD 100M)	Publish a frontier AI safety and security framework; disclose safety incidents within 72 hours. A chapter amendment signed 2026-03-27 sets the effective date at 2027-01-01 and creates an oversight office within the New York Department of Financial Services (DFS) [19][24][25]	Published safety framework; incident and disclosure pipeline reporting to the state	5 · 4

Other state AI laws

Two further state laws reach beyond frontier developers to ordinary developers and deployers.

Law	Scope	Obligation	Engineering artefact	Layer
Texas TRAIGA (HB 149), in force 2026-01-01	Developers and deployers doing business in Texas	Intent-based prohibitions (social scoring, unlawful discrimination); AI-use disclosure; a regulatory sandbox; Attorney-General enforcement; local AI rules preempted [35]	Prohibited-use policy-as-code; AI-use disclosure controls; complaint and incident handling	1 · 4
Colorado AI Act (SB 24-205)	Developers and deployers of high-risk (consequential) automated decisions	Duty of reasonable care against algorithmic discrimination; risk management and consumer notice. Its 1 Feb 2026 start was delayed to 30 June 2026, then the Act was replaced by SB 26-189 (signed 14 May 2026), a narrower transparency law effective 1 Jan 2027, after a federal court blocked enforcement of the original [36]	High-risk ADM inventory; algorithmic-discrimination impact assessments; consumer disclosure	1 · 2 · 5

Other jurisdictions

The map's spine is the EU AI Act, but a governance function working across borders answers to more than one regime. These rows are stamped as of 2026-09-19; where a rule is still moving, the copy says so.

Jurisdiction / instrument	Status (as of 2026-09-19)	What it asks for	Engineering artefact	Layer
South Korea — AI Basic Act	In force 2026-01-22; a grace period runs through 2026 [34]	Baseline duties for AI operators, heightened duties for "high-impact" AI in sensitive sectors, and AI-content labelling	Risk register for high-impact AI; AI-use notification; AI-content labelling	1 · 2 · 4
Singapore — IMDA Model AI Governance Framework for Generative AI	Voluntary; published May 2024 [39]	Governance dimensions incl. testing, transparency, incident reporting, security and content provenance	Eval suite; model cards; content provenance and watermarking	2 · 3 · 4
ETSI EN 304 223 (Securing AI)	Published (V2.1.1, Dec 2025) [40]	Baseline cyber-security requirements across the AI lifecycle (13 principles over five stages)	AI-system security controls across the lifecycle; supply-chain and AIBOM checks; runtime hardening	4

United Kingdom

The UK has no horizontal AI act. It governs AI through existing sector regulators (the ICO, the FCA, the MHRA and others), coordinated centrally, plus the **AI Security Institute** — renamed from the AI Safety Institute in February 2025 — for frontier-model evaluation [38]. For automated decision-making, the Data (Use and Access) Act 2025 replaced UK GDPR Article 22 with new **Articles 22A–22D** (in force 5 February 2026): a permission-plus-safeguards model for significant, solely automated decisions, with tighter conditions where special-category data is used. The safeguards — a meaningful-human-review path, a channel to make representations and to contest, and a decision notice — are the artefact the engineer builds [37].

What is NOT harmonised yet

The map has a hole, and it is important to state it plainly rather than paper over it.

- **No harmonised standard is cited in the Official Journal.** As of 2026-09-19, Article 40's presumption of conformity is available to no one, because no harmonised standard has been OJ-cited [20].
- **EN 18286 is published but not cited.** The Article 17 QMS standard EN 18286:2026 was published in July 2026 — the first JTC 21 AI Act standard to reach publication — but it is not yet cited in the Official Journal, so it carries no presumption of conformity [21]. The Article 9 (risk), Article 12 (logging) and Article 15 (cybersecurity) standards were still at the Enquiry stage, targeting the end of 2026 [20].
- **ISO/IEC 42001 is not the Article 17 QMS.** Certifying to EN ISO/IEC 42001:2026 evidences an AI management system; it does not confer an AI Act presumption of conformity, because it is not a harmonised standard and its scope differs from the Article 17 QMS [11][12].
- **The Code of Practice is voluntary.** Signing the GPAI Code is a way to demonstrate compliance with GPAI obligations; it is not a legal presumption of conformity [9].

The practical reading: for the period this edition covers, you cannot buy a presumption of conformity off the shelf. The obligation-to-artefact rows above are how a governance function evidences the obligation on its own merits while the harmonised standards are still being written.

In practice (illustrative) A governance team maintained this map not as a slide but as a machine-readable crosswalk: a versioned file linking each obligation ID to the artefact that produced its evidence and the layer it lived in, emitted as `oscal` component definitions. When the Omnibus moved the high-risk dates, the change was a diff to one field per affected row, and every "Maps to" line downstream re-resolved from the same file. The audit question "show me what answers Article 15" became a query against the crosswalk, not a hunt through a wiki.

Maps to: this chapter is the reverse index for the whole book; every EU AI Act article, the GPAI Code of Practice, ISO/IEC 42001, 42005, 42006 and 23894, NIST AI RMF and the newer NIST AI work, CSA AICM, OWASP GenAI/Agentic, the US federal and state laws and the other jurisdictions named above map onto the five-layer stack (chapter 04) and the pattern catalogue (chapter 05). Mappings are illustrative, not a claim of conformity.

Sources

[1] "AI Omnibus enters into force" (Reg. (EU) 2026/1744, in force 2026-07-27; Annex III high-risk → 2 Dec 2027; Annex I → 2 Aug 2028; legacy public-authority → 2 Aug 2030). European Commission. 2026-07-27. <https://digital-strategy.ec.europa.eu/en/news/ai-omnibus-enters-force> (verified: primary) [2] Consolidated changes after the Digital Omnibus (Art. 4a special-category data; Art. 5 NCII/CSAM from 2 Dec 2026; transitional dates). AI Act Explorer (Future of Life Institute). 2026. <https://artificialintelligenceact.eu/ai-act-explorer/digital-omnibus/> (verified: secondary) [3] Commission enforcement powers over GPAI providers apply from 2 August 2026; obligations since 2 August 2025 (fines up to 3% of worldwide turnover or EUR 15M under Art. 101). European Commission — AI Act Service Desk. 2026-08-02. <https://ai-act-service-desk.ec.europa.eu/en/ai-act/faq/commissions-enforcement-powers-related-ai-act-obligations-providers-most-advanced-models> (verified: primary) [4] EU AI Act Art. 101 (Commission fines for GPAI providers) and Art. 99 (penalties by national authorities: 7% / 3% / 1% ceilings). AI Act (Reg. (EU) 2024/1689). 2024. <https://artificialintelligenceact.eu/article/101/> (verified: primary) [5] "Safer and more transparent AI" (Art. 50 transparency live 2 Aug 2026; marking grace for existing generative systems to 2 Dec 2026). European Commission. 2026-08-02. https://commission.europa.eu/news-and-media/news/safer-and-more-transparent-ai-2026-08-02_en (verified: primary) [6] "AI literacy, the Digital Omnibus and Article 4 of the AI Act" (Art. 4 reworded to "support the development of" AI literacy; applies from 27 Jul 2026). Law & Technology. 2026. <https://lawandtechnology.eu/en/ai-literacy-digital-omnibus-article-4-ai-act/> (verified: secondary) [7] EU AI Act Art. 27 (FRIA for deployers of Annex III high-

risk; Art. 27(4) cross-reference to a GDPR Art. 35 DPIA). AI Act (Reg. (EU) 2024/1689). 2024.
<https://artificialintelligenceact.eu/article/27/> (verified: primary) [8] Arts. 75a–75d: AI Office investigation powers with periodic penalty payments up to 5% of average daily turnover per day for continuing breaches (Art. 75c(5)). AI Act Explorer (Future of Life Institute). 2026. <https://artificialintelligenceact.eu/ai-act-explorer/digital-omnibus/> (verified: secondary) [9] GPAI Code of Practice (published 10 Jul 2025; voluntary; three chapters: Transparency, Copyright, Safety and Security). AI Act Explorer / European Commission. 2025-07-10. <https://artificialintelligenceact.eu/introduction-to-code-of-practice/> (verified: primary) [10] GPAI Code of Practice — contents and signatories (Safety & Security applies to systemic-risk models; official signatory list). European Commission. 2026. <https://digital-strategy.ec.europa.eu/en/policies/contents-code-gpai> (verified: primary) [11] "ISO/IEC 42001 and the AI Act: why certification is not yet a presumption of conformity" (ISO 42001 AIMS ≠ Art. 17 QMS). Law & Technology. 2026. <https://lawandtechnology.eu/en/iso-iec-42001-and-the-ai-act-why-certification-is-not-yet-a-presumption-of-conformity/> (verified: secondary) [12] CSA research note on EU AI Act, prEN 18286 and ISO/IEC 42001 (scope difference; EN ISO/IEC 42001:2026 not a harmonised standard). Cloud Security Alliance. 2026-04-28. <https://labs.cloudsecurityalliance.org/research/csa-research-note-eu-ai-act-pren-18286-iso-42001-20260428-cs/> (verified: secondary) [13] ISO/IEC 42005:2025 — AI system impact assessment (companion to Art. 27 and ISO 42001 Annex A.5). ISO/IEC. 2025-05. <https://www.iso.org/standard/44545.html> (verified: secondary) [14] AI Risk Management Framework 1.0 (functions: Govern, Map, Measure, Manage). NIST. 2023-01-26. <https://www.nist.gov/itl/ai-risk-management-framework> (verified: primary) [15] AI Controls Matrix (AICM) v1.1 (247 control objectives, 18 domains) and STAR for AI. Cloud Security Alliance. 2026-06-22. <https://cloudsecurityalliance.org/artifacts/ai-controls-matrix-v1-1> (verified: primary) [16] Top 10 for Agentic Applications 2026 (ASI01 ... ASI10). OWASP GenAI Security Project. 2025-12-09. <https://genai.owasp.org/resource/owasp-top-10-for-agentic-applications-for-2026/> (verified: primary) [17] 2026 Top 10 for LLM Applications, Agent Control Standard (ACS) and AIBOM release wave. OWASP GenAI Security Project. 2026-09-01. <https://genai.owasp.org/2026/09/01/owasp-genai-security-project-unveils-2026-top-10-for-llm-applications-new-agent-control-standard-and-sponsors-as-community-tops-30000-members/> (verified: primary) [18] "California's SB 53: the first frontier AI law explained" (in force 1 Jan 2026; frontier developers over USD 500M revenue and ~10²⁶ FLOP; up to USD 1M/violation; AG enforcement). Future of Privacy Forum. 2026. <https://fpf.org/blog/californias-sb-53-the-first-frontier-ai-law-explained/> (verified: secondary) [19] "Governor Hochul Signs Nation-Leading Legislation to Require AI Frameworks for AI Frontier Models" (RAISE Act, S6953B/A6453B, signed 19 Dec 2025; agreed chapter amendment; creates an oversight office within the Department of Financial Services; 72-hour incident reporting). Governor Kathy Hochul (New York State). 2025-12-19. <https://www.governor.ny.gov/news/governor-hochul-signs-nation-leading-legislation-require-ai-frameworks-ai-frontier-models> (verified: primary) [20] CEN-CENELEC JTC 21 standards tracker (no harmonised standard cited in the OJ → no Art. 40 presumption; risk/logging/cybersecurity standards at Enquiry, Q4 2026 target). CEN-CENELEC JTC 21 (via kla.digital). 2026. <https://kla.digital/blog/jtc-21-standards-tracker> (verified: secondary) [21] EN 18286:2026 (Art. 17 QMS) published July 2026, not yet OJ-cited. CEN-CENELEC news. 2026-07-30. <https://www.cencenelec.eu/news-events/news/2026/en-in-the-spotlight/2026-07-30-ai-quality-management/> (verified: secondary) [22] Regulation (EU) 2026/1744 (Digital Omnibus on AI), of 8 July 2026, amending Reg. (EU) 2024/1689 et al.; OJ L, 24 July 2026; in force 27 Jul 2026. Publications Office of the EU (EUR-Lex). 2026-07-24. <https://eur-lex.europa.eu/eli/reg/2026/1744/oj/eng> (verified: primary) [23] EU AI Act Art. 25 (responsibilities along the AI value chain; conditions under which a value-chain actor becomes a provider; information flow to downstream actors). AI Act (Reg. (EU) 2024/1689). 2024. <https://artificialintelligenceact.eu/article/25/> (verified: primary) [24] NY State Senate Bill 2025-S6953B (RAISE Act; signed 19 Dec 2025 as Chapter 699; frontier model = trained with over 10²⁶ operations costing over USD 100M; safety protocols and 72-hour incident disclosure; Attorney General and Division of Homeland Security and Emergency Services). New York State Senate. 2025-12-19. <https://www.nysenate.gov/legislation/bills/2025/S6953/amendment/B> (verified: primary) [25] "New York Finalizes RAISE Act for Frontier AI Models; Law Takes Effect January 1, 2027" (chapter amendment introduced 6 Jan 2026, passed 11 Mar 2026, signed 27 Mar 2026; effective 1 Jan 2027; DFS oversight office). Wiley. 2026. <https://www.wiley.law/alert-New-York-Finalizes-RAISE-Act-for-Frontier-AI-Models-Law-Takes-Effect-January-1-2027> (verified: secondary) [26] "AI Act: Commission publishes a reporting template for serious incidents involving general-purpose AI models with systemic risk" (template for serious-incident reporting under Art. 55; aligned to Commitment 9 of the GPAI Code). European Commission. 2025-11-04. <https://digital-strategy.ec.europa.eu/en/library/ai-act-commission-publishes-reporting-template-serious-incidents-involving-general-purpose-ai> (verified: primary) [27] ISO/IEC 42006:2025 — Requirements for bodies providing audit and certification of AI management systems (builds on ISO/IEC 17021-1; who may credibly certify to 42001). ISO/IEC. 2025. <https://www.iso.org/standard/42006> (verified: secondary) [28] ISO/IEC 23894:2023 — Guidance on AI risk management (adapts ISO 31000 to AI). ISO/IEC. 2023-02. <https://www.iso.org/standard/77304.html> (verified: secondary) [29] "Announcing the AI Agent Standards Initiative for Interoperable and Secure Innovation" (CAISI initiative; agent identity, authentication and security). NIST. 2026-02-17. <https://www.nist.gov/news-events/news/2026/02/announcing-ai-agent>

standards-initiative-interoperable-and-secure (verified: primary) [30] NIST IR 8596 (initial public draft) — Cybersecurity Framework Profile for Artificial Intelligence (Cyber AI Profile); Secure / Defend / Thwart. NIST. 2025-12-16. <https://csrc.nist.gov/pubs/ir/8596/iprd> (verified: primary) [31] NIST AI 800-1 (second public draft) — Managing Misuse Risk for Dual-Use Foundation Models (voluntary; still in draft). NIST. 2025-01. <https://www.nist.gov/news-events/news/2025/01/updated-guidelines-managing-misuse-risk-dual-use-foundation-models> (verified: primary) [32] AICM Agentic Control Supplement — proposed agent-specific controls extending the AI Controls Matrix. Cloud Security Alliance. 2026. <https://cloudsecurityalliance.org/blog/2026/04/29/securing-the-agentic-control-plane-key-progress-at-the-csai-foundation> (verified: primary) [33] AICM Catastrophic Risk Annex — enhanced AICM controls for high-autonomy systems with catastrophic-risk potential. Cloud Security Alliance. 2026-08-05. <https://cloudsecurityalliance.org/csai-foundation/catastrophic-risk-annex> (verified: primary) [34] South Korea AI Basic Act (Basic Act on the Development of AI and Establishment of Trust; in force 22 Jan 2026; high-impact AI duties). International Trade Administration (US). 2026. <https://www.trade.gov/market-intelligence/south-korea-ai-basic-act> (verified: secondary) [35] Texas Responsible Artificial Intelligence Governance Act (HB 149), enrolled text; effective 1 Jan 2026. Texas Legislature (89R). 2025. <https://capitol.texas.gov/tlodocs/89R/billtext/pdf/HB00149F.pdf> (verified: primary) [36] "Colorado AI law in flux: comprehensive replacement bill signed after federal court blocks predecessor's enforcement" (SB 24-205 delayed to 30 Jun 2026, then replaced by SB 26-189, effective 1 Jan 2027). McDermott Will & Emery. 2026. <https://www.mcdermottlaw.com/insights/colorado-ai-law-in-flux-comprehensive-replacement-bill-signed-after-federal-court-blocks-predecessors-enforcement/> (verified: secondary) [37] Data (Use and Access) Act 2025, s. 80 (replaces UK GDPR Art. 22 with Arts. 22A–22D; in force 5 Feb 2026). legislation.gov.uk. 2025. <https://www.legislation.gov.uk/ukpga/2025/18/section/80> (verified: primary) [38] "AI Security Institute" — written statement announcing the rename of the AI Safety Institute. UK Parliament. 2025-02-24. <https://questions-statements.parliament.uk/written-statements/detail/2025-02-24/hlws454> (verified: primary) [39] Model AI Governance Framework for Generative AI (voluntary). IMDA / AI Verify Foundation. 2024-05. <https://aiverifyfoundation.sg/wp-content/uploads/2024/05/Model-AI-Governance-Framework-for-Generative-AI-May-2024-1-1.pdf> (verified: primary) [40] ETSI EN 304 223 — Securing Artificial Intelligence (SAI); Baseline Cyber Security Requirements for AI Models and Systems (V2.1.1, Dec 2025; 13 principles across five lifecycle stages). ETSI. 2025-12. <https://www.etsi.org/newsroom/press-releases/2627-etsi-releases-world-leading-standard-for-securing-ai/> (verified: primary)

09. Glossary

The canonical definitions for the book: every term defined once, alphabetically, and cross-referenced to the chapter that treats it in full.

Terms are listed alphabetically. Each is one to three sentences. Where a term has a canonical spelling in the style guide (§8), that spelling is used here and everywhere in the book. A parenthetical points to the chapter that develops the term. Numbers and dates carry a [n] citation; plain definitions do not.

Agent (agentic AI). An AI system that acts — browses, executes code, calls APIs, moves data or delegates to other agents — under delegated authority, rather than only producing text. Agents are the hardest object to govern because their behaviour is emergent and their actions have external effects. (ch. 01)

Agent registry. The runtime-aware inventory of every non-human actor — model, service and agent — each with an owner, a declared scope, a status and a kill switch, fed by a runtime data path rather than typed by hand. It is the artefact that answers "what AI is running?". (ch. 04, 06)

AESIA. Spain's Agencia Española de Supervisión de la Inteligencia Artificial, based in A Coruña — the first operational national AI supervisor in the EU, with sanctioning powers since 2025 [1]. (ch. 08)

AI Act (EU). Regulation (EU) 2024/1689, the EU's horizontal, risk-tiered law for AI, amended by the Digital Omnibus [2]. It classifies systems by risk (prohibited, high-risk, limited, minimal) and imposes obligations accordingly. (ch. 08)

AI governance engineer. The person who holds the capability of AI governance engineering and is accountable for the three questions in production; a capability and a role, not necessarily a job title. (ch. 06)

AI governance engineering. The application of engineering practice — systems thinking, product thinking and code — to the governance of AI systems; measured by realised risk reduction and audit-ready evidence. (ch. 01)

AI Office. The European Commission body that supervises general-purpose AI and coordinates AI Act enforcement, with investigation powers and the ability to levy penalties on GPAI providers [2]. (ch. 08)

AI RMF functions. The four core functions of the NIST AI Risk Management Framework — **Govern, Map, Measure, Manage** — used throughout the book as a mapping target for controls [3]. (ch. 08)

AIBOM. AI bill of materials: the machine-readable inventory of an AI system's components — models, datasets, dependencies — in formats such as CycloneDX ML-BOM or the SPDX 3.0 AI profile. (ch. 04, 05)

AICM. The CSA AI Controls Matrix, a control framework (v1.1, 247 control objectives across 18 domains) that maps to ISO 42001, ISO 27001 and NIST AI RMF and underpins STAR for AI [4]. (ch. 08)

AIMA. The OWASP AI Maturity Assessment, reported at v1.0 (Aug 2025), which scores the breadth of an AI security and governance programme across domains [5]. (ch. 07)

AIMS. An AI management system: the governance structure, roles, controls and continual-improvement loop that ISO/IEC 42001 certifies. An AIMS is not the AI Act's Article 17 quality management system. (ch. 07, 08)

Annex I (EU AI Act). The AI Act annex listing the Union harmonisation legislation under which AI is embedded in regulated products (machinery, medical devices, toys and the like); obligations for these high-risk embedded systems phase in from 2 August 2028 under the Digital Omnibus timeline [2]. (ch. 08)

Annex III. The AI Act annex listing high-risk use cases (biometrics, critical infrastructure, employment, essential services, law enforcement, migration, justice); obligations for these phase in under the Digital Omnibus timeline [2]. (ch. 08)

ASI01–ASI10. The ten risks of the OWASP Top 10 for Agentic Applications 2026 — from ASI01 Agent Goal Hijack and ASI02 Tool Misuse through ASI03 Agent Identity & Privilege Abuse to ASI10 Rogue Agents — the canonical threat list for agents [6]. (ch. 05)

ATLAS. MITRE's Adversarial Threat Landscape for Artificial-Intelligence Systems, a knowledge base of adversary tactics and techniques against AI, including agent-specific techniques [7]. (ch. 05, 10)

Audit-ready evidence. Evidence emitted as a by-product of the build in a form an auditor can read directly — machine-readable, signed, timestamped — so the audit is a query, not a collection project. (ch. 01, 04)

Cedar. An open-source policy language for fine-grained authorization, used as a policy-as-code engine for runtime access decisions; a schema-typed, analysable alternative to [OPA/Rego](#). (ch. 04, 05)

CIMD. Client ID Metadata Document: the MCP mechanism (2026 spec) by which a client identifies itself via a URL-addressable metadata document, replacing deprecated Dynamic Client Registration [8]. (ch. 05)

Continuous assurance. Assurance produced continuously from telemetry rather than at a point in time; the control's status is a live query, not an annual sign-off. It is Level 5 of the maturity model. (ch. 04, 07)

Data card. Structured, versioned documentation of a dataset — provenance, lawful basis, rights, composition and known limitations — maintained as code alongside the system. (ch. 04)

Digital Omnibus. The 2026 reform package amending the EU AI Act (in force 27 Jul 2026), which adjusted the high-risk timeline, added AI Office investigation powers and reworked several articles [2]. (ch. 08)

DPIA. Data Protection Impact Assessment: the GDPR Article 35 assessment of processing likely to result in high risk to individuals, maintained in this discipline as a versioned artefact, not a one-off document. (ch. 04, 05)

Drift. The gradual divergence of a model's inputs, outputs or performance from its validated baseline over time; a runtime signal that a control or eval must catch. (ch. 04)

Duty holder. Who an obligation legally binds — under the EU AI Act, the provider, the deployer or both — as distinct from who enforces it; chapter 08 carries a duty-holder column so an engineer can tell which artefacts their organisation is responsible for producing. (ch. 08)

EN 18286. The European standard for the AI Act's Article 17 quality management system, published by CEN-CENELEC in July 2026 — the first JTC 21 AI Act standard to reach publication — but not yet cited in the Official Journal, so it confers no presumption of conformity [10]. (ch. 08)

Eval gate. A pipeline stage that fails the build when an eval fails; the mechanism that turns an evaluation into an enforced control rather than a report. (ch. 04, 05)

Evals. Automated tests of a model's or agent's behaviour — capability, safety and adversarial — run as controls, not as one-off research. (ch. 04)

Evals as evidence. The principle that the eval run *is* the assurance evidence: a failing eval blocks the build and its structured result is stored as proof the control fired. (ch. 03, 04)

Framework crosswalk. A mapping of one framework's controls onto another's; useful as an index, but a crosswalk proves you read the framework, not that the mapped control fires. (ch. 08)

FRIA. Fundamental Rights Impact Assessment: the AI Act Article 27 assessment of a high-risk system's impact on rights, maintained here as a versioned, reviewable artefact. (ch. 04, 05)

Governance-as-code. Governance rules expressed as executable code that evaluates pull requests, deployments and runtime calls and returns a decision; the umbrella term of which policy-as-code is the CI/CD subset. (ch. 03, 04)

GPAI. General-purpose AI: models that can serve many tasks, governed by dedicated AI Act obligations (transparency, copyright, and for systemic-risk models, evaluation and incident reporting) enforced by the AI Office from 2 Aug 2026 [2]. (ch. 08)

GPAI Code of Practice. The voluntary instrument (published 10 July 2025) that general-purpose-AI providers use to demonstrate compliance with their AI Act obligations until harmonised standards exist; three chapters — Transparency, Copyright, and Safety and Security (the last for systemic-risk models) [16]. (ch. 08)

Guardian agent. An AI agent whose job is to supervise, check or constrain other agents at runtime; Gartner is reported to project guardian agents as a material share of the agentic market by 2030 [9]. (ch. 04, 05)

Harmonised standard. A European standard cited in the Official Journal that, once cited, grants a presumption of conformity with a specific AI Act requirement. As of the book's date, none is yet cited [10]. (ch. 08)

ISO/IEC 42005. ISO/IEC 42005:2025, the AI system impact-assessment standard — a companion to the AI Act's Article 27 FRIA and to ISO/IEC 42001 Annex A.5 — giving a structured method for assessing an AI system's impacts on people and society [17]. (ch. 08)

Kill switch. A tested mechanism to stop an agent or system from acting; a precondition of granting autonomy, registered against the agent's identity. (ch. 03, 05)

Machine-readable evidence. Evidence a machine can query, diff and aggregate — **OSCAL** artefacts, structured eval results, signed logs — as opposed to screenshots and exported spreadsheets. (ch. 03, 04)

Market surveillance authority. The national authority designated to enforce the AI Act for products placed on its market, with powers to investigate, demand documentation and require corrective action. (ch. 08)

MCP. Model Context Protocol: an open protocol for connecting AI applications to tools and data sources; its 2026 specification adds OAuth 2.1 resource-server patterns and issuer-bound credentials for agent authorisation [8]. (ch. 05)

Model card. Structured, versioned documentation of a model — provenance, intended use, capabilities, evaluations and known failure modes — maintained as code. (ch. 04)

Model risk management. The SR 11-7-tradition practice of validating models for conceptual soundness and back-testing them; a neighbour of this discipline, extended here to runtime behaviour and agents. (ch. 01)

NHI. Non-human identity: the identity of an agent, service account or machine actor. Every NHI gets a registry entry, an owner and a scope before it is allowed to act. (ch. 04, 05)

OPA/Rego. The Open Policy Agent and its Rego policy language, a general-purpose policy-as-code engine that evaluates governance rules in CI/CD and at runtime admission; the canonical example of executable policy-as-code. (ch. 04, 05)

OSCAL. The Open Security Controls Assessment Language, a NIST machine-readable format for controls, assessments and evidence, used here as the format for audit-ready evidence [3]. (ch. 04, 10)

Paved path. A supported, low-friction default route — a template, library or pipeline — that makes the governed way the easiest way to ship, so engineers adopt governance without asking permission. (ch. 03, 06)

Policy Card. A JSON-schema, machine-readable governance artefact that declares an agent's allowed and forbidden behaviours for runtime enforcement [11]. (ch. 04, 10)

Policy-as-code. Governance policy expressed in an executable policy language (OPA/Rego, Cedar) that evaluates in CI/CD and at admission; the narrower, pipeline subset of governance-as-code. (ch. 04, 05)

Post-market monitoring. The AI Act Article 72 duty to actively monitor a high-risk system's performance and risks after deployment, throughout its lifetime [2]. (ch. 08)

Presumption of conformity. The legal effect by which conforming to a cited harmonised standard is taken as meeting the corresponding AI Act requirement; unavailable until a standard is OJ-cited [10]. (ch. 08)

QMS (Art. 17). The quality management system that AI Act Article 17 requires of high-risk providers; distinct from an ISO/IEC 42001 AIMS, which certifies a management system but is not harmonised [2][10]. (ch. 08)

RAISE Act. New York's Responsible AI Safety and Education Act, a frontier-AI safety law binding large frontier developers to publish a safety framework and disclose incidents; signed 19 December 2025 and taking effect 1 January 2027 after a March 2026 chapter amendment that placed oversight in an office within the Department of Financial Services (DFS) [12][15]. (ch. 08)

Realised risk reduction. The measured drop in a named failure mode's rate or blast radius in production — one of the two tests of the discipline, against framework coverage. (ch. 01, 03)

Red teaming. Structured adversarial testing of a model or agent to elicit failures (jailbreaks, injection, tool misuse) before an attacker does; treated here as an evidence-producing control. (ch. 04, 05)

Runtime data path. The live connection between production and the governance function — discovery, telemetry and enforcement — without which a registry or dashboard describes the program but cannot see what is running [13]. (ch. 04, 07)

SB 53. California's frontier-AI transparency law (TFAIA), in force 1 Jan 2026, covering large frontier developers training models above 10^{26} FLOP, with transparency and safety-framework obligations [14]. (ch. 08)

Serious incident. Under the AI Act, an incident causing death, serious harm to health, fundamental rights or property, or serious disruption of critical infrastructure, triggering Article 73 reporting duties [2]. (ch. 04, 08)

Shadow AI. An AI system, model or agent running in production without registering — the failure mode that makes an inventory complete only for the honest. (ch. 07)

STAR for AI. CSA's security assurance and certification programme for AI, built on the AICM, with a self-assessment tier, an automated "Valid-AI-ted" tier and a Level 2 combining ISO/IEC 42001 with the validated assessment [4]. (ch. 07, 08)

Systemic risk. Under the AI Act, the risk posed by the most capable general-purpose AI models, triggering extra evaluation, adversarial-testing and incident-reporting duties on their providers [2]. (ch. 08)

Sources

[1] AESIA (first operational national AI supervisor; sanctioning powers 2025). Research digest / AESIA. 2026. <https://www.aesia.gob.es/> (verified: secondary) [2] EU AI Act + Digital Omnibus (Regulation (EU) 2026/1744, in force 27 Jul 2026; Arts. 17, 27, 50, 55, 72, 73; Annex III). AI Act Explorer. 2026. <https://artificialintelligenceact.eu/ai-act-explorer/digital-omnibus/> (verified: primary) [3] NIST AI Risk Management Framework 1.0 (Govern, Map, Measure,

Manage); OSCAL. NIST. 2023. <https://www.nist.gov/itl/ai-risk-management-framework> (verified: primary) [4] AI Controls Matrix v1.1 (247 control objectives, 18 domains) and STAR for AI. Cloud Security Alliance. 2026-06-22. <https://cloudsecurityalliance.org/star/ai> (verified: primary) [5] OWASP AI Maturity Assessment (AIMA), reported at v1.0 (Aug 2025). OWASP GenAI Security Project. 2025. <https://genai.owasp.org/initiatives/> (verified: reported) [6] Top 10 for Agentic Applications 2026 (ASI01–ASI10). OWASP GenAI Security Project. 2025-12-09. <https://genai.owasp.org/resource/owasp-top-10-for-agentic-applications-for-2026/> (verified: primary) [7] MITRE ATLAS (adversarial threat knowledge base for AI). MITRE. 2026. <https://atlas.mitre.org/> (verified: primary) [8] Model Context Protocol specification 2026-07-28 (OAuth 2.1 resource servers; Client ID Metadata Documents; issuer-bound credentials). Anthropic / MCP. 2026-07-28. <https://blog.modelcontextprotocol.io/posts/2026-07-28/> (verified: primary) [9] Guardian agents reported as a material share (reported figure: 10-15%) of the agentic AI market by 2030 — a separate Gartner guardian-agent prediction; the figure is not stated in the June 2025 project-cancellation release. Gartner (reported). 2025. <https://www.gartner.com/en/newsroom> (verified: reported) [10] No harmonised standard cited in the OJ → no presumption of conformity; ISO/IEC 42001 not harmonised. JTC 21 standards tracker. 2026. <https://kla.digital/blog/jtc-21-standards-tracker> (verified: secondary) [11] Policy Cards: machine-readable runtime governance artefacts for agents. arXiv 2510.24383. 2025-10. <https://arxiv.org/abs/2510.24383> (verified: primary) [12] "Governor Hochul Signs Nation-Leading Legislation to Require AI Frameworks for AI Frontier Models" (RAISE Act, S6953B/A6453B, signed 19 Dec 2025; oversight office within the Department of Financial Services). Governor Kathy Hochul (New York State). 2025-12-19. <https://www.governor.ny.gov/news/governor-hochul-signs-nation-leading-legislation-require-ai-frameworks-ai-frontier-models> (verified: primary) [13] Runtime data path critique of the AI-governance platform category. Kosmoy. 2026. <https://www.kosmoy.com/resources/blog/best-ai-governance-platforms-2026/> (verified: secondary) [14] California SB 53 / TFAIA (in force 1 Jan 2026; models above 10²⁶ FLOP; large frontier developers). Future of Privacy Forum. 2026. <https://fpf.org/blog/californias-sb-53-the-first-frontier-ai-law-explained/> (verified: secondary) [15] "New York Finalizes RAISE Act for Frontier AI Models; Law Takes Effect January 1, 2027" (chapter amendment signed 27 Mar 2026; effective 1 Jan 2027; DFS oversight office). Wiley. 2026. <https://www.wiley.law/alert-New-York-Finalizes-RAISE-Act-for-Frontier-AI-Models-Law-Takes-Effect-January-1-2027> (verified: secondary) [16] GPAI Code of Practice (published 10 Jul 2025; voluntary; three chapters: Transparency, Copyright, Safety and Security). AI Act Explorer / European Commission. 2025-07-10. <https://artificialintelligenceact.eu/introduction-to-code-of-practice/> (verified: primary) [17] ISO/IEC 42005:2025 — AI system impact assessment (companion to Art. 27 and ISO 42001 Annex A.5). ISO/IEC. 2025-05. <https://www.iso.org/standard/44545.html> (verified: secondary)

10. Reading list

The sources that formed the discipline, curated and annotated — each with a verified URL and a one-line note on why it matters.

This is a working bibliography, not a canon. Entries are grouped by theme and annotated in one line. URLs are given inline with a verification tag (`primary` , `secondary` , `reported`) so the chapter is self-documenting; every URL is either drawn from the book's research digest or verified for this edition. Tools are named as category examples, illustrative and not endorsements.

Foundational texts (the form and the method)

- **GRC Engineering Manifesto** — the parent discipline's founding statement; the structural and philosophical model for this book. <https://grc.engineering/> (verified: primary)
- **"What is GRC Engineering" (Ayoub Fandi)** — the clearest definition of the parent method and the source of the "green dashboard over a broken control is theatre" test. <https://grcengineer.com/what-is-grc-engineering/> (verified: primary)
- **The Agile Manifesto** — the numbered values-and-principles form and the signatory model this book borrows. <https://agilemanifesto.org/> (verified: primary)
- **The Twelve-Factor App** — the template for a numbered, practitioner-facing body of practice with a "who should read this" framing. <https://12factor.net/> (verified: primary)
- **CSIRO Responsible AI Pattern Catalogue** — the pattern template (chapter 05) and proof that responsible-AI practice can be written as reusable patterns. <https://research.csiro.au/ss/science/projects/responsible-ai-pattern-catalogue/> (verified: primary)
- **privacypatterns.org** — the precedent for translating a legal principle (privacy by design) into engineering patterns under CC BY. <https://privacypatterns.org/> (verified: primary)

Regulation and standards

- **EU AI Act + Digital Omnibus explorer** — the consolidated, navigable text of the Act as amended; the primary obligation source for chapter 08. <https://artificialintelligenceact.eu/ai-act-explorer/digital-omnibus/> (verified: primary)
- **GPAI Code of Practice** — the Commission's code for general-purpose AI, including the safety and security chapter that requires model evaluations. <https://digital-strategy.ec.europa.eu/en/policies/contents-code-gpai> (verified: primary)
- **"ISO/IEC 42001 and the AI Act: why certification is not yet a presumption of conformity"** — the key relationship between the AIMS standard (and 42005/42006) and the Act. <https://lawandtechnology.eu/en/iso-iec-42001-and-the-ai-act-why-certification-is-not-yet-a-presumption-of-conformity/> (verified: secondary)
- **JTC 21 harmonised-standards tracker** — the live status of the European standards that would grant a presumption of conformity; as of the book's date, none is OJ-cited. <https://kla.digital/blog/jtc-21-standards-tracker> (verified: secondary)
- **NIST AI Risk Management Framework 1.0** — the Govern/Map/Measure/Manage functions used as a mapping target throughout. <https://www.nist.gov/itl/ai-risk-management-framework> (verified: primary)
- **NIST NCCoE, "Software and AI Agent Identity and Authorization" (concept paper)** — the emerging reference for non-human identity, the precondition of agent governance. <https://www.nccoe.nist.gov/news-insights/new-concept-paper-identity-and-authority-software-agents> (verified: primary)
- **NIST CAISI AI Agent Standards Initiative** — the effort to make agent interoperability and security standard, not per-vendor. <https://www.nist.gov/news-events/news/2026/02/announcing-ai-agent-standards-initiative-interoperable-and-secure> (verified: primary)
- **OWASP GenAI Security Project** — the home of the Top 10 for LLM Applications, the Top 10 for Agentic Applications, the AIBOM project and the AI Maturity Assessment. <https://genai.owasp.org/> (verified: primary)
- **CSA AI Controls Matrix and STAR for AI** — the control framework and assurance programme mapped to ISO 42001 and NIST AI RMF (chapters 07-08). <https://cloudsecurityalliance.org/star/ai> (verified: primary)

- **MITRE ATLAS** — the adversarial tactics-and-techniques knowledge base for AI, including agent techniques, that threat models draw on. <https://atlas.mitre.org/> (verified: primary)
- **NSA CSI, "MCP: Security Design Considerations"** — government guidance on securing the Model Context Protocol that connects agents to tools. <https://www.nsa.gov/Press-Room/Press-Releases-Statements/Press-Release-View/Article/4496698/> (verified: primary)
- **"California's SB 53: the first frontier-AI law explained" (FPF)** — the clearest read on California's SB 53 (TFAIA), the first US frontier-AI transparency law. <https://fpf.org/blog/californias-sb-53-the-first-frontier-ai-law-explained/> (verified: secondary)
- **New York RAISE Act — Governor's signing announcement** — New York's frontier-AI safety law (S6953B), signed 19 December 2025 and effective 1 January 2027 after a March 2026 chapter amendment, with an oversight office in the Department of Financial Services. <https://www.governor.ny.gov/news/governor-hochul-signs-nation-leading-legislation-require-ai-frameworks-ai-frontier-models> (verified: primary)
- **NIST AI 800-1, "Managing Misuse Risk for Dual-Use Foundation Models" (second public draft)** — US NIST/CAISI voluntary guidance on identifying, measuring and mitigating misuse risk across the model lifecycle; still a draft. <https://www.nist.gov/news-events/news/2025/01/updated-guidelines-managing-misuse-risk-dual-use-foundation-models> (verified: primary)
- **NIST ARIA (Assessing Risks and Impacts of AI)** — NIST's evaluation environment that tests model risks and impacts through model testing, red teaming and field testing; a reference for layer 03. <https://ai-challenges.nist.gov/aria> (verified: primary)
- **UK AI Security Institute** — the UK government institute that evaluates advanced-AI risks and publishes the Inspect eval framework this book uses as the layer-03 reference. <https://www.aisi.gov.uk/> (verified: primary)
- **US Center for AI Standards and Innovation (CAISI)** — NIST's center for AI standards, testing and security, formerly the US AI Safety Institute; the US government's point of contact for industry. <https://www.nist.gov/caisi> (verified: primary)

Frontier safety frameworks (the labs' own commitments)

- **Anthropic Responsible Scaling Policy** — Anthropic's own frontier-safety framework of capability thresholds and required safeguards; the page states version 3.4, effective 8 July 2026. <https://www.anthropic.com/responsible-scaling-policy> (verified: primary)
- **OpenAI Preparedness Framework** — OpenAI's own framework for tracking and preparing for frontier capabilities that could cause severe harm; the document states Version 2, last updated 15 April
<https://cdn.openai.com/pdf/18a02b5d-6b67-4cec-ab64-68cdfbddebcd/preparedness-framework-v2.pdf> (verified: primary)
- **Google DeepMind Frontier Safety Framework** — DeepMind's own framework of critical capability levels and mitigations; the page states version 3.1, updated 17 April 2026.
<https://deepmind.google/discover/blog/strengthening-our-frontier-safety-framework/> (verified: primary)

Papers (machine-readable evidence and agent governance)

- **"Making AI Compliance Evidence Machine-Readable" (arXiv 2604.13767)** — extends OSCAL for AI and argues frameworks specify *what* to assure but no executable *how*. <https://arxiv.org/html/2604.13767v1> (verified: primary)
- **"Audit-as-code" (Frontiers in AI)** — an assured-readiness score with proceed/remediate/block gates; audit output as a build artefact. <https://pubmed.ncbi.nlm.nih.gov/41837238/> (verified: primary)
- **Policy Cards (arXiv 2510.24383)** — JSON-schema, machine-readable runtime governance artefacts for agents. <https://arxiv.org/abs/2510.24383> (verified: primary)
- **TAIP (arXiv 2603.03340)** — treats NIST TEVV outputs as AI assurance objects. <https://arxiv.org/abs/2603.03340> (verified: primary)
- **AI Trust OS (arXiv 2604.04749)** — an operating-system framing for continuous AI trust and assurance. <https://arxiv.org/abs/2604.04749> (verified: primary)
- **AAGATE (arXiv 2510.25863)** — a NIST AI RMF-aligned agent governance platform design. <https://arxiv.org/abs/2510.25863> (verified: primary)

Reports (the market and the profession)

- **IAPP AI Governance Profession Report 2025 (with Credo AI)** — where the function sits and how it is staffed; the profession's baseline census. <https://iapp.org/resources/article/ai-governance-profession-report/> (verified: primary)
- **IAPP Salary & Jobs Report 2025-26** — the salary bands that anchor chapter 06, including the technical-AI-governance premium. <https://iapp.org/resources/article/salary-survey-summary/> (verified: primary)
- **IAPP AI Governance Vendor Report 2026** — the four vendor categories and the claim that AI governance "is not a single function, discipline or technology". <https://iapp.org/resources/article/ai-governance-vendor-report> (verified: primary)
- **State of GRC 2026** — the practitioner survey behind the "spreadsheet is still the #1 GRC tool" reality the discipline reacts against. <https://grcengineer.com/report/> (verified: primary)
- **Gartner Magic Quadrant for AI Governance Platforms 2026 (via IBM)** — the first MQ for the category and its inclusion criteria (discovery, registry, policy, evidence). <https://www.ibm.com/new/announcements/ibm-recognized-as-a-leader-in-gartner-magic-quadrant-for-ai-governance-platforms> (verified: secondary)
- **HiddenLayer Threat Report 2026** — the source of the reported "~1 in 8 AI breaches involve autonomous agents" figure; read as reported. <https://www.hiddenlayer.com/report-and-guide/threatreport2026> (verified: reported)

Incident and risk repositories (the empirical record)

- **AI Incident Database** — the Responsible AI Collaborative's catalogue of real-world AI harms and near-harms; a reference corpus for threat modelling and post-market monitoring. <https://incidentdatabase.ai/> (verified: primary)
- **AIAAIC Repository** — an independent, open register of AI, algorithmic and automation incidents and controversies across sectors. <https://www.aiaaic.org/aiaaic-repository> (verified: primary)
- **OECD.AI Incidents & Hazards Monitor (AIM)** — the OECD's live monitor of AI incidents and hazards drawn from global media, built to inform policy. <https://oecd.ai/en/incidents> (verified: primary)
- **AI Vulnerability Database (AVID)** — the AI Risk and Vulnerability Alliance's open knowledge base of failure modes for general-purpose AI systems. <https://avidml.org/> (verified: primary)
- **MIT AI Risk Repository** — MIT FutureTech's living database of 1,700+ AI risks classified across 65 frameworks; a taxonomy for intake and risk tiers. <https://airisk.mit.edu/> (verified: primary)

Tools (illustrative categories, not endorsements)

- **Inspect AI (UK AI Security Institute)** — an open eval framework, the reference example for evals-as-evidence and eval gates. https://github.com/UKGovernmentBEIS/inspect_ai (verified: primary)
- **Awesome Responsible AI (AthenaCore)** — a broad, maintained index of responsible-, trustworthy- and human-centred-AI resources spanning every layer of the stack. <https://github.com/AthenaCore/AwesomeResponsibleAI> (verified: primary)
- **Awesome Production Machine Learning (EthicalML)** — the reference index of open-source libraries to deploy, monitor, version and scale ML; the tooling map for layers 02 and 04. <https://github.com/EthicalML/awesome-production-machine-learning> (verified: primary)
- **awesome-opa (Open Policy Agent)** — the OPA ecosystem's own index of policy-as-code tools and integrations for layer 01. <https://github.com/open-policy-agent/awesome-opa> (verified: primary)
- **Awesome OSCAL (OSCAL Club)** — a curated index of OSCAL tools and resources for the machine-readable evidence of layer 05. <https://github.com/oscal-club/awesome-oscal> (verified: primary)
- **awesome-ml-security (Trail of Bits)** — a curated index of machine-learning security resources feeding layer 03 red teaming and layer 04 runtime controls. <https://github.com/trailofbits/awesome-ml-security> (verified: primary)

Communities and newsletters

- **GRC Engineer (grcengineer.com)** — the parent community's hub; the analyst-vs-engineer framing and the role definitions this book adapts. <https://grcengineer.com/> (verified: primary)

- **blog.grc.engineering, "GRC Engineering in 2026" (Justin Pagano)** — the forward view: policy-as- code guardrails in CI/CD, trust operations centres, agentic extensions. <https://blog.grc.engineering/p/grc-engineering-in-2026> (verified: primary)
- **IAPP (iapp.org)** — the professional body whose reports, certifications and events map the AI governance profession. <https://iapp.org/> (verified: primary)

Maps to: this chapter makes no normative claim; the standards and frameworks it lists are treated in full in chapters 04, 05, 07 and 08. Mappings elsewhere in the book are illustrative, not a claim of conformity.

Sources

The reading list is its own source set: each entry above carries its URL and a verification tag inline, and every URL is recorded as a row in this chapter's section of `sources/SOURCES.md`. Items whose URL could not be drawn from the research digest or verified for this edition were left out — most notably the International AI Safety Report 2026, cited elsewhere in the digest but without a URL that resolved at the time of writing.

Contributors and signatories

Initial author

Jorge García Aibar — AI Governance & Privacy Engineer. Author of version 0.1; editor of version 0.2 (peer-review round 1 revisions). LinkedIn: <https://www.linkedin.com/in/jorgara>

Version 0.1 was written by one practitioner as a first public draft; version 0.2 applies the findings of a first peer-review round. It is meant to be co-authored. Home: <https://aigovernanceengineer.com> · source repository github.com/losanchos5/aige .

How to become a co-author

Co-authors write or substantially revise a chapter, a pattern, or the Thesis, and are credited by name. To co-author:

1. Read `STYLEGUIDE.md` . Chapters 00–10 are drafted in v0.1; co-authors extend and sharpen them, add patterns to `bok/05-patterns.md` , or propose new chapters from `OUTLINE.md` .
2. Write it to the template. Every factual claim needs a `[n]` citation tagged `primary` , `secondary` or `reported` , with a matching row in `SOURCES/SOURCES.md` under that chapter's section.
3. Open a pull request. Add yourself to the **Co-authors** list below in the same PR.
4. Keep the voice practitioner-direct, name categories rather than vendors, and never invent a figure. Respect the BRIEF blocklist recorded in `STYLEGUIDE.md` §7.

Substantive fixes — a corrected fact, a sharper argument, a better source — are also credited, in the **Contributors** list.

Co-authors

Aurélié Pols — Responsible AI (EU/Global), Privacy & Data Governance. Co-author of the Thesis and Body of Knowledge (v0.1–v0.4.0). LinkedIn: <https://www.linkedin.com/in/aureliepols>

Contributors

None yet. This section lists people who have made substantive corrections or additions.

Signatories

Signing the Thesis means you endorse *The AI Governance Engineering Thesis* — its definition, values and principles. Signatories are practitioners and organisations who support the discipline; signing does not require writing a chapter.

To sign: open a pull request that adds a line to the list below, in the form:

- Name – role / organisation (optional) – LinkedIn or GitHub link (optional)

Add yourself at the end of the list. Keep it to one line. By signing you confirm you are adding your own name and that the entry contains nothing confidential.

Signatories

- Jorge García Aibar — AI Governance & Privacy Engineer — <https://www.linkedin.com/in/jorgara>

Licence

This work is licensed under **CC BY 4.0**. It may be freely copied and shared, but only in its entirety and including this notice, so that it continues to identify *AI Governance Engineering: The Thesis & Body of Knowledge* as the source.

Attribution: Jorge García Aibar and contributors.

Changelog

All notable changes to *AI Governance Engineering: The Thesis & Body of Knowledge* are recorded here. Versioning is semantic in spirit: patch = fact/typo fixes, minor = new chapters or patterns, major = a completed, reviewed core (1.0).

[0.4.0] — 2026-09-19

Minor release: two new patterns, a fuller regulatory map, curated resources, and a visual layer for the whole book. No change to the definition, values or principles.

Added

- `bok/05-patterns.md` — two patterns the outline had promised: **Adversarial Red-Team Suite** (Layer 03) and **Runtime Guardrail** (Layer 04, distinct from Kill Switch / Circuit Breaker). The catalogue now has 17 patterns; `OUTLINE.md` updated. Four illustrative JSON schemas (Policy Card verdict, Eval Gate result, Continuous Assurance evidence record, Agent Registry entry) placed where the prose already specified their fields.
- `bok/04-the-stack.md` — one named, anonymised system carried through the five "In practice" boxes and a short "One system through the five layers" section after the minimum viable stack; bold pattern names now link to the catalogue.
- `bok/08-regulatory-map.md`, `site/src/data/frameworks.ts` — EU AI Act rows for Arts. 6, 43, 47 and 60; the Commission's Art. 55(1)(c) serious-incident template as the named artefact on the Art. 55 row; the Art. 73 windows as a standalone decision table; ISO/IEC 42006:2025 and ISO/IEC 23894; NIST AI Agent Standards Initiative, NIST IR 8596 and NIST AI 800-1 (drafts flagged); CSA AICM supplements; South Korea AI Basic Act, Texas TRAI GA, Colorado, a UK subsection and Singapore's IMDA framework, each dated "as of 2026-09-19" and hedged where rules are still in draft.
- `bok/10-reading-list.md`, `sources/SOURCES.md` — incident and risk repositories (AIID, AIAAIC, OECD.AI, AVID, MIT AI Risk Repository), NIST AI 800-1 and ARIA, UK AISI, US CAISI, and the three frontier-lab safety frameworks; the tools index rebuilt on five verified curated lists.
- `site/src/data/stack.ts`, `site/src/data/path.ts` — illustrative tool names per layer (incl. new categories for MCP / tool-call security, kill switches, model and data cards, FRIA/DPIA tooling and open-source OSCAL tooling) and learning-path resources (IAPP AIGP, Stanford CS120, hands-on labs, a first `template` resource), every URL fetched on 2026-09-19.
- Figures — an archify figure at the head of every pattern (17), openers for chapters 02 and 04, and seven hand-drawn infographics (the three questions, values and principles, the minimum viable stack, the maturity grid, the Art. 73 clock, the pattern map, reading paths by persona), all governed by `site/VISUAL-GUIDE.md`.
- Site — duty-holder and applies-from columns with a duty-holder filter on the obligation table; "Patterns" in the navigation with cross-links between tools, obligations and patterns; CSV/JSON export of the obligation index; a mobile table of contents; pattern links from the maturity model; accessibility and Lighthouse gates in CI; a weekly external link-rot check; opt-in cookieless analytics; `CONTRIBUTING.md`.

Removed

- `github.com/systempromptio/awesome-ai-agent-governance` — cited five times as a curated index; on inspection it is a promotional fork-farm list (64 forks for 41 stars, one-commit contributors each adding their own tool). Replaced by AthenaCore/AwesomeResponsibleAI, EthicalML/awesome-production-machine-learning, open-policy-agent/awesome-opa, oscal-club/awesome-oscail and trailofbits/awesome-ml-security.

[0.3.1] — 2026-09-19

Patch release: a credibility pass. Facts corrected and re-sourced, co-authorship recorded, and the version single-sourced. No change to the definition, values or principles.

Changed

- `THESIS.md`, `bok/00-preface.md`, `bok/CONTRIBUTORS.md`, `site/src/data/site.ts`, `site/src/components/Citation.astro`, `site/src/pages/about/index.astro` — Aurélie Pols added as co-author (matching the Thesis Authors section and LinkedIn); the suggested citation now lists both authors. "written by one practitioner" reworded to "started by one practitioner".

- `site/src/data/site.ts` and everywhere the current version is stated — bumped to **v0.3.1**; the version is now single-sourced from `bokVersion`, with prose pointing at this changelog.
- `OUTLINE.md`, `STYLEGUIDE.md`, `sources/SOURCES.md` — all chapters 00–10 marked ****[drafted]****; dead `posts/...` and `RESEARCH-DIGEST.md` pointers removed; the chapter-04 brief lists its five extra sections and its word target updated to ~5,000.
- `THESIS.md`, `bok/CONTRIBUTORS.md`, `README.md` — the placeholder repository URL replaced with the live `github.com/losanchos5/aige`; `README.md` links `CONTRIBUTING.md`.
- `build/build_pdf.py` — PDF output filenames now read the version from `site/src/data/site.ts`, so they carry v0.3.1.
- `site/scripts/content-lint.mjs` — new rule: fail the build when a current-version statement in `THESIS.md`, `README.md`, `bok/00-preface.md` OR `OUTLINE.md` disagrees with `bokVersion`.

Fixed (fact-check)

- `bok/08-regulatory-map.md`, `bok/09-glossary.md`, `bok/10-reading-list.md`, `site/src/data/frameworks.ts`, `sources/SOURCES.md` — the New York RAISE Act status corrected: it is enacted (S6953B, signed 19 Dec 2025), effective 1 Jan 2027 after a March 2026 chapter amendment, with oversight in an office within the NY Department of Financial Services. The earlier provisional-status hedge is gone; retagged to primary/secondary and cited to the Governor's release, the NY Senate bill and a law-firm alert.
- `bok/02-why-now.md`, `sources/SOURCES.md` — citation [9], which attributed three job postings to one URL, split into three per-posting citations; downstream markers renumbered.
- `bok/06-the-role.md`, `site/src/data/role.ts`, `sources/SOURCES.md` — the unsourced LinkedIn "+150% YoY" figure removed (it is not in the cited article, nor stated per-skill in a LinkedIn primary); the qualitative demand signal kept.
- `bok/02-why-now.md`, `bok/08-regulatory-map.md` — "as of 2026-09-10" currency markers re-verified and moved to "as of 2026-09-19" (no harmonised standard is yet OJ-cited).

Added

- `bok/09-glossary.md` — seven glossary terms: OPA/Rego, Cedar, GPAI Code of Practice, ISO/IEC 42005, Annex I (EU AI Act), Duty holder, EN 18286.
- `bok/08-regulatory-map.md`, `site/src/data/frameworks.ts` — an EU AI Act **Art. 25** (value-chain responsibilities) row, mirroring the pattern in chapter 05.

[0.3] — 2026-09-15

Editorial pass. The founding document is retitled and its values restated; the substance of the discipline is unchanged.

Changed

- `THESIS.md` — the founding document is now **The AI Governance Engineering Thesis** ("the Thesis"), and the work is *AI Governance Engineering: The Thesis & Body of Knowledge*. Its file is `THESIS.md` and its canonical route is `/thesis`. Bumped to v0.3.
- `THESIS.md`, `bok/03-values-principles.md` — the eight values drop the Agile "X over Y" grammar and are stated as affirmations (e.g. "Governance is code, not a document"); their bodies, anti-patterns and citations are unchanged. The GRC engineering precedent is folded into a short list of influences (SRE, DevSecOps, policy-as-code, software supply-chain security) and no longer leads the document.
- `bok/00-preface.md`, `bok/02-why-now.md`, `bok/04-the-stack.md`, `bok/06-the-role.md`, `bok/10-reading-list.md`, `bok/CONTRIBUTORS.md`, `README.md`, `OUTLINE.md`, `STYLEGUIDE.md`, `sources/SOURCES.md`, `build/build_pdf.py` — refer to the document as "the Thesis" throughout; external work titles (GRC Engineering Manifesto, The Agile Manifesto) are left intact.

[0.2] — 2026-09-10

Peer-review revision. Applies the HIGH and MEDIUM findings of **peer review round 1** (a senior- practitioner review and an adversarial fact-check), plus the cheap LOW findings.

Changed

- `bok/03-values-principles.md`, `THESIS.md` — reworked the six principles into strict commitments to action with no lexical overlap with the eight values (principles renamed accordingly); reframed value 6 from "Practitioner-built open tooling over closed platforms" to "**Inspectable, composable tooling** over black boxes"; noted which values are inherited from GRC engineering.
- `bok/01-definition.md` — added "the limits of the eval gate" (evals necessary, not sufficient; point-in-time, Goodhartable, blind to novelty); strengthened the AI-security-engineering boundary (the deliverable is a governed, evidenced system; overlap is a feature).
- `THESIS.md`, `bok/01-definition.md`, `bok/04-the-stack.md` — conceded that three of the five layers are inherited from GRC engineering and two (evals-as-controls, agent identity/runtime) are what AI forces us to add.
- `bok/04-the-stack.md` — distinguished MCP channel authentication from agent workload identity; presented Policy Cards / the OSCAL-extension preprint / TAIP / AAGATE as "one proposed approach" and named OSCAL's native model as the stable substrate; added Cedar-vs-Rego nuance; added sections on data governance, designing human oversight (Art. 14), third-party and procured AI, and the cost of the stack (FinOps).
- `bok/05-patterns.md` — added the pattern **Vendor / Model Due-Diligence Gate** (CSIRO template); sharpened Agent Identity (channel vs workload identity); tightened Continuous Assurance Telemetry to a concrete evidence schema; hedged the OSCAL-extension and TAIP/AAGATE proposals (now 14 patterns).
- `bok/06-the-role.md` — softened the analyst-vs-engineer table (cadence and artefact, not competence or access); moved named job postings to a footnote, keeping the IAPP bands and skills-demand data.
- `bok/07-maturity-model.md` — added a Level 4 eval-quality criterion (suite coverage / adversarial quality assessed, not just the gate's existence) and a "partial maturity is normal" reading.
- `bok/08-regulatory-map.md` — added a **Duty holder** column (provider / deployer / both); aligned the FRIA layer placement with chapter 05.

Fixed (fact-check round 1)

- `bok/09-glossary.md` — RAISE Act date softened to a reported 1 Jan 2027, pending confirmation of enactment; guardian-agent figure decoupled from the June 2025 project-cancellation release (now a separate Gartner prediction, reported).
- `bok/08-regulatory-map.md` — Regulation (EU) 2026/1744 verified on EUR-Lex and cited as primary.
- `bok/04-the-stack.md` — GPAI Code of Practice source date corrected to 10 Jul 2025.
- `bok/05-patterns.md` — TAIP arXiv date corrected to 2026-02 (submitted 15 Feb 2026).
- OWASP Agentic ASI titles left as drafted; could not be re-confirmed verbatim from the resource page (the enumerated list is in the downloadable PDF, and the web-search budget was exhausted).

Added

- `THESIS.md` — a second, independent source (IBM/Gartner Magic Quadrant, 2026) hedging the Kosmoy "no runtime data path" thesis.

[0.1] — 2026-09

First public draft. Founding release.

Added

- `THESIS.md` — the founding statement: definition and "more than X" clarifier; five fundamental problems with legacy AI governance; eight values (Agile grammar); six principles; what AI governance engineers build; authors and "co-authors wanted"; sign / get involved; CC BY 4.0 licence.
- `STYLEGUIDE.md` — voice and formatting rules; chapter template; CSIRO pattern template; citation format with `primary` / `secondary` / `reported` tags; canonical terminology; words to avoid; handling of unverified figures.
- `OUTLINE.md` — full table of contents (chapters 00–10) with a per-chapter brief.
- `bok/00-preface.md` — provenance, audience, what it is not, how to cite, versioning, contributing.
- `bok/01-definition.md` — the definition, three clarifiers, the eight-neighbour disambiguation cluster (table + prose), the object of governance, the three questions.
- `bok/02-why-now.md` — the five problems with the evidence; profession, market, standards-gap, regulatory-wave and agent-shift signals; what changes when governance is engineered.

- `bok/03-values-principles.md` — the eight values and six principles, each expanded with an "In practice" example and an anti-pattern.
- `bok/04-the-stack.md` — the five-layer reference architecture, per layer (what it proves, artefacts, illustrative tools, definition of done, anti-patterns, evidence flow), plus the minimum viable stack.
- `bok/05-patterns.md` — the pattern catalogue: 13 CSIRO-template patterns, each named to a layer.
- `bok/06-the-role.md` — the role by workflow; skills; analyst-vs-engineer; career ladder; entry paths; the market; common JD mistakes.
- `bok/07-maturity-model.md` — the five levels (Documented → Continuous), observable criteria by layer, metrics, self-assessment checklist, relation to ISO 42001 / AIMA / STAR for AI.
- `bok/08-regulatory-map.md` — the reverse index: EU AI Act (post-Omnibus), GPAI Code of Practice, ISO/IEC 42001 and 42005, NIST AI RMF, CSA AICM, OWASP GenAI, US frontier laws, and what is not harmonised yet.
- `bok/09-glossary.md` — canonical definitions, alphabetical, cross-referenced to chapters.
- `bok/10-reading-list.md` — annotated bibliography grouped by theme, each with a verified URL.
- `README.md` , `bok/CONTRIBUTORS.md` (authors + signatories), `bok/CHANGELOG.md` .
- `sources/SOURCES.md` — consolidated verified source table, one section per chapter.
- `build/build_pdf.py` — assembles the Markdown into `dist/site-preview.html` (PDF export available but not shipped).

Notes

- This is a first public draft: chapters 00–10 are drafted, and the catalogue, mappings and arguments remain open for co-authors to extend and sharpen.
- All facts are current as of 2026-09-10 and carry verified citations. The regulatory and standards landscape (EU AI Act Digital Omnibus, CEN-CENELEC JTC 21 harmonised standards, OWASP and CSA releases) is expected to move; chapters will be revised.